

Universitatea Națională de Știință și Tehnologie
POLITEHNICA București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației

HomeForge. Platformă pentru automatizare rezidențială și experimentare IoT

Proiect de diplomă

prezentat ca cerință parțială pentru obținerea titlului de
Inginer în domeniul Electronică, Telecomunicații și Tehnologia Informației
programul de studii de licență *Rețele și software de telecomunicații (RST)*

Conducător științific
Ș.L. Dr. Ing. Laurențiu Boicescu

Absolvent
Olteanu Rareș-Cristian

Anul 2026

Universitatea Națională de Știință și Tehnologie POLITEHNICA București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației
Program de studiu **RST**

Anexa 1

TEMA PROIECTULUI DE DIPLOMĂ
a studentului **OLTEANU C. Rareș-Cristian, 444D**

1. Titlul temei: HomeForge. Platformă pentru automatizare rezidențială și experimentare IoT

2. Descrierea temei:

- Proiectul își propune proiectarea și implementarea unei platforme IoT destinate automatizării rezidențiale.
- Sistemul va fi conceput ca un ecosistem modular care facilitează interacțiunea dintre dispozitive hardware personalizate (construite pe baza microcontrolerelor cu conectivitate Wi-Fi) și utilizatorul final, prin intermediul unei interfețe web. Scopul principal este oferirea unui mediu de tip „sandbox” în care studenții și pasionații pot învăța principiile fundamentale ale automatizării — de la achiziția datelor de la senzori, la transmiterea acestora prin rețea și luarea deciziilor automate.

3. Contribuția originală:

- Proiectarea unui sistem de definire dinamică a dispozitivelor (Schema Builder UI)
 - Se va implementa un editor vizual care va permite utilizatorilor să definească „schema” unui nou dispozitiv. În loc să se programeze o interfață backend care ascultă doar pentru un tip specific de date, aceasta va asculta dinamic în funcție de dispozitivul creat de utilizator (stocat ca o structură de date de tip JSON).
 - Aceasta permite ca interfața de control (Dashboard) să fie generată automat în momentul în care un dispozitiv se conectează, pe baza schemei stocate.
- Dezvoltarea unui Sistem de gestiune de Reguli (Rule Engine) Vizual
 - Se va realiza motorul de automatizare care permite utilizatorilor să creeze scenarii complexe de tipul „If-This-Then-That” fără a scrie cod.
- Mecanisme Avansate de Descoperire și Diagnoză (Health Check and AutoDiscovery)
 - Se va implementa un sistem de vizualizare pe bază de graf orientat în care se vor afișa toate dispozitivele conectate la ecosistem. În cadrul acestuia se pot vizualiza, pentru fiecare dispozitiv, senzorii atașați, tot sub formă de graf.
- Baza de date care va asigura persistența datelor pentru configurațiile dispozitivelor (scheme JSON, etc.), regulile de automatizare și istoricul citirilor de la senzori.

4. Resurse și bibliografie:

- Gunawan, T. S., et al. "Prototype of an IoT-based Home Automation System using Python and Raspberry Pi." IEEE International Conference on Applied Engineering (ICAE), 2018.
<https://ieeexplore.ieee.org/iel7/6287639/8600701/08846205.pdf>
DOI: 10.1109/ACCESS.2019.2942846

- Faheem, M. R., et al. "WoT: Findability Taxonomy and JSON Schema." IEEE Access, vol. 7, 2019.
<https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=8935107>, DOI:10.1109/ACCESS.2019.2960446

- Charpenay, V., et al. "The BIG IoT API - Semantically Enabling IoT Interoperability." IEEE International Conference on Smart Computing, 2018. https://www.researchgate.net/publication/326835830_The_BIG_IoT_API_-_Semantically_Enabling_IoT_Interoperability,

DOI:10.1109/MPRV.2018.2873566

5. Data înregistrării temei: 2026-01-10 18:28:46

Conducător(i) lucrare,

Ș. L. Dr. Ing. Laurențiu BOICESCU

Director departament,

Conf. dr. ing. Șerban OBREJA

Student,

OLTEANU C. Rareș-Cristian

Decan,

Prof. dr. ing. Mihnea UDREA

Cod Validare: **d44cd45022**

Declarație de onestitate academică

Prin prezenta declar că lucrarea cu titlul HomeForge. Platformă pentru automatizare rezidențială și experimentare IoT, prezentată în cadrul Facultății de Electronică, Telecomunicații și Tehnologia Informației a Universității Naționale de Știință și Tehnologie POLITEHNICA București ca cerință parțială pentru obținerea titlului de Inginer în domeniul Inginerie electronică, telecomunicații și tehnologii informaționale, programul de studii Rețele și software de telecomunicații este scrisă de mine și nu a mai fost prezentată niciodată la o facultate sau instituție de învățământ superior din țară sau străinătate.

Declar că toate sursele utilizate, inclusiv cele de pe Internet, sunt indicate în lucrare, ca referințe bibliografice. Fragmentele de text din alte surse, reproduse exact, chiar și în traducere proprie din altă limbă, sunt scrise între ghilimele și fac referință la sursă. Reformularea în cuvinte proprii a textelor scrise de către alți autori face referință la sursă. Înțeleg că plagiatul constituie infracțiune și se sancționează conform legilor în vigoare.

Declar că toate rezultatele simulărilor, experimentelor și măsurărilor pe care le prezint ca fiind făcute de mine, precum și metodele prin care au fost obținute, sunt reale și provin din respectivele simulări, experimente și măsurători. Înțeleg că falsificarea datelor și rezultatelor constituie fraudă și se sancționează conform regulamentelor în vigoare.

București, Septembrie 2026

Absolvent Olteanu Rareș-Cristian

(semnătura în original)

Cuprins

Lista figurilor	xi
Lista tabelelor	xii
Lista acronimelor	xiii
Introducere	1
Context: IoT și casele inteligente	1
Limitări ale soluțiilor comerciale	2
Platforme open-source și nivelul de abstractizare	2
Viziunea HomeForge	3
Hardware-agnostic prin MQTT	3
Stocare fără schemă fixă peste PostgreSQL	3
Sincronizare optimistă a interfeței	3
Sandbox educațional	4
Obiective concrete ale lucrării	4
Structura lucrării	5
1. Stadiul actual și analiza tehnologiilor	7
1.1. De la HTTP la MQTT: comunicația în IoT	7
1.2. Platforme open-source: unde se poziționează HomeForge	7
1.3. Justificarea stivei tehnologice	9
1.3.1. Backend: Python, Django și DRF	9
1.3.2. Bază de date: PostgreSQL cu JSONB	9
1.3.3. Magistrală de mesaje: MQTT (Mosquitto)	9
1.3.4. Frontend: React, Next.js și TanStack Query	10
1.3.5. Containerizare: Docker și Docker Compose	10
2. Arhitectura backend-ului HomeForge	11
2.1. Vedere de ansamblu	11
2.2. Modelele de date	11
2.2.1. Utilizator și profil	12
2.2.2. Cameră și dispozitiv	12
2.2.3. Tipuri de dispozitive, șabloane de card și widget-uri	13
2.2.4. Notificări	14
2.2.5. Layout-ul dashboard-ului	14
2.3. API-ul REST	14
2.3.1. Autentificare JWT	14
2.3.2. Controlul accesului pe roluri (RBAC)	15
2.3.3. Endpoint-uri principale	15
2.3.4. Auto-configurarea dispozitivelor	16
2.4. Daemonul MQTT (<code>mqtt_listener</code>)	16

2.4.1.	Bucă principală și <i>watchdog</i>	17
2.4.2.	Auto-binding MAC-IP	17
2.4.3.	Re-mapare cheile firmware → widget-uri	17
2.4.4.	Comanda inversă (de la server către dispozitiv)	18
2.5.	Mecanismul de propunere a tipurilor de dispozitive	18
2.5.1.	Import/export și defaults	19
2.5.2.	Stocarea imaginilor direct în baza de date	19
2.6.	Alte comenzi de management Django	19
2.6.0.1.	<code>load_default_types</code>	19
2.6.0.2.	<code>monitor_devices</code>	20
2.6.0.3.	Configurația-cheie din <code>settings.py</code>	20
2.7.	Integrarea sistemelor de panouri solare	20
2.7.1.	Modelul <code>SolarSystem</code>	21
2.7.2.	Pattern-ul adapter / registry	21
2.7.3.	Schema normalizată <i>overview</i>	22
2.7.4.	Poller-ul de fundal pentru <i>liveness</i>	22
2.7.5.	Endpoint-urile expuse	22
2.7.6.	Endpoint-ul <code>/widget-types/</code>	23
2.8.	Vizualizarea topologiei	23
2.9.	Optimistic UI pe partea de server	24
2.10.	Sincronizarea preferințelor de dashboard	24
3.	Aplicația web Next.js	27
3.1.	Structura proiectului	27
3.2.	Stack-ul de interfață: shadcn/ui, Radix și Tailwind	29
3.2.0.1.	Tailwind CSS 4: sistemul de stiluri.	29
3.2.0.2.	Radix UI: primitivele de comportament.	29
3.2.0.3.	shadcn/ui: stratul de „componente proprii”.	29
3.2.0.4.	Lucide React: iconițe.	30
3.2.0.5.	Sonner: sistemul de toast.	30
3.2.0.6.	Framer Motion: animații.	30
3.2.0.7.	Restul pieselor.	30
3.3.	Layout-ul aplicației: sidebar, breadcrumbs, header	31
3.3.1.	Sidebar-ul adaptiv	31
3.3.2.	Breadcrumb-uri dinamice	32
3.3.3.	Tema și culorile de accent	32
3.3.4.	<code>UserProvider</code> : sursa unică de adevăr pe client	33
3.4.	Stratul de comunicare cu API-ul	33
3.4.1.	Determinarea URL-ului backend-ului	34
3.4.2.	Gestiunea token-urilor JWT	34
3.4.3.	Tratarea uniformă a erorilor	34
3.5.	Gestiunea stării: TanStack Query	34
3.6.	Dashboard-ul principal	35
3.6.1.	Hook-ul <code>useDashboardLayout</code>	35
3.6.2.	Drag-and-drop și foldere	35
3.7.	Cardurile inteligente: <code>SmartDeviceCard</code>	36
3.7.1.	Randarea condusă de <code>card.template</code>	36

3.7.2. Pattern-ul <i>Optimistic UI</i>	37
3.8. Centrul de notificări	38
3.9. Vizualizarea topologiei	38
3.10. Pagina de monitorizare solară	39
3.10.1. Cicluri de <i>polling</i> și gestionarea stării	39
3.10.2. Diagrama de flux de putere	39
3.10.3. Graficul de putere în timp real	39
3.10.4. Cardurile cu indicatori	40
3.10.5. Dialogul administrator și starea „gol”	40
3.10.6. Pasul „Solar” din setup wizard	40
3.11. Editorul vizual de tipuri de dispozitive (<i>device builder</i>)	41
3.11.1. Paleta de componente: <i>supported</i> vs. <i>future</i>	41
3.12. Setup wizard-ul	42
3.13. Onboarding-ul: OnboardingChecklist și PageTooltip	42
3.13.0.1. Checklist-ul de pe dashboard.	43
3.13.0.2. Tooltip-urile de pagină.	43
3.14. Pagina de setări	44
3.15. Colecția de dispozitive a comunității	45
3.16. Adăugarea unui dispozitiv: AddDeviceDialog	45
3.17. Panoul de administrare	46
3.17.1. Hub-ul: <code>/dashboard/admin/</code>	46
3.17.2. Aprobarea tipurilor de dispozitive: <code>approvals/</code>	46
3.17.3. Gestiunea utilizatorilor: <code>users/</code>	47
3.17.4. Camerele, tipurile de dispozitive, debug-ul	47
4. Firmware-ul ESP32 și integrarea hardware	49
4.1. Alegerea platformei: de ce ESP32	49
4.2. Anatomia unui sketch HomeForge	49
4.2.1. Variabile de configurare (placeholder-uri)	50
4.2.2. Identitatea dispozitivului: deviceMac	50
4.2.3. Conectarea la broker și subscribe	51
4.2.4. Telemetria: publishState()	51
4.2.5. Recepția comenzilor: callback()	52
4.2.6. Bucla principală	53
4.3. Descoperirea automată a brokerului prin mDNS	54
4.3.1. Principiul de funcționare	54
4.3.2. Ierarhia celor trei niveluri de descoperire	54
4.3.3. Anunțarea propriei identități prin mDNS	55
4.4. Bibliotecile Arduino utilizate	56
4.5. Limitări ale implementării curente	57
5. Deployment, rulare și testare	59
5.1. Modurile de rulare ale platformei	59
5.1.1. Modul de dezvoltare locală	59
5.1.2. Modul containerizat de dezvoltare (<i>bridge networking</i>)	59
5.1.3. Modul de producție pe Linux nativ (<i>host networking</i>)	60
5.2. Imaginea Docker a backend-ului	60

5.3. Scriptul de start <code>run.sh</code>	61
5.4. Imaginea Docker a frontend-ului	62
5.5. Migrațiile bazei de date	62
5.6. Pași de testare manuală	63
5.6.1. Test 1: înregistrarea primului utilizator devine <code>owner</code>	63
5.6.2. Test 2: propunere și aprobare de tip de dispozitiv	63
5.6.3. Test 3: adăugarea unui dispozitiv real ESP32	63
5.6.4. Test 4: detecția dispozitivelor offline	64
5.6.5. Test 5: persistența layout-ului dashboard-ului	64
5.6.6. Sinteza rezultatelor	65
5.7. Discuție: limitări și considerente de securitate	65
Concluzii și direcții viitoare	67
Rezumatul lucrării	67
Contribuții proprii	67
Limitări cunoscute	68
Direcții viitoare	69
Învățări personale	70
Reflecție finală	70
Bibliografie	71
Anexa A. Backend	73
A.1. Modelele de date (<code>api/models.py</code>)	73
A.2. Serializare și validare (<code>api/serializers.py</code>)	80
A.3. Permisii RBAC (<code>api/permissions.py</code>)	87
A.4. Daemonul MQTT (<code>api/management/commands/mqtt_listener.py</code>)	87
A.5. Clientul MQTT de publicare (<code>api/mqtt_client.py</code>)	90
A.6. Adaptorul Fronius pentru sisteme solare (<code>api/solar/providers/fronius.py</code>)	90
Anexa B. Frontend	95
B.1. Comunicarea cu API-ul și gestiunea JWT (<code>lib/apiClient.js</code>)	95
B.2. Cardul inteligent și optimist UI (<code>components/devices/SmartDeviceCard.tsx</code>)	103
B.3. Hook-ul de layout al dashboard-ului (<code>hooks/useDashboardLayout.ts</code>)	112
B.4. Editorul vizual de tipuri de dispozitive (<code>device-builder/DeviceUICreator.tsx</code>)	117

Lista figurilor

1.	Schema conceptuală a unei case inteligente folosind HomeForge	1
2.	Identitatea vizuală a platformei HomeForge	4
1.1.	Comparație arhitecturală între modelul HTTP cerere-răspuns și modelul <i>publish/subscribe</i> bazat pe MQTT	8
1.2.	Stiva tehnologică decuplată a platformei HomeForge	10
2.1.	Diagrama de arhitectură a backend-ului HomeForge	12
2.2.	Vizualizarea radială a topologiei rețelei locale)	23
3.1.	Structura directoarelor aplicației Next.js (selecție a celor mai relevante rute și componente)	28
3.2.	Primitive shadow/ui în interfața HomeForge: (a) dialogul prin care, din colecția de dispozitive (<i>Device Collection</i>), utilizatorul introduce credențialele Wi-Fi și adresa IP a serverului pentru a descărca sketch-ul de firmware personalizat; (b) meniul <i>dropdown</i> cu submeniu pentru schimbarea rolului unui utilizator, din pagina de administrare; (c) notificarea <i>toast</i> oferită de biblioteca Sonner, afișată la finalizarea unei acțiuni.	31
3.3.	Sidebar-ul HomeForge cu sub-meniul <i>Admin Panel</i> activ	32
3.4.	Aceeași pagină <i>Settings</i> cu două <i>accent colors</i> diferite, ilustrând schimbarea instantanee a culorii de accent prin variabilele CSS <code>--primary</code> și <code>--ring</code> , fără reîncărcarea paginii.	33
3.5.	Crearea unui folder prin drag-and-drop în modul de editare al dashboard-ului: (a) cardul tras lateral declanșează indicatorul de reordonare (stânga/dreapta); (b) suprapunerea a două carduri activează zona de mijloc, care le combină într-un folder nou; (c) folderul rezultat, cu butonul-creion pentru redenumire.	36
3.6.	Diagrama temporală a unei apăsări de toggle în interfața HomeForge	38
3.7.	Pagina <i>Solar</i> : diagramă de flux de putere, indicatori și grafic în timp real	40
3.8.	Device builder-ul HomeForge	42
3.9.	Fluxul de onboarding al platformei: pașii <i>account</i> (a) și <i>devices</i> (b) din setup wizard, ecranul de finalizare care confirmă resursele configurate (c) și checklist-ul <i>Getting Started</i> afișat ulterior pe dashboard (d).	44
3.10.	Pagina <i>Device Collection</i> : grila tipurilor de dispozitive aprobate, propuse de comunitate, fiecare card afișând badge-uri pentru senzori și o miniatură a topologiei hardware.	45
3.11.	Pagina de aprobare a tipurilor de dispozitive propuse de utilizatori	47
4.1.	Modulul ESP32 DevKit V1 utilizat în prototipurile HomeForge	49
4.2.	Schemele de conexiuni (breadboard) ale celor patru tipuri de dispozitive predefinite: releul 5V comandând un consumator (a), termostatul cu DHT11 (b), modulul de temperatură și presiune BMP180 pe I2C (c) și varianta combinată DHT11 + BMP180 (d).	56
5.1.	Testul 3: dispozitiv ESP32 real adăugat și controlat	64

Lista tabelelor

1.	Cloud-centric vs. Edge: o sinteză practică	2
1.1.	MQTT vs. HTTP/REST în context IoT	8
4.1.	Conexiunile pinilor pentru tipul <i>ESP32 5V Relay</i>	56
5.1.	Rezultatele celor cinci scenarii de testare manuală	65

Lista acronimelor

ACID = Atomicity, Consistency, Isolation, Durability (atomicitate, consistență, izolare, durabilitate)

AMQP = Advanced Message Queuing Protocol (protocol avansat de transport al mesajelor)

API = Application Programming Interface (interfață de programare a aplicațiilor)

CLI = Command-Line Interface (interfață în linie de comandă)

CoAP = Constrained Application Protocol (protocol pentru aplicații cu resurse limitate)

CORS = Cross-Origin Resource Sharing (partajare a resurselor între origini diferite)

CRUD = Create, Read, Update, Delete (creare, citire, actualizare, ștergere)

CSRF = Cross-Site Request Forgery (falsificare a cererilor între situri)

CSS = Cascading Style Sheets (foi de stil în cascadă)

CTA = Call To Action (element de îndemn la acțiune)

DHCP = Dynamic Host Configuration Protocol (protocol de configurare dinamică a gazdelor)

DNS = Domain Name System (sistem de nume de domeniu)

DNS-SD = DNS-based Service Discovery (descoperire de servicii bazată pe DNS)

DOM = Document Object Model (model de obiecte al documentului)

DRF = Django REST Framework (cadru de lucru pentru API-uri REST în Django)

GIN = Generalized Inverted Index (index inversat generalizat)

GPIO = General-Purpose Input/Output (intrare/ieșire de uz general)

HTTP = HyperText Transfer Protocol (protocol de transfer al hipertextului)

I2C = Inter-Integrated Circuit (magistrală serială între circuite integrate)

IoT = Internet of Things (Internetul lucrurilor)

IP = Internet Protocol (protocol de internet)

JSON = JavaScript Object Notation (notație de obiecte JavaScript)

JSONB = JSON Binary (format binar JSON în PostgreSQL)

JSX = JavaScript XML (sintaxă XML în JavaScript)

JWT = JSON Web Token (token web în format JSON)

LAN = Local Area Network (rețea locală)

M2M = Machine-to-Machine (comunicație mașină-la-mașină)

MAC = Media Access Control (control al accesului la mediu, adresă fizică)

MCU = Microcontroller Unit (unitate de microcontroler)

mDNS = Multicast DNS (DNS prin multicast)

MQTT = Message Queuing Telemetry Transport (protocol de mesagerie de tip publish/subscribe)

NVS = Non-Volatile Storage (memorie nevolatilă)

OASIS = Organization for the Advancement of Structured Information Standards (organizație de standardizare)

OKLCH = spațiu de culoare Oklab (coordonate Lightness, Chroma, Hue)

ORM = Object-Relational Mapping (mapare obiect-relațională)

OTA = Over-The-Air (actualizare fără fir)

PV = Photovoltaic (fotovoltaic)

QoS = Quality of Service (calitatea serviciului)

RBAC = Role-Based Access Control (control al accesului bazat pe roluri)

RDBMS = Relational Database Management System (sistem de gestiune a bazelor de date)

relaționale)

REST = Representational State Transfer (transfer reprezentativ al stării)

RFC = Request for Comments (document de standardizare IETF)

RGB = Red, Green, Blue (roșu, verde, albastru)

SOC = State of Charge (stare de încărcare a bateriei)

SQL = Structured Query Language (limbaj structurat de interogare)

SRV = Service record (înregistrare DNS de serviciu)

SSID = Service Set Identifier (identificator al rețelei Wi-Fi)

TCP = Transmission Control Protocol (protocol de control al transmisiei)

TLS = Transport Layer Security (securitate la nivelul de transport)

UART = Universal Asynchronous Receiver-Transmitter (receptor-transmițător asincron universal)

UDP = User Datagram Protocol (protocol de datagrame)

UI = User Interface (interfață cu utilizatorul)

URL = Uniform Resource Locator (localizator uniform de resurse)

USB = Universal Serial Bus (magistrală serială universală)

UUID = Universally Unique Identifier (identificator unic universal)

UX = User Experience (experiența utilizatorului)

VM = Virtual Machine (mașină virtuală)

XSS = Cross-Site Scripting (injectare de scripturi între situri)

Introducere

Context: IoT și casele inteligente

În ultimii ani, conceptul de *Internet of Things* (IoT) s-a mutat dintr-un termen de cercetare într-un fenomen care influențează direct modul în care folosim locuința. O „casă inteligentă” înseamnă, în practică, o rețea de senzori și actuatori conectați la o unitate de control. Aceasta interpretează datele și execută acțiuni, fie automate, fie declanșate de utilizator. Un *review* amplu al aplicațiilor Smart Home propune o clasificare a acestora pe domenii precum confortul rezidențial, securitatea, eficiența energetică și asistența pentru sănătate [1].

Adopția acestor tehnologii nu depinde doar de cât de avansat este hardware-ul, ci și de cât de mult au încredere utilizatorii în sistem. O analiză a cercetărilor privind adopția caselor inteligente arată că factorii decisivi sunt utilitatea percepută, ușurința de utilizare și îngrijorările legate de confidențialitate [2]. Observația contează pentru lucrarea de față. Ea explică de ce o platformă transparentă, locală și inspectabilă de utilizator, are sens chiar și acolo unde există alternative comerciale.

Pe partea de comunicație, soluțiile actuale s-au mutat în mare parte de la modelul cerere-răspuns specific HTTP la o paradigmă asincronă bazată pe *publish/subscribe*. Protocolul MQTT (Message Queuing Telemetry Transport) a devenit standardul *de facto* pentru schimbul de mesaje între microcontrolere și server. Două motive cântăresc cel mai mult: overhead-ul redus și faptul că un singur broker poate distribui eficient mesaje la mulți abonați [3]. În arhitectura HomeForge, MQTT joacă exact acest rol: este magistrala prin care firmware-ul ESP32 comunică cu backend-ul Django (Figura 1).

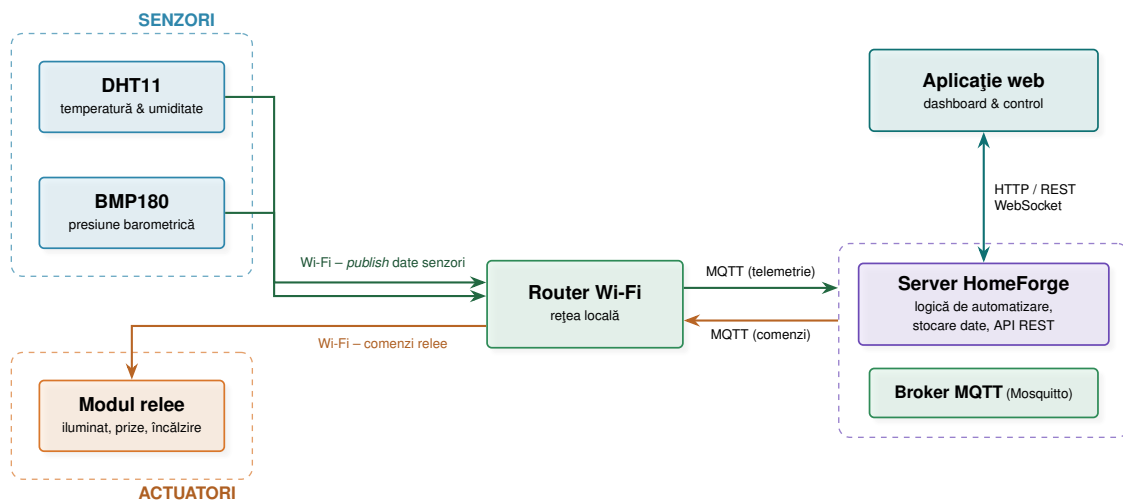


Figura 1: Schema conceptuală a unei case inteligente folosind HomeForge

Limitări ale soluțiilor comerciale

Marea majoritate a produselor de pe piață (Google Nest, Amazon Alexa, Tuya, Sonoff și variațiunile lor) lucrează după un model centrat pe cloud. Dispozitivul fizic trimite datele către serverele producătorului, acolo rulează logica de automatizare, de acolo se întorc comenzile. Există trei probleme practice cu această abordare, observate sistematic în literatură [4].

Latență și dependență de internet.

O comandă banală (*aprinde becul din bucătărie*) presupune un drum dus-întors prin internet către serverul producătorului. Dacă rețeaua locală e instabilă sau dacă serverele furnizorului au probleme, casa „inteligentă” devine dintr-o dată surdă. La un sistem local, același mesaj traversează doar LAN-ul și ajunge la dispozitiv în câteva milisecunde.

Confidențialitate.

Atunci când toate datele trec prin servere terțe, utilizatorul pierde controlul asupra lor. Un studiu de caz privind arhitecturile IoT centralizate analizează tocmai aceste vulnerabilități și motivează prin ele propunerea de a folosi blockchain ca strat alternativ de securitate [5]. Datele acumulate (când e cineva acasă, ce dispozitive se folosesc) pot fi monetizate sau scurse într-o breșă, fără ca proprietarul să aibă vreo cale de control direct.

Vendor lock-in.

Producătorii folosesc protocoale închise, firmware blocat și API-uri nedocumentate pentru a împiedica integrările între ecosisteme. În momentul în care un furnizor decide să închidă serviciile cloud, dispozitivele cumpărate devin practic deșeuri electronice [4].

Răspunsul cercetării și al comunității open-source la aceste limitări este *Edge Computing*. Pe scurt: muți logica și stocarea cât mai aproape de sursa datelor, ideal chiar în rețeaua locală a utilizatorului [6]. HomeForge se înscrie în această direcție, dar adaugă un obiectiv suplimentar pe care îl detaliez în secțiunea următoare: *transparența* sistemului.

Aspect	Cloud-centric	Edge / Local
Locul procesării	Servere externe	Server local (LAN)
Dependență de internet	Critică	Aproape inexistentă
Latență tipică	50–1000 ms	sub 5 ms
Proprietatea datelor	La furnizor	La utilizator
Interoperabilitate	Limitată artificial	Bazată pe protocoale deschise
Suprafața de atac	Globală	Limitată la LAN

Tabela 1: Cloud-centric vs. Edge: o sinteză practică

Platforme open-source și nivelul de abstractizare

În jurul ideii de control local s-a dezvoltat o comunitate amplă, iar cea mai răspândită platformă rezultată este Home Assistant. Aceasta este o soluție stabilă, scrisă în Python, cu mii de integrări ce permit comunicarea cu o gamă largă de dispozitive de pe piață. Pentru un utilizator final orientat exclusiv către funcționalitate, aceasta este o alegere solidă.

Dificultatea apare însă atunci când Home Assistant este analizată din perspectiva unui student sau a unui inginer interesat de mecanismele subiacente. Pentru a integra atâtea categorii de dispozitive, platforma a adoptat un nivel ridicat de abstractizare. Orice dispozitiv devine o „entitate” cu o „stare” discretă, iar comunicarea internă trece printr-o magistrală de evenimente. Fluxurile concrete de rețea, adică interlocutorii, topicile MQTT și structura payload-urilor JSON, rămân ascunse sub stratul grafic. Pe măsură ce platforma a evoluat, configurațiile au migrat din fișiere YAML editabile către baze de date interne (directorul `.storage` din versiunile recente). Rezultatul? O platformă potrivită pentru consumatorul final, dar opacă pentru cineva interesat de funcționarea la nivel inferior.

Această observație este punctul de pornire al proiectului HomeForge. Obiectivul nu a fost replicarea funcționalităților Home Assistant. Am urmărit transparența end-to-end a fluxului de date, păstrând în același timp o bază de cod redusă și ușor de parcurs.

Viziunea HomeForge

HomeForge este o platformă Smart Home cu execuție locală, concepută ca instrument didactic și mediu de experimentare pentru aplicații IoT. Spre deosebire de soluțiile care abstractizează complet stiva tehnologică, platforma ține componentele la vedere. Portul 1883 al brokerului Mosquitto este expus în rețeaua locală, fiecare dispozitiv are un topic MQTT documentat, iar codul backend-ului (Django cu Django REST Framework) este suficient de mic încât logica să poată fi parcursă integral într-un timp rezonabil. Arhitectura se sprijină pe patru decizii principale, descrise în continuare.

Hardware-agnostic prin MQTT

Spre deosebire de un middleware care ar cunoaște explicit fiecare producător, HomeForge folosește MQTT ca unică magistrală de comunicație. Orice ESP32 (sau alt microcontroller) este recunoscut de platformă dacă respectă un contract minimal: publică pe topicul `homeforge/devices/{MAC}/state` și, pentru dispozitivele cu actuator, se abonează la `homeforge/devices/{MAC}/command`. Topicile și structura payload-urilor JSON sunt definite în codul firmware distribuit inline în fixture-ul `default_device_types.json` și, simetric, în comanda Django `mqtt_listener` care le procesează (`mqtt_listener.py`). Detaliile sunt tratate în Capitolul 2.

Stocare fără schemă fixă peste PostgreSQL

Datele raportate de senzori sunt eterogene. Un releu transmite o valoare booleană, o stație meteorologică transmite mai mulți parametri de tip *float*, iar un termostat raportează simultan temperatura curentă și valoarea de referință. O coloană SQL distinctă pentru fiecare câmp nou ar însemna migrații dese. Soluția adoptată: starea se păstrează într-o coloană JSONB denumită `current_state` pe modelul `Device` (`models.py:96`). Relațiile structurale (utilizator, cameră, tip de dispozitiv) rămân însă strict normalizate, fiind utilizate de mecanismul de control al accesului.

Sincronizare optimistă a interfeței

Pe partea de frontend (Next.js + React), s-a urmărit ca utilizatorul să nu aștepte confirmarea hardware înainte ca butonul să își schimbe starea vizuală. Implementarea folosește pattern-ul *optimistic update* oferit de TanStack Query: la `onMutate` se actualizează imediat

cache-ul React, iar la o eventuală eroare (`onError`) se efectuează rollback la starea anterioară (`SmartDeviceCard.tsx:78-127`). Detaliile sunt tratate în Capitolul 3.

Sandbox educațional

Codul este menținut minimalist. Backend-ul este o aplicație Django REST Framework standard, structura bazei de date poate fi inspectată prin migrațiile Django, traficul MQTT poate fi observat din linia de comandă cu `mosquitto_sub`, iar întreaga aplicație rulează prin Docker Compose. Astfel, fluxul de la apăsarea unui buton în interfața web până la comutarea fizică a unui releu poate fi urmărit pas cu pas. Nu e nevoie să traversezi straturi succesive de abstractizare.



Figura 2: Identitatea vizuală a platformei HomeForge

Obiective concrete ale lucrării

Lucrarea își propune să descrie proiectarea și implementarea HomeForge ca platformă funcțională, urmărind explicit următoarele obiective:

Hardware și firmware embedded:

- firmware ESP32 modular care suportă cel puțin patru variante (releu 5V, termostat cu DHT11, senzor de presiune și temperatură cu BMP180, variantă combinată DHT11 + BMP180), livrate inline în setul predefinit de tipuri de dispozitive;
- descoperirea automată a brokerului prin mDNS / DNS-SD (`_mqtt._tcp.local`) [7], [8], cu trei niveluri de fallback: `server_ip` manual, cache local în memoria nevolatilă (NVS) și un endpoint web `/config`;
- mecanism de reconectare automată la Wi-Fi și la brokerul MQTT, cu `ClientID` bazat pe adresa MAC; raportare ciclică al unui semnal de prezență (*heartbeat*) la fiecare 5 secunde și ascultare continuă pe topicul de comandă.

Backend Django:

- API REST construit pe Django REST Framework, urmând principiile arhitecturale ale lui Fielding [9], cu autentificare JWT [10] și paginare implicită;
- daemon MQTT integrat ca `management command` Django (`python manage.py mqtt_listener`), care primește telemetria, o asociază cu device-ul corect prin MAC sau IP și marchează dispozitivele ca `offline` după 30 de secunde fără raportare a stării;

- modul de integrare cu sisteme externe de panouri fotovoltaice, structurat ca pattern adapter/registry; prima implementare acoperă API-ul oficial Fronius Solar V1 [11], cu o schemă internă normalizată ce permite adăugarea unor furnizori suplimentari fără modificări la nivelul interfeței;
- model `Device` cu o coloană `JSONField` pentru starea curentă, mapată de Django pe tipul nativ `JSONB` din PostgreSQL [12];
- sistem ierarhic de roluri (`owner`, `admin`, `user`) și un mecanism *first-user-becomes-owner* aplicat la înregistrare.

Frontend Next.js:

- dashboard cu carduri configurabile care suportă *toggle*, *slider* și widget-uri de senzor (temperatură, umiditate, presiune, plus tipuri rezervate pentru extensii ulterioare: mișcare, lumină și CO_2);
- pagină dedicată de monitorizare fotovoltaică, cu diagramă animată de flux de putere (producție PV, consum casnic, rețea, baterie) și grafic de putere în timp real;
- gestionarea layout-ului prin TanStack Query, persistat pe server și cu fallback `localStorage`, suportând foldere ad-hoc de 2–4 dispozitive în stil iOS;
- editor vizual pentru tipuri de dispozitive (*device builder*), cu propunere către administrator, aprobare/respingere și editor de firmware/wiring/documentație;
- vizualizare a topologiei rețelei locale folosind React Flow, cu noduri distincte pentru gateway-ul MQTT, dispozitivele MCU și sistemele solare înregistrate.

Deployment:

- ambalare în containere Docker pentru backend, frontend și PostgreSQL, cu două configurații complementare: una pentru dezvoltare (bridge networking, cu porturi mapate explicit, pentru Docker Desktop pe Windows/macOS) și una pentru producție pe Linux nativ (`network_mode: host` pe Raspberry Pi sau echivalent);
- brokerul Mosquitto rulează în același container cu backend-ul, alături de un serviciu Avahi care anunță `_mqtt._tcp` în rețeaua locală în configurația de producție;
- script unic de dezvoltare (`./dev.sh`) care pornește în paralel backend și frontend pentru iterații locale fără Docker.

Structura lucrării

Lucrarea este organizată pe capitole care urmează firul natural de la teorie la implementare:

- Capitolul 1 prezintă stadiul actual al tehnologiilor IoT și justifică alegerile făcute (MQTT vs. HTTP, PostgreSQL cu JSONB, stiva Next.js/React).
- Capitolul 2 descrie arhitectura backend-ului HomeForge: modelele de date, API-ul REST, autentificarea, daemonul MQTT și mecanismele de aprobare a tipurilor de dispozitive.
- Capitolul 3 detaliază aplicația web (Next.js), componentele dashboard-ului, motorul de layout, optimist UI-ul și editorul vizual de tipuri de dispozitive.

- Capitolul 4 acoperă firmware-ul ESP32: protocolul de comunicație, structura proiectelor template și mecanismele de auto-detectie a dispozitivelor.
- Capitolul 5 documentează procesul de deployment, containerizarea și pașii de testare manuală.
- Concluziile rezumă rezultatele obținute și schițează direcții viitoare de dezvoltare.

Capitolul 1

Stadiul actual și analiza tehnologiilor

1.1 De la HTTP la MQTT: comunicația în IoT

La prima vedere, un microcontroler care raportează date către un server este o aplicație web ca oricare alta. În realitate, dacă încercăm să folosim HTTP/REST pentru un nod IoT, costurile devin rapid prohibitive.

Modelul cerere-răspuns presupune că senzorul deschide o conexiune TCP nouă pentru fiecare mesaj. Trimite un header HTTP care, cu cookie-uri și meta-informații, poate ajunge la câteva sute de bytes, apoi așteaptă răspunsul și închide conexiunea. Iar când serverul vrea să trimită o comandă către dispozitiv, problema se mută. HTTP nu suportă push nativ. Fie clientul interoghează ciclic (*polling*), fie se trece la WebSocket-uri, ceea ce ne aduce, de fapt, foarte aproape de paradigma *publish/subscribe* - un model în care emițătorii (*publishers*) și receptorii (*subscribers*) nu comunică direct, ci printr-un intermediar care rutează mesajele pe baza unor *topicuri*, eliminând nevoia unei conexiuni permanente client-server (Figura 1.1).

MQTT alege o cale diferită. Există un broker central, iar fiecare client (senzor sau aplicație) menține o singură conexiune TCP persistentă cu el. Mesajele se publică pe „topice” (șiruri ierarhice de forma `homeforge/devices/AABBCCDDEEFF/state`), iar brokerul rutează automat fiecare publicare către toți abonații topicului respectiv. Antetul minim al unui pachet MQTT este de doar 2 bytes [3]. În plus, protocolul oferă nativ trei niveluri de QoS și un mecanism de *Last Will and Testament*, prin care brokerul detectează automat deconectarea unui nod.

Un studiu comparativ al principalelor protocoale de mesagerie pentru IoT (MQTT, CoAP, AMQP și HTTP) arată o împărțire clară. MQTT și CoAP se potrivesc rețelelor de senzori cu lățime de bandă redusă și nodurilor cu resurse limitate, în timp ce HTTP rămâne firesc pentru integrarea cu sisteme web tradiționale [13]. Un alt *survey*, dedicat utilizării MQTT în sistemele M2M și IoT, inventariază implementările uzuale de broker: Mosquitto, HiveMQ, EMQ X, VerneMQ. Dintre ele, Mosquitto este una dintre cele mai răspândite alegeri, datorită amprente reduse de resurse și a faptului că rulează inclusiv pe un Raspberry Pi [3].

În HomeForge folosesc o abordare hibridă, pentru că fiecare protocol are zona în care este eficient:

- MQTT pentru comunicația cu dispozitivele fizice: telemetrie și comenzi, în timp real;
- HTTP/REST pentru interacțiunea dintre browser și backend: preluarea listei de dispozitive, autentificarea, profilul și layout-ul dashboard-ului.

Frontend-ul nu vorbește direct cu brokerul MQTT; el comunică doar cu API-ul REST al Django, iar backend-ul este cel care traduce între cele două lumi.

1.2 Platforme open-source: unde se poziționează HomeForge

Soluțiile open-source dedicate Smart Home-ului sunt deja complexe și stabile. Cele mai cunoscute, Home Assistant și OpenHAB, au câștigat o tracțiune semnificativă în comunitate prin integrările cu sute de producători, prin suportul pentru protocoale specializate (Zigbee, Z-Wave,

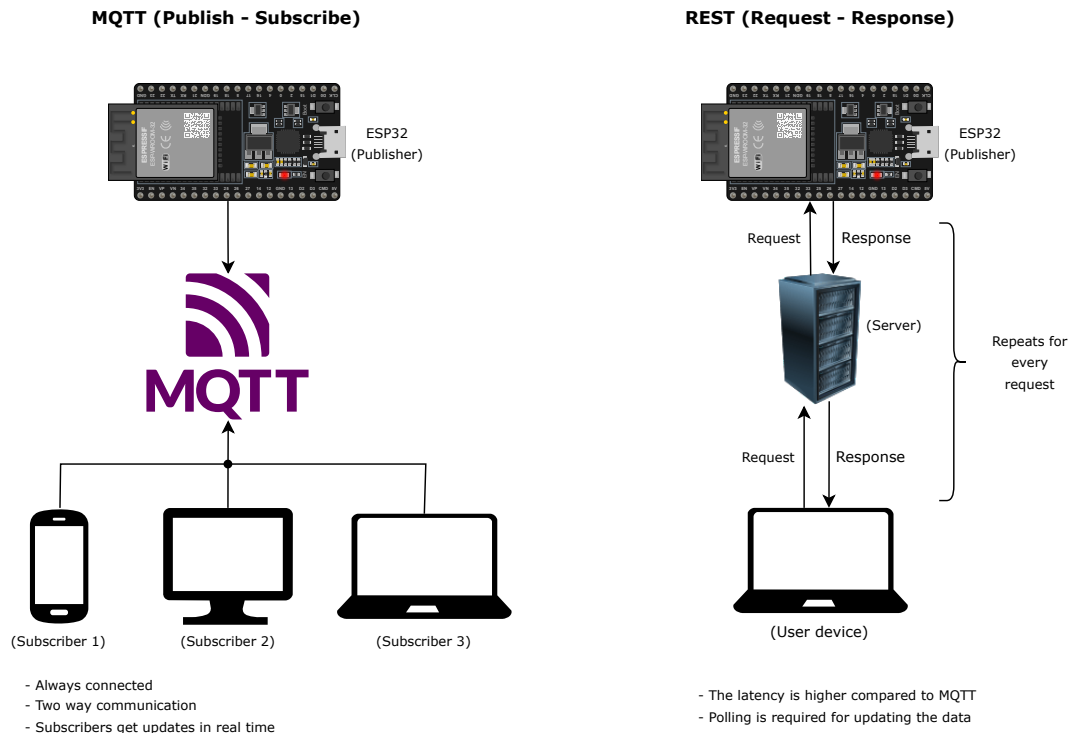


Figura 1.1: Comparație arhitecturală între modelul HTTP cerere-răspuns și modelul *publish/subscribe* bazat pe MQTT

Caracteristică	MQTT	HTTP / REST
Model	Asincron, <i>publish-subscribe</i> (broker)	Sincron, cerere-răspuns
Overhead antet	de la 2 bytes	sute de bytes (headere, cookies)
Conexiune TCP	Persistentă (<i>stateful</i>)	Per cerere (<i>stateless</i>)
QoS nativ	Da, 3 niveluri	Nu (depinde de aplicație)
Push server → client	Suportat nativ	Necesită <i>polling</i> sau WebSocket-uri
Caz tipic	Telemetrie, microcontrolere	API web, operații CRUD

Tabela 1.1: MQTT vs. HTTP/REST în context IoT

KNX, Matter), prin reguli avansate de automatizare construite vizual și prin documentația vastă creată de utilizatori.

Pentru un utilizator final care vrea o casă inteligentă completă, oricare dintre ele este o alegere mai bună decât HomeForge. Diferența este nișa în care lucrăm. HomeForge nu vrea să suporte 100 de producători, nici 10. Vrea să suporte un singur lucru bine: dispozitive făcute manual, pe ESP32, care vorbesc MQTT printr-un format JSON simplu. În schimb, oferă:

- un cod sursă scurt (sub 5000 de linii pentru API-ul backend);
- fără mecanisme ascunse: fiecare endpoint este vizibil în `urls.py`;
- un *device builder* grafic, care permite construirea unui tip de dispozitiv vizual;
- o curbă de învățare scăzută pentru studenți, care pot extinde sistemul fără să citească arhitectura unei platforme cu 10 ani de dezvoltare în spate.

Prin urmare, HomeForge nu intră în competiție cu Home Assistant pe terenul lui. Acoperă o zonă diferită: a celui care vrea să prototipeze rapid, să înțeleagă fluxul de date end-to-end și,

eventual, să pornească de aici către un sistem mai serios.

1.3 Justificarea stivei tehnologice

Alegerile tehnologice nu sunt accidentale. Fiecare componentă rezolvă o problemă concretă, întâlnită în implementare. Mai jos justific principalele opțiuni, care se vor regăsi în capitolele următoare; stiva rezultată, cu straturile decuplate prin MQTT și REST, este arătată în Figura 1.2.

1.3.1 Backend: Python, Django și DRF

Python a fost ales pentru că oferă cele mai avansate biblioteci pentru manipularea datelor heterogene (`paho-mqtt`, `Pillow`, `ArduinoJson` la nivel de embed). Django adaugă peste asta un ORM stabil, un sistem de migrații versionabil și protecții implicite împotriva atacurilor uzuale (CSRF, XSS, SQL injection). Django REST Framework completează stack-ul cu *serializers*, *viewsets* și *permission classes*, care reduc semnificativ codul boilerplate atunci când construiești un API.

Concret, în HomeForge, un endpoint precum `POST /api/devices/` este descris în doar câteva zeci de linii, validările sunt declarate în serializer, iar permisiunile sunt asignate per view.

1.3.2 Bază de date: PostgreSQL cu JSONB

La nivel de bază de date, am avut nevoie de două lucruri în același loc: relații stricte (un dispozitiv aparține unui utilizator și opțional unei camere) și structuri flexibile (starea curentă a unui dispozitiv este JSON arbitrar). PostgreSQL este unul dintre puținele RDBMS-uri care oferă bine ambele lumi, prin tipul `JSONB` și suport pentru indexare GIN.

Documentația oficială PostgreSQL precizează diferența față de tipul `json`, care păstrează textul JSON brut. Tipul `JSONB` pre-parsează documentul într-un format binar decompus. Astfel, citirea ulterioară devine mai rapidă și se poate indexa cu GIN pentru căutări în interiorul JSON-ului [12]. Pentru o platformă didactică, combinația este ideală. Studenții văd o schemă SQL clară pentru `User`, `Room` și `Device`, dar nu sunt obligați să facă o migrație ori de câte ori adaugă un senzor cu un format nou.

În cod, decizia se traduce într-un singur câmp:

```
1 class Device(models.Model):
2     # ... relațiile clasice ...
3     current_state = models.JSONField(
4         default=dict, blank=True,
5         help_text="Current state of device controls"
6     )
```

1.3.3 Magistrală de mesaje: MQTT (Mosquitto)

Pentru implementarea concretă a brokerului, am folosit Mosquitto: open-source, standardizat OASIS și suficient de ușor încât poate rula în același container Docker cu backend-ul. Pe portul implicit 1883, brokerul ascultă atât de la clienții ESP32 cât și de la procesul `mqtt_listener` al backend-ului.

În implementarea curentă, mecanismul *Last Will and Testament* oferit de MQTT nu este utilizat. Statusul `offline` este detectat printr-un timer aplicativ, în comanda `mqtt_listener`, care marchează drept `offline` orice dispozitiv ce nu a mai publicat în ultimele 30 de secunde.

Această abordare simplifică firmware-ul ESP32 și centralizează logica de gestionare a stării în partea Python a aplicației.

1.3.4 Frontend: React, Next.js și TanStack Query

Pe partea de UI, dashboard-ul este un *Single Page Application* care își actualizează cardurile în timp real, pe baza datelor preluate de la backend. React rezolvă re-arendarea eficientă prin Virtual DOM, iar Next.js oferă deasupra un **app router** modern, suport *Server-Side Rendering* (pe care nu îl folosesc agresiv în acest proiect) și un sistem de bundling deja optimizat. În practică, cele două combinate oferă un timp scurt până la prima interacțiune, fără să configurez manual Webpack sau alte unelte de build.

În HomeForge folosesc TanStack Query (fost React Query) pentru toată stocarea de date la nivel de client. Decizia este motivată de două lucruri: (1) cache automat și deduplicare a cererilor, (2) suport excelent pentru *optimistic updates*, care este coloana vertebrală a UI-ului responsiv din dashboard. Detaliile sunt în Capitolul 3.

1.3.5 Containerizare: Docker și Docker Compose

Problema „pe calculatorul meu funcționează” este reală în orice proiect cu mai multe servicii. Cercetări din domeniul Edge Computing au demonstrat că rularea containerelor Docker este fezabilă chiar și pe noduri cu resurse limitate, precum Raspberry Pi [14]. Aceeași imagine poate fi distribuită fără modificări, de la un server obișnuit până la un nod de margine. În HomeForge:

- **backend** (Django + Mosquitto + Avahi) și **db** (PostgreSQL) rulează ca două servicii separate;
- **frontend** este al treilea container, cu o imagine Next.js standard;
- dezvoltatorul rulează totul cu **docker compose up**, fără să instaleze nimic local.

Pentru dezvoltare rapidă există și varianta `./dev.sh`, care pornește backend-ul și frontend-ul direct pe mașina gazdă, ocolind containerele.

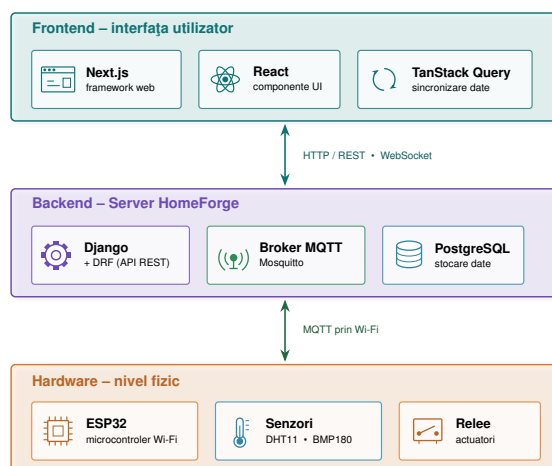


Figura 1.2: Stiva tehnologică decuplată a platformei HomeForge

Capitolul 2

Arhitectura backend-ului HomeForge

2.1 Vedere de ansamblu

Backend-ul HomeForge este un serviciu Django cu două roluri simultane. Expune un API REST (în accepțiunea originală a lui Roy Fielding, cel care a introdus stilul REST în teza sa de doctorat, ca stil arhitectural orientat pe resurse, reprezentări și mesaje auto-descriptive [9]), consumat de aplicația web Next.js. În paralel, rulează un proces dedicat care ascultă mesajele MQTT publicate de microcontrolere. Cele două canale de comunicație sunt complet decuplate: dispozitivele fizice nu comunică direct cu frontend-ul, iar frontend-ul nu deschide o conexiune MQTT proprie (Figura 2.1).

Alegerea REST pentru comunicația dintre browser și backend, respectiv a MQTT pentru cea dintre backend și hardware, este justificată de profilul fiecărui canal de comunicație. O analiză comparativă între REST și stilurile de tip *big web services* (SOAP, WS-*) arată o delimitare clară. REST este indicat când clienții au nevoie de un model simplu, *stateless* și ușor de cache-uit. SOAP și WS-* devin relevante doar când sunt necesare mecanisme avansate de transport, precum tranzacțiile distribuite sau mesageria cu garanții formale de livrare [15]. Pentru un dashboard care interoghează lista de dispozitive și transmite comenzi punctuale, REST oferă un model potrivit. Pentru telemetria continuă provenită de la senzori, REST devine însă inadecvat, întrucât nu suportă nativ *push*-ul de la server către client și nici conexiuni persistente, iar aceste cerințe sunt acoperite de MQTT.

În practică, separarea responsabilităților se vede în structura proiectului:

- `my.backend/settings.py` configurează Django, baza de date PostgreSQL, JWT și permisiunile implicite;
- `api/models.py` definește modelele relaționale și câmpurile JSONB pentru stările dispozitivelor;
- `api/views.py` și `api/urls.py` implementează endpoint-urile REST;
- `api/management/commands/mqtt_listener.py` este procesul de fundal care ascultă brokerul Mosquitto.

Atunci când brokerul, baza de date și serverul rulează în Docker, brokerul este pornit în același container cu Django, de către scriptul `run.sh`. Acesta pornește în ordine: `dbus-daemon`, `avahi-daemon`, `mosquitto`, apoi lansează în fundal atât `mqtt_listener` cât și comanda de *polling* solar (`poll_solar`), și termină cu serverul de dezvoltare Django. Pentru dezvoltare locală fără Docker există scriptul `dev.sh`, care poate porni doar backend-ul sau, în modul implicit, backend-ul și frontend-ul în paralel.

2.2 Modelele de date

Toate modelele aplicației sunt definite în `models.py`, reprodus integral în Anexa A.1.

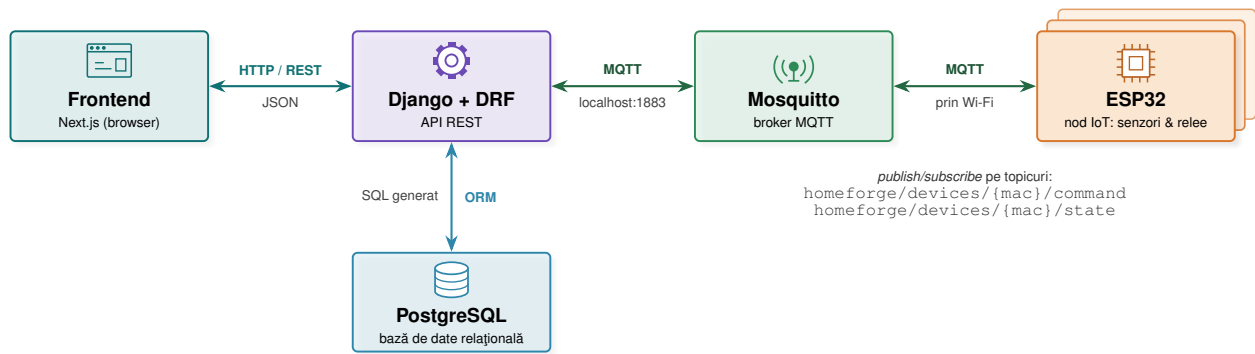


Figura 2.1: Diagrama de arhitectură a backend-ului HomeForge

2.2.1 Utilizator și profil

Django oferă deja un model `User` complet. Pe lângă acesta, am definit un model `Profile` legat `OneToOne` de utilizator, care stochează rolul, culoarea de accent a interfeței și avatarul:

```

1 class Profile(models.Model):
2     ROLE_OWNER = 'owner'
3     ROLE_ADMIN = 'admin'
4     ROLE_USER = 'user'
5
6     user = models.OneToOneField(
7         settings.AUTH_USER_MODEL,
8         on_delete=models.CASCADE,
9         related_name='profile',
10    )
11    avatar = models.ImageField(upload_to=avatar_upload_path, null=True, blank=True)
12    role = models.CharField(max_length=20, choices=ROLE_CHOICES, default=ROLE_USER)
13    accent_color = models.CharField(max_length=7, default='#3B82F6', blank=True)

```

Două semnale `post_save` se ocupă de crearea automată a profilului la înregistrarea unui utilizator nou, iar un semnal `pre_save` șterge fizic avatarul vechi atunci când se încarcă unul nou. Este un detaliu minor, dar util pentru a evita acumularea de fișiere orfane.

Mecanismul de tip *first-user-becomes-owner* este implementat în `serializers.py` (reprodus integral în Anexa A.2): la primul `POST /register/`, dacă nu există niciun utilizator în baza de date, contul nou primește rolul `owner`, `is_staff` și `is_superuser`. Toți utilizatorii ulteriori sunt creați cu rolul `user` și pot fi promovați doar manual de un administrator. Detaliul este expus public prin endpoint-ul `GET /api/system-status/`, care returnează `{"is_fresh": true}` atunci când baza de date nu conține niciun utilizator. Frontend-ul îl folosește pentru a redirecționa primul vizitator către un *setup wizard*.

2.2.2 Cameră și dispozitiv

Modelele `Room` și `Device` sunt minimaliste. O cameră are un nume, o iconiță (clasa unui icon `FontAwesome`) și un utilizator-proprietar; numele este unic la nivel de utilizator. Modelul `Device` este însă piesa centrală a sistemului:

```

1 class Device(models.Model):
2     STATUS_ONLINE = 'online'
3     STATUS_OFFLINE = 'offline'
4     STATUS_ERROR = 'error'
5
6     name = models.CharField(max_length=100)
7     ip_address = models.GenericIPAddressField(protocol='IPv4')
8     mac_address = models.CharField(max_length=17, blank=True, null=True, unique=True)
9     status = models.CharField(max_length=10, choices=STATUS_CHOICES,
10                             default=STATUS_OFFLINE, db_index=True)
11    icon = models.CharField(max_length=50, default='fa-cube', blank=True)

```

```

12     device_type = models.ForeignKey('CustomDeviceType', on_delete=models.CASCADE,
13                                     related_name='devices')
14     room = models.ForeignKey(Room, on_delete=models.SET_NULL, null=True, blank=True,
15                               related_name='devices')
16     user = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE,
17                               related_name='devices', db_index=True)
18     current_state = models.JSONField(default=dict, blank=True)
19     updated_at = models.DateTimeField(auto_now=True)

```

Trei observații merită subliniate:

1. Adresa MAC nu este obligatorie la creare. La înregistrarea manuală a unui dispozitiv (POST /api/devices/), utilizatorul completează doar adresa IP. MAC-ul este completat automat la primul mesaj MQTT primit de la dispozitiv (vezi secțiunea de auto-discovery).
2. Câmpul `current_state` este de tip `JSONField`, ceea ce permite stocarea oricărei structuri JSON arbitrare; pe PostgreSQL, Django mapează tipul către `JSONB`, formatul binar pre-parsat documentat de echipa PostgreSQL [12].
3. Câmpul `updated_at` folosește `auto_now=True` și este actualizat de fiecare dată când dispozitivul publică un mesaj. `mqtt_listener` se bazează pe el pentru a detecta dispozitivele care nu au mai dat semne de viață.

Pentru a evita problema clasică $N+1$ pe interogările cu mai multe relații, toate `get_queryset()`-urile importante folosesc `select_related` și `prefetch_related`:

```

1 Device.objects.select_related(
2     'room', 'user', 'device_type', 'device_type__card_template'
3 ).prefetch_related(
4     'device_type__card_template__controls'
5 )

```

Rezultatul este o singură interogare SQL cu INNER JOIN-uri, indiferent de câte dispozitive sunt afișate pe ecran.

2.2.3 Tipuri de dispozitive, șabloane de card și widget-uri

Aici intervine partea mai puțin obișnuită a arhitecturii. Pentru ca un utilizator să poată adăuga un tip de dispozitiv nou (o placă DIY, o variantă proprie de termostat), are nevoie de două lucruri: o descriere a *topologiei electrice* a dispozitivului (ce microcontroler, ce senzori, pe ce pini) și o descriere a *interfeței grafice* care apare pe card (un toggle pentru releu, un widget pentru temperatură, eventual un slider pentru luminozitate).

Aceste două descrieri sunt persistate prin trei modele înlănțuite:

- **CustomDeviceType**: nume, definiție topologică (JSON), cod firmware, schemă electrică, documentație, status de aprobare;
- **DeviceCardTemplate**: legat **OneToOne** de **CustomDeviceType**, conține un câmp `layout_config` (JSON cu configurația de grid);
- **DeviceControl**: legat **ManyToOne** de **DeviceCardTemplate**, descrie fiecare widget vizual (toggle, slider, sensor) împreună cu maparea către o variabilă din starea dispozitivului.

Tipurile de widget-uri suportate sunt enumerate explicit în `models.py:163–195`: pe lângă cele interactive (**TOGGLE**, **SLIDER**, **BUTTON**), există widget-uri specializate pentru senzori uzuali (**TEMPERATURE**, **HUMIDITY**, **MOTION**, **LIGHT**, **CO2**, **PRESSURE**, **POWER**, **BATTERY**, **STATUS**, **GAUGE**). Fiecare control are un câmp `variable_mapping` care leagă widget-ul de o cheie din `current_state` (de exemplu `relay_1` sau `temperature`).

Procesul de adăugare a unui tip de dispozitiv este descris ca o operațiune cu mai mulți pași în secțiunea dedicată *crowdsourcing*-ului, mai jos în acest capitol.

2.2.4 Notificări

Modelul `Notification` stochează evenimentele din sistem care merită prezentate utilizatorului: propuneri de tipuri de dispozitive în așteptare, aprobări, respingeri, dispozitive offline, mesaje broadcast, etc. Sunt definite 10 tipuri și 4 niveluri de prioritate (`low`, `normal`, `high`, `urgent`). Două metode de clasă merită menționate:

```
1 | @classmethod
2 | def create_notification(cls, user, notification_type, title, message, ...):
3 |     return cls.objects.create(...)
4 |
5 | @classmethod
6 | def notify_admins(cls, notification_type, title, message, ...):
7 |     admin_profiles = Profile.objects.filter(
8 |         role__in=[Profile.ROLE_ADMIN, Profile.ROLE_OWNER]
9 |     )
10 |     admin_users = User.objects.filter(profile__in=admin_profiles)
11 |     # ... bulk_create ...
```

`notify_admins` este apelată din `DeviceTypeProposeView`: când un utilizator obișnuit propune un tip nou, toți administratorii primesc automat o notificare. Folosirea `bulk.create` face o singură interogare SQL chiar dacă există 50 de admini.

2.2.5 Layout-ul dashboard-ului

Una dintre cele mai recente îmbunătățiri a fost mutarea layout-ului dashboard-ului din `localStorage` pe server. Modelul `DashboardLayout` este construit astfel încât un singur câmp `user` (`OneToOne`, `null=True`) să rezolve simultan două nevoi: layout personal pentru fiecare utilizator și layout partajat „de fabrică” setat de admin. Constrângerea de unicitate condiționată asigură că există cel mult un singur layout partajat:

```
1 | class Meta:
2 |     constraints = [
3 |         models.UniqueConstraint(
4 |             fields=['user'],
5 |             condition=models.Q(user__isnull=True),
6 |             name='unique_shared_layout',
7 |         ),
8 |     ]
```

Logica de fallback (personal → partajat → null) trăiește în `DashboardLayoutView.get()`.

2.3 API-ul REST

Lista completă a endpoint-urilor este expusă în `urls.py`. Sunt 44 de rute, grupate logic în șase familii. În continuare descriu mecanismele importante; pentru contractul exact al fiecărui endpoint trimit la fișierul de cod și la `API_GUIDE.md` din rădăcina backend-ului.

2.3.1 Autentificare JWT

Autentificarea folosește `djangorestframework-simplejwt`. La login (`POST /api/login/`), serverul emite două token-uri:

- `access`, valabil 5 minute, folosit la fiecare cerere prin antetul `Authorization: Bearer <token>`;
- `refresh`, cu durată mai lungă (1 zi), folosit doar pentru a reînnoi token-ul de acces prin `POST /api/token/refresh/`.

Specificația oficială a formatului (RFC 7519, publicată de IETF în 2015) definește un JWT ca o secvență compactă, URL-safe, de părți `base64url` separate prin punct; în varianta semnată (JWS), folosită și de noi, structura este `header.payload.signature`, iar în varianta cifrată (JWE) sunt cinci asemenea părți [10]. RFC 7519 definește și un set de claim-uri standard, printre care `exp` (expiration), `iat` (issued at) și `nbf` (not before), pe care validatorul backend-ului trebuie să le verifice împreună cu semnătura. Avantajul direct pentru un API decuplat este că serverul nu mai trebuie să țină o tabelă de sesiuni. Pe frontend, `apiClient.js` interceptează automat răspunsurile 401 și apelează `refreshAccessToken()` înainte de a reîncerca cererea originală. În cea mai mare parte a timpului, utilizatorul nu observă că token-ul a expirat.

Pe lângă politica implicită Django (`MinimumLengthValidator` cu 4 caractere, `CommonPasswordValidator`, `NumericPasswordValidator`), am adăugat un validator propriu care impune cel puțin o literă majusculă:

```
1 class UppercaseValidator:
2     def validate(self, password, user=None):
3         if not re.search(r'[A-Z]', password):
4             raise ValidationError(
5                 "The password must contain at least one uppercase letter.",
6                 code='password_no_upper',
7             )
```

Fișierul `validators.py` este înregistrat în `AUTH_PASSWORD_VALIDATORS` din `settings.py`.

2.3.2 Controlul accesului pe roluri (RBAC)

Permisele sunt implementate ca clase DRF în `permissions.py` (Anexa A.3):

```
1 class IsOwner(permissions.BasePermission):
2     def has_permission(self, request, view):
3         return (
4             request.user and request.user.is_authenticated and
5             hasattr(request.user, 'profile') and
6             request.user.profile.role == Profile.ROLE_OWNER
7         )
8
9 class IsAdmin(permissions.BasePermission):
10     def has_permission(self, request, view):
11         if not (request.user and request.user.is_authenticated and
12                 hasattr(request.user, 'profile')):
13             return False
14         return request.user.profile.role in [Profile.ROLE_OWNER, Profile.ROLE_ADMIN]
```

`IsAdmin` acoperă atât `owner` cât și `admin`, ceea ce simplifică majoritatea verificărilor (există o singură ierarhie liniară: `owner > admin > user`). În views, verificarea apare în `perform_create`, `perform_update`, `perform_destroy` sau direct la începutul metodelor `post/patch`, pentru a refuza operațiunile cu 403 Forbidden.

2.3.3 Endpoint-uri principale

Mai jos sintetizez câteva dintre rutele cele mai relevante. Listă completă: `urls.py`.

GET `/api/system-status/`

Public. Returnează `{"is_fresh": <bool>}`. Folosit de setup wizard pentru a detecta o instalație goală.

POST `/api/register/`

Public. Creează un cont nou. Dacă este primul utilizator, devine `owner`.

POST `/api/login/`, **POST** `/api/token/refresh/`

Public. Emite token-urile JWT.

GET / PUT /api/me/

Autentificat. Citește și actualizează profilul. Acceptă `multipart/form-data` pentru upload de avatar.

GET / POST /api/devices/, GET / PATCH / DELETE /api/devices/<id>/

Autentificat. CRUD pentru dispozitive. La crearea unui dispozitiv, backend-ul încearcă auto-configurarea.

PATCH /api/devices/<id>/state/

Autentificat. Trimite o comandă către dispozitiv. Răspunde 202 `Accepted` înainte ca dispozitivul să confirme.

GET / POST / PUT / DELETE /api/rooms/

Citire publică pentru utilizatori autentificați; scriere doar pentru admin/owner.

GET /api/topology/

Autentificat. Returnează direct structura `nodes/edges` pe care React Flow o cere.

GET / PUT / DELETE /api/dashboard-layout/

Autentificat. Layout personal cu fallback la cel partajat.

PATCH /api/device-order/

Autentificat. Schimbă preferința de grupare pe dashboard (`room`, `type`, `status`, `name`, `custom`).

2.3.4 Auto-configurarea dispozitivelor

La crearea unui dispozitiv (`POST /api/devices/`), `DeviceListCreateView.perform_create` încearcă să spună microcontrolerului adresa IP a brokerului MQTT. Pașii sunt în `views.py:490-559`:

1. caută variabila de mediu `MQTT_BROKER_HOST`;
2. dacă nu există, extrage IP-ul gazdă din antetul `Host` al cererii;
3. dacă nici acesta nu este utilizabil (de exemplu `localhost`), deschide un socket UDP către dispozitiv și extrage adresa IP a interfeței prin care iese pachetul;
4. trimite un `POST http://<device_ip>/config` cu payload-ul `{"mqtt_server": <ip>}` și timeout 2 secunde.

Există și o protecție specifică Docker: dacă IP-ul detectat este într-o subrețea Docker (`172.x.x.x`) iar dispozitivul este în LAN (`192.168.x.x`), backend-ul renunță la auto-configurare și loghează un avertisment. Soluția este setarea explicită a variabilei `MQTT_BROKER_HOST` la IP-ul real al gazdei.

2.4 Daemonul MQTT (`mqtt_listener`)

Procesul care leagă lumea hardware de baza de date este implementat ca o comandă Django în `mqtt_listener.py` (Anexa A.4). Pornește un client `paho-mqtt`, se abonează la `homeforge/devices/+state`, intră într-o buclă infinită și verifică la fiecare 10 secunde dispozitivele care nu mai dau semne de viață.

2.4.1 Buclă principală și *watchdog*

```
1 def handle(self, *args, **options):
2     client = mqtt.Client()
3     client.on_connect = self.on_connect
4     client.on_message = self.on_message
5
6     client.connect("localhost", 1883, 60)
7     client.loop_start()
8
9     while True:
10         self.check_offline_devices()
11         time.sleep(10)
```

`check_offline_devices` este un *watchdog* aplicativ: orice dispozitiv cu status `online` și `updated_at` mai vechi de 30 de secunde este marcat `offline`. Pragul este aliniat cu intervalul de *heartbeat* al firmware-ului (5 secunde), oferind o marjă de 6 cicluri ratate înainte de a declara dispozitivul mort.

2.4.2 Auto-binding MAC–IP

Atunci când vine un mesaj nou, listener-ul ia adresa MAC din topic și caută un dispozitiv în baza de date. Dacă nu există încă un dispozitiv legat de acel MAC, încearcă să găsească unul după adresa IP raportată în payload:

```
1 device = Device.objects.filter(mac_address=device_mac_from_topic).first()
2
3 if not device and "ip" in data:
4     reported_ip = data["ip"]
5     device = Device.objects.filter(ip_address=reported_ip).first()
6     if device:
7         device.mac_address = device_mac_from_topic
8         device.save(update_fields=['mac_address'])
```

Mecanismul rezolvă un caz real: utilizatorul cunoaște IP-ul dispozitivului (l-a setat printr-un *Captive Portal* la prima pornire), dar nu și MAC-ul. Înregistrează dispozitivul cu IP-ul, iar prima dată când firmware-ul publică pe broker, listener-ul completează automat MAC-ul.

2.4.3 Re-mapare cheile firmware → widget-uri

Firmware-ul ESP32 publică datele cu nume „standard”: `temperature`, `humidity`, `pressure`, `relay_1`. Tipul de dispozitiv, însă, poate avea widget-uri cu identificatori generați dinamic, cum ar fi `temperature-1771357525497`. Metoda `map_standard_key` a listener-ului aplică o tabelă de potrivire pe baza tipului de widget:

```
1 def map_standard_key(self, device, key):
2     dev_type = device.device_type
3     template = dev_type.card_template
4     controls = template.controls.all()
5
6     target_types = []
7     if key == "temperature":
8         target_types = ['TEMPERATURE', 'GAUGE']
9     elif key == "humidity":
10        target_types = ['HUMIDITY', 'GAUGE']
11    elif key == "pressure":
12        target_types = ['PRESSURE', 'GAUGE']
13    elif key == "relay_1" or key.startswith("switch-"):
14        target_types = ['TOGGLE', 'BUTTON']
15
16    for control in controls:
17        if control.widget_type in target_types:
18            return control.variable_mapping
19    return None
```

Astfel, dacă șablonul vizual al unui termostat are un widget de tip `TEMPERATURE` cu `variable_mapping = "temperature-1771357525497"`, valoarea `"temperature": 23.5` primită de la firmware ajunge în `current_state` sub această cheie dinamică, iar widget-ul o afișează corect.

2.4.4 Comanda inversă (de la server către dispozitiv)

Atunci când un utilizator apasă un toggle pe interfață, browser-ul trimite un `PATCH /api/devices/<id>/state/`. Backend-ul folosește o instanță singleton de client MQTT (`mqtt_client.py`, Anexa A.5) pentru a publica payload-ul pe `homeforge/devices/<mac>/command`:

```
1 class MQTTClient:
2     _instance = None
3
4     def __new__(cls):
5         if cls._instance is None:
6             cls._instance = super(MQTTClient, cls).__new__(cls)
7             cls._instance.client = mqtt.Client()
8             cls._instance.connected = False
9             return cls._instance
10
11     def publish(self, device_id, command_data):
12         if not self.connected:
13             self.connect()
14         topic = f"{MQTT_TOPIC_PREFIX}/{device_id}/command"
15         payload = json.dumps(command_data)
16         self.client.publish(topic, payload)
17         logger.info(f"Published to {topic}: {payload}")
```

Aici, parametrul `device_id` primit de către `publish` este, de fapt, adresa MAC a dispozitivului. Am păstrat numele `device_id` pentru compatibilitate cu restul codului, însă identificatorul real în MQTT este adresa MAC. În `DeviceStateUpdateView.sync_with_hardware` (`views.py:617-658`), payload-ul este pre-procesat: dacă utilizatorul a trimis o valoare booleană fără cheie `relay_1`, backend-ul o injectează automat, astfel încât firmware-ul de tip releu să primească un mesaj pe care îl recunoaște.

2.5 Mecanismul de propunere a tipurilor de dispozitive

Una dintre ideile centrale ale platformei este să permită utilizatorilor să adauge tipuri proprii de dispozitive, fără să modifice codul backend-ului. Întregul flux trece prin patru stări și este orchestrat de `views.py` împreună cu modelul `CustomDeviceType`.

1. **Propunere.** Un utilizator obișnuit apelează `POST /api/device-types/propose/` cu o structură JSON care descrie atât topologia (microcontroler, senzori, pini) cât și `card_template` (widget-urile vizuale). `DeviceTypeProposeView` salvează tipul cu `approved=False` și apelează `Notification.notify_admins`.
2. **Revizuire.** Adminii văd propunerile prin `GET /api/admin/device-types/pending/`. Pentru o vizualizare separată a celor respinse, există `/admin/device-types/denied/`.
3. **Aprobare sau respingere.** `POST /api/admin/device-types/<id>/approve/` setează `approved=True` și invalidează cache-ul de tipuri aprobate. `POST /api/admin/device-types/<id>/deny/` cere obligatoriu un câmp `reason` și stochează motivul în `rejection_reason`, astfel încât utilizatorul să poată corecta propunerea.
4. **Curățare.** Tipurile respinse pot fi șterse individual sau în bloc prin `DELETE /api/admin/device-types/denied/` (cu sau fără listă de id-uri în body).

La nivelul stratului de acces, listarea tipurilor aprobate pentru utilizatorii fără privilegii administrative este servită dintr-un cache local cu durată de 5 minute (`LocMemCache`, configurat în `settings.py`). Cheia `approved_device_types_qs` este invalidată explicit după fiecare aprobare, respingere sau import în masă. Această optimizare este justificată de profilul de acces: lista tipurilor aprobate se modifică rar, dar este interogată frecvent de pagina principală.

2.5.1 Import/export și defaults

Pentru a nu obliga adminii să introducă manual fiecare tip de dispozitiv, există trei rute auxiliare:

GET / POST `/api/device-types/import-defaults/`

Încarcă tipurile predefinite din fișierul `api/fixtures/default_device_types.json`. Începând cu versiunea actuală a platformei, fișierul conține toate artefactele asociate fiecărui tip direct în interior: cod firmware, text al schemei electrice, documentație Markdown și imagini codificate `base64`, fără referințe la fișiere externe. Această decizie elimină dependența de structura unui director de proiect și simplifică distribuția: o singură resursă JSON, lipsită de fișiere însoțitoare, conține tot ce trebuie pentru a aproviziona o instanță proaspătă.

GET `/api/device-types/export/`

Exportă tipurile aprobate (sau o listă explicită prin parametrul `ids`) într-un fișier JSON. Toate imaginile sunt incluse `base64`-encodeate, astfel încât exportul este self-contained.

POST `/api/device-types/import/`

Importă un fișier JSON exportat anterior; dispozitivele cu nume existent sunt sărite tăcut.

2.5.2 Stocarea imaginilor direct în baza de date

O decizie deliberată pe care am luat-o este să stochez schemele electrice și imaginile din documentație ca `base64` în PostgreSQL, prin câmpurile `wiring_diagram_base64` și `documentation_images_base64` ale modelului `CustomDeviceType`. Avantajul principal este simplitatea operațională: backup-ul bazei de date conține tot ce trebuie, iar exportul/importul tipurilor de dispozitive nu depinde de existența unui director media partajat. Dezavantajul este creșterea bazei de date pentru imagini mari; din acest motiv, există o limită aplicată în view (`MAX_SIZE = 5 * 1024 * 1024` bytes, adică 5 MB) și o listă de tipuri MIME acceptate (PNG, JPEG, WEBP, eventual GIF pentru documentație).

Imaginile sunt servite înapoi prin endpoint-uri dedicate care decodează `base64` și răspund cu `HttpResponse` având `content_type` potrivit (`views.py:1013-1038`).

2.6 Alte comenzi de management Django

Pe lângă `mqtt_listener`, repository-ul mai include două comenzi Django utile pentru dezvoltare, ambele în `api/management/commands/`.

2.6.0.1 `load_default_types`.

Citește același fixture `api/fixtures/default_device_types.json` pe care îl folosește și endpoint-ul HTTP `import-defaults`, dar oferă un flag `--force` util în dezvoltare: șterge și recrează tipurile cu nume identic. Astfel pot itera rapid pe definițiile tipurilor predefinite

(relay, termostat DHT11, termostat BMP180, stație meteo combinată) fără să curăț manual baza de date.

2.6.0.2 monitor_devices.

O comandă moștenită din etapele timpurii ale proiectului, înainte de implementarea daemon-ului MQTT. Iterează prin toate dispozitivele din baza de date și, pentru fiecare, fie face un ping real (când IP-ul nu este în spațiul 192.168.1.1XX considerat „mock”), fie alege un status aleatoriu (online/offline/error) după o distribuție 80/10/10. A fost utilă pentru a popula dashboard-ul cu dispozitive „active” fără să flash-uiesc nimic. Acum nu mai este invocată automat, dar codul rămâne în repository pentru cei care vor să simuleze o rețea fictivă.

2.6.0.3 Configurația-cheie din settings.py.

Tot pentru completitudine, câteva detalii ale configurației Django pe care le-am atins doar în treacăt anterior:

- `REST_FRAMEWORK['PAGE_SIZE'] = 50`: paginare implicită pentru toate `ListAPIView`-urile (folosind `PageNumberPagination`), ceea ce previne răspunsuri mari când există multe dispozitive sau notificări;
- `CACHES` folosește `django.core.cache.backends.locmem.LocMemCache` cu `TIMEOUT=300` și `MAX_ENTRIES=1000`: suficient pentru o instanță locală, ușor de înlocuit cu Redis pentru producție;
- `AUTH_PASSWORD_VALIDATORS` include `api.validators.UppercaseValidator` alături de validatorii standard Django;
- `CORS_ALLOW_ALL_ORIGINS = True` și `CORS_ALLOW_CREDENTIALS = True` (prin `django-cors-headers`): permisiv pentru un demo, dar comentat pentru a fi restrâns la `http://localhost:3000` în producție;
- `INSTALLED_APPS` include `channels` (Django Channels): păstrat ca dependență pentru o viitoare implementare cu WebSocket-uri, dar momentan neutilizat;
- variabila de mediu `MQTT_BROKER_HOST` controlează IP-ul anunțat dispozitivelor în pasul de auto-configurare (vezi mai sus); când nu e setată, backend-ul îl deduce.

2.7 Integrarea sistemelor de panouri solare

Pe lângă dispozitivele ESP32 conectate prin MQTT, platforma include un modul dedicat pentru consumul de date de la sisteme externe de monitorizare fotovoltaică. Spre deosebire de dispozitivele DIY, sistemele solare comerciale (precum invertoarele Fronius, SolarEdge sau Huawei) expun deja propriile API-uri HTTP locale; HomeForge le tratează drept surse de telemetrie externă, fără a încerca să le înlocuiască firmware-ul. Codul aferent este izolat în pachetul `api/solar/`, cu modelul `SolarSystem` adăugat în `api/models.py` și vederile dedicate în `api/views_solar.py`.

2.7.1 Modelul SolarSystem

Modelul stochează minimul necesar pentru a putea reinteroga sistemul: numele afișat, URL-ul de bază al API-ului vendor (fără credențiale), un discriminator `provider` care selectează adaptorul la momentul interogării, un flag `enabled` și câmpuri auxiliare populate la momentul descoperirii (`api_version`, `capabilities`, `last_seen`).

```
1 class SolarSystem(models.Model):
2     PROVIDER_FRONIUS = 'fronius'
3     PROVIDER_CHOICES = [(PROVIDER_FRONIUS, 'Fronius Solar API V1')]
4
5     name = models.CharField(max_length=100)
6     base_url = models.URLField(max_length=500)
7     provider = models.CharField(max_length=30, choices=PROVIDER_CHOICES,
8                               default=PROVIDER_FRONIUS)
9     enabled = models.BooleanField(default=True, db_index=True)
10    api_version = models.CharField(max_length=20, blank=True, default='')
11    capabilities = models.JSONField(default=dict, blank=True)
12    last_seen = models.DateTimeField(null=True, blank=True)
13    user = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE)
14
15    class Meta:
16        unique_together = ['base_url', 'user']
```

Trei observații merită semnalate. Prima: nu se persistă nicio măsurătoare (P_PV, energie totală, stare de încărcare a bateriei) în baza de date; informația este obținută la fiecare cerere de la sursa originală, ceea ce evită replicarea unei serii de timp deja întreținute de inverter. A doua: constrângerea `unique_together = ['base_url', 'user']` împiedică același utilizator să înregistreze aceeași instalație de două ori. A treia: câmpul `capabilities` reține semnale obținute la descoperire (ex. `{"battery": true, "meter": true}`), folosite ulterior de UI pentru a decide dacă să afișeze cardul de baterie sau nodul de rețea.

2.7.2 Pattern-ul adapter / registry

Pentru a putea adăuga ulterior alte API-uri (Tesla, SolarEdge, Huawei, Enphase) fără a modifica restul backend-ului sau frontend-ului, modulul `api/solar/` folosește un pattern adapter / registry. Clasa abstractă `BaseSolarProvider` expune trei metode: `discover()` (validare conectivitate și extragere metadata), `get_overview()` (citirea snapshot-ului instantaneu) și `get_inverters()` (rezervată pentru o fază viitoare).

```
1 class BaseSolarProvider:
2     key: str = ''
3
4     def __init__(self, base_url: str):
5         self.base_url = base_url
6
7     def discover(self) -> dict: raise NotImplementedError
8     def get_overview(self) -> dict: raise NotImplementedError
9     def get_inverters(self) -> list[dict]: raise NotImplementedError
```

Prima implementare concretă, `FroniusProvider` (`fronius.py`, Anexa A.6), mapază răspunsurile API-ului oficial Fronius Solar V1 [11] (în special endpoint-ul `GetPowerFlowRealtimeData.fcgi`) pe schema normalizată. Pachetul Fronius are o anvelopă Head/Body standardizată, cu `Status.Code = 0` drept semnal de succes; câmpurile numerice sunt convertite atent prin `_to_float()`, pentru a păstra distincția între zero și valoare lipsă (`null`).

Registry-ul (`api/solar/registry.py`) este o tabelă simplă `provider_key → clasă adaptor`; adăugarea unui furnizor nou se reduce la implementarea unei subclase `BaseSolarProvider` și înregistrarea ei în `_PROVIDERS`.

2.7.3 Schema normalizată *overview*

Vederea `SolarOverviewView` (`GET /api/solar/systems/<id>/overview/`) este contractul stabil dintre backend și UI, independent de furnizor. Răspunsul include patru blocuri:

- **power** cu `solarW`, `gridW`, `loadW`, `batteryW`: valori instantanee în Watt, cu convenții de semn definite explicit (rețea pozitivă = import, baterie pozitivă = încărcare);
- **battery** cu `present`, `socPct`, `mode`;
- **energy** cu `todayWh`, `yearWh`, `totalWh` (toate cu `null` acolo unde furnizorul nu le expune; la modelele Fronius GEN24, de exemplu, contoarele cumulative nu sunt accesibile prin acest endpoint);
- **ratios** cu `selfConsumptionPct` și `autonomyPct`.

Pe lângă acestea, mai sunt câmpuri `provider`, `online`, `mode`, `timestamp` și `status` cu un mesaj uman-lizibil. Erorile previzibile (sistem dezactivat, API inaccesibil, sistem oprit) sunt convertite în răspunsuri 200 OK cu `online: false` și un mesaj corespunzător, astfel încât interfața să poată afișa o stare *offline* grafică fără a trebui să distingă codurile de eroare HTTP.

Cererile către vendor-ul real sunt protejate de o memorie cache aplicativă cu durată de viață scurtă (`OVERVIEW_CACHE_TTL = 8` secunde), necesară pentru a respecta rate-limit-ul Fronius și a deduplica cererile simultane de la mai mulți clienți care urmăresc același sistem. Frontend-ul efectuează *polling* la 5–10 secunde, iar cache-ul absoarbe interogările intermediare.

2.7.4 Poller-ul de fundal pentru *liveness*

Topologia globală (descrișă în secțiunea următoare) include nodurile solar ca prim-rang, iar statusul lor *online/offline* derivă dintr-un câmp `last_seen` actualizat la fiecare interogare reușită. Dacă utilizatorul nu vizitează pagina *Solar*, nicio cerere nu mai ajunge la vendor și `last_seen` devine învechit. Pentru a evita acest comportament, există o comandă Django dedicată, `python manage.py poll_solar --interval 30`, care iterează periodic prin sistemele activate, apelează `discover()` și stampilează `last_seen`. Intervalul implicit (30 s) este sub fereastra de liveness folosită de topologie (90 s), garantând că reprezentarea grafică reflectă starea reală indiferent de pagina vizualizată.

2.7.5 Endpoint-urile expuse

Modulul completează API-ul existent cu următoarele rute:

GET / POST `/api/solar/systems/`

Lista sistemelor înregistrate (orice utilizator autentificat) și înregistrarea unui nou (admin sau owner). La creare, backend-ul efectuează un `discover()` sincron; un sistem inaccesibil este respins cu 400 Bad Request și un mesaj explicit, evitând persistența unor înregistrări inutile.

GET / PUT / DELETE `/api/solar/systems/<id>/`

Operațiuni pe o resursă individuală. Câmpurile `api_version`, `capabilities` și `last_seen` sunt *read-only*, fiind gestionate intern.

GET `/api/solar/systems/<id>/overview/`

Snapshot-ul normalizat descris mai sus, cu cache de 8 secunde.

2.7.6 Endpoint-ul /widget-types/

Pentru a sincroniza paleta de widget-uri din device builder cu setul real recunoscut de backend, a fost introdus un endpoint auxiliar GET /api/widget-types/. Acesta returnează două liste: tipurile *supported* (efectiv procesate end-to-end: TOGGLE, SLIDER, BUTTON, GAUGE, TEMPERATURE, HUMIDITY, PRESSURE, POWER, BATTERY, STATUS) și tipurile *future*, marcate vizual ca rezervate (MOTION, LIGHT, CO2). Această separare a apărut din observația că widget-urile pentru mișcare, lumină ambientală și CO₂ fuseseră introduse în model înainte de a exista un firmware care să publice valorile corespunzătoare; păstrarea lor în model permite migrarea fluentă către o implementare ulterioară, fără a sparge tipurile de dispozitive deja propuse.

2.8 Vizualizarea topologiei

Endpoint-ul GET /api/topology/ (views.py:287-472) returnează direct formatul cerut de biblioteca React Flow, evitând orice procesare suplimentară pe frontend; rezultatul este vizualizarea radială din Figura 2.2. Backend-ul:

1. creează nodul central **homeforge-gateway** (brokerul MQTT);
2. adaugă fiecare dispozitiv ESP32 ca nod periferic;
3. adaugă sistemele solare înregistrate ca noduri distincte de tip **solar**, marcate cu flag-ul `is_solar: true` și cu o iconiță **fa-sun**; statusul lor online derivă din `last_seen` întreținut de poller-ul descris mai sus;
4. generează muchii între gateway și fiecare nod periferic, indiferent de tip;
5. alege culorile direct după statusul nodului: verde (#10B981) pentru online, roșu (#EF4444) pentru offline, galben (#F59E0B) pentru eroare;
6. marchează muchiile către nodurile online cu `animated=True`, ceea ce face ca React Flow să le deseneze cu o linie animată.

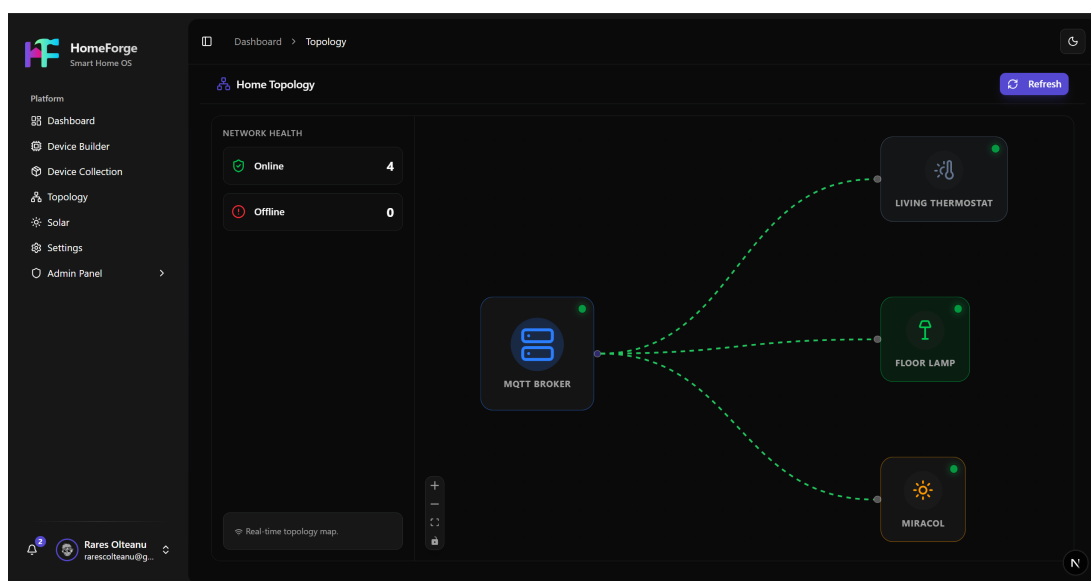


Figura 2.2: Vizualizarea radială a topologiei rețelei locale)

2.9 Optimistic UI pe partea de server

Endpoint-ul `PATCH /api/devices/<id>/state/` este construit explicit ca să nu blocheze browser-ul în așteptarea unei confirmări hardware. Logica este în `DeviceStateUpdateView.patch` (views.py:587-615):

```
1 def patch(self, request, pk):
2     try:
3         device = Device.objects.get(pk=pk)
4     except Device.DoesNotExist:
5         return Response({"detail": "Device not found."}, status=status.HTTP_404_NOT_FOUND)
6
7     new_state = request.data
8     if not isinstance(new_state, dict):
9         return Response({"detail": "State must be a JSON object."}, status=status.HTTP_400_BAD_REQUEST)
10
11     self.sync_with_hardware(device, new_state)
12
13     current_state = device.current_state or {}
14     optimistic_state = {**current_state, **new_state}
15
16     return Response({
17         "status": "Command sent",
18         "detail": "State will update when device confirms.",
19         "device_status": device.status,
20         "current_state": optimistic_state
21     }, status=status.HTTP_202_ACCEPTED)
```

Cele trei detalii importante:

1. `sync_with_hardware` doar publică pe MQTT, nu așteaptă răspuns. Confirmarea reală va veni asincron pe topicul de `state` și va actualiza baza de date prin `mqtt_listener`.
2. Răspunsul conține o stare *anticipată* (`optimistic_state`), pe care frontend-ul o folosește pentru a afișa imediat cum va arăta cardul dacă totul merge bine.
3. Codul HTTP 202 **Accepted** este alegerea semantic corectă: cererea a fost acceptată, dar procesarea efectivă nu s-a încheiat încă.

Pragurile clasice formulate de Nielsen în lucrarea sa despre *usability engineering* sunt aici de bun-simț: până la circa 0,1 secunde reacția este percepută ca instantanee, până la 1 secundă utilizatorul își păstrează firul gândirii, peste 10 secunde atenția se rupe [16]. Pentru un buton care comută un releu real prin Wi-Fi + MQTT, nu putem garanta cele 0,1 secunde; ce putem însă garanta este că *interfața* răspunde sub acest prag, iar contractul tehnic care permite asta este tocmai HTTP 202 cu starea anticipată.

2.10 Sincronizarea preferințelor de dashboard

Layout-ul dashboard-ului este salvat pe server pentru a fi coerent între dispozitivele utilizatorului (telefon, laptop). Două endpoint-uri lucrează împreună:

PUT `/api/dashboard-layout/`

Cu semantică **upsert**, înlocuiește layout-ul personal al utilizatorului curent. Serializer-ul (`serializers.py:387-519`) face o validare amănunțită: verifică versiunea (trebuie să fie 1), forma fiecărui item, dimensiunea folderelor (între 2 și 4 dispozitive), unicitatea `deviceId`-urilor și, opțional (dacă contextul conține `user`), existența efectivă a dispozitivelor referite.

PATCH `/api/device-order/`

Actualizează doar preferința de grupare (`room`, `type`, `status`, `name`, `custom`). Dacă utilizatorul nu are încă un layout personal, view-ul îl creează pornind de la cel partajat (sau

de la unul gol). Astfel, schimbarea grupării nu obligă utilizatorul să re-construiască întreg layout-ul.

Exemplu de payload acceptat de PUT:

```
1  {
2    "version": 1,
3    "items": [
4      { "type": "device", "deviceId": 5 },
5      {
6        "type": "folder",
7        "folderId": "folder-m1kx2j-a3b7",
8        "name": "Living Room",
9        "deviceIds": [1, 3]
10     }
11   ]
12 }
```


Capitolul 3

Aplicația web Next.js

3.1 Structura proiectului

Aplicația de frontend este construită cu Next.js 16 și React 19. Folosește *App Router*-ul Next.js, ceea ce înseamnă că rutele sunt definite implicit prin structura folderelor din directorul `app/`. Două rute sunt publice și separate de zona autenticată:

- `app/login/`: formular de autentificare;
- `app/register/`: formular de înregistrare;
- `app/setup/`: *setup wizard*-ul afișat când backend-ul răspunde cu `is_fresh: true`.

Restul aplicației trăiește sub `/dashboard/...` și folosește un layout comun definit în `layout.tsx`. Acesta verifică, la fiecare reîncărcare, profilul curent prin `fetchProfile()` și redirecționează către `/login` dacă răspunsul este 401. Tot el inserează `SidebarProvider`-ul, breadcrumb-urile dinamice și butonul de schimbare a temei (light/dark).

Sub `/dashboard` găsim:

- `page.tsx`: dashboard-ul principal cu cardurile dispozitivelor;
- `devices/`: lista detaliată a dispozitivelor;
- `device-builder/`: editorul vizual pentru tipuri de dispozitive (*device builder*);
- `device-collection/`: colecția de dispozitive, cu tipurile propuse de comunitate;
- `device-types/`: pagina cu tipurile de dispozitive disponibile;
- `topology/`: vizualizarea de topologie cu React Flow;
- `settings/`: pagina de setări utilizator;
- `admin/`: secțiune restricționată cu sub-paginile `approvals/`, `users/`, `rooms/`, `device-types/`, `debug/`.

Componentele reutilizabile sunt în `components/`, grupate pe domenii: `devices/` pentru cardurile de dispozitive, `device-builder/` pentru editorul vizual, `notifications/` pentru centrul de notificări, `setup/` pentru wizardul inițial, `topology/` pentru nodurile React Flow și `ui/` pentru primitive (Button, Card, Slider, etc.) generate prin `shadcn/ui`. Structura completă a directoarelor este redată în Figura 3.1.



Figura 3.1: Structura directoarelor aplicației Next.js (selecție a celor mai relevante rute și componente)

3.2 Stack-ul de interfață: shadcn/ui, Radix și Tailwind

Înainte de a intra în paginile concrete, este utilă o privire asupra straturilor de UI care le compun. HomeForge nu folosește o singură bibliotecă de componente „închisă” (în stilul Material UI sau Ant Design), ci o stivă compusă din trei nivele, fiecare cu un rol bine definit.

3.2.0.1 Tailwind CSS 4: sistemul de stiluri.

Tailwind este folosit drept singura unealtă de stilizare; nu există fișiere CSS scrise de mână (cu excepția lui `globals.css`, care declară doar variabilele de temă și câteva animații `@keyframes`). Tot ce înseamnă spațiere, tipografie, culori, breakpoints este exprimat prin clase utilitare direct în JSX. Decizia, pe lângă viteza de iterare, are un avantaj practic: un component poate fi mutat sau copiat fără să trebuiască să sincronizez și un fișier CSS separat.

3.2.0.2 Radix UI: primitivele de comportament.

Pentru toate elementele care au logică interactivă complexă (Dropdown, Dialog, Popover, Slider, Switch, Select, Tooltip, Tabs, Collapsible, Avatar, Scroll Area), HomeForge folosește primitivele `@radix-ui/react-*`. Acestea oferă comportamentul corect din punct de vedere accesibilitate (focus trap, navigare cu tastatura, atribute `aria-*`, gestiunea `Escape`), dar fără niciun stil impus, iar partea vizuală o pune Tailwind.

3.2.0.3 shadcn/ui: stratul de „componente proprii”.

Deasupra Radix-ului trăiește un al treilea nivel pe care îl materializează `shadcn/ui`. Acesta nu este o bibliotecă instalată ca dependență `npm`, ci un CLI care *copiază* cod TypeScript direct în repository-ul meu, în `components/ui/`. Configurația proiectului (`components.json`) declară stilul „new-york”, `rsc=true`, `tsx=true`, `baseColor=neutral`, `iconLibrary=lucide` și aliasurile de import `@/components`, `@/lib/Utils`, `@/components/ui`, `@/hooks`.

Diferența față de o bibliotecă tradițională este filozofică, dar contează: codul fiecărui `Button`, `Card`, `Dialog` *îmi aparține* și pot să-l modific fără să mă confrunt cu API-ul cuiva. Folderul `components/ui/` conține 29 de astfel de componente (printre ele `button.tsx`, `card.tsx`, `dialog.tsx`, `dropdown-menu.tsx`, `slider.tsx`, `sidebar.tsx`, `tabs.tsx`, `tooltip.tsx`, `command.tsx`, `markdown-editor.tsx`), toate generate inițial cu `shadcn/ui`, apoi adaptate la nevoile mele.

Pentru a vedea pe ce se sprijină acest model, exemplific cu `Button`, unde apar și `class-variance-authority` (cva) și utility-ul `cn` compus din `clsx` + `tailwind-merge`:

```
1 | const buttonVariants = cva("inline-flex items-center ...", {
2 |   variants: {
3 |     variant: { default: "...", destructive: "...", outline: "...",
4 |               secondary: "...", ghost: "...", link: "..."},
5 |     size: { default: "h-9 px-4 py-2", sm: "h-8 px-3",
6 |            lg: "h-10 px-6", icon: "size-9", ... },
7 |   },
8 |   defaultVariants: { variant: "default", size: "default" },
9 | })
```

`cva` permite definirea variantelor ca o tabelă declarativă, iar `cn` din `lib/Utils.ts` rezolvă coliziunile între clase Tailwind:

```
1 | import { clsx, type ClassValue } from "clsx"
2 | import { twMerge } from "tailwind-merge"
3 | export function cn(...inputs: ClassValue[]) {
4 |   return twMerge(clsx(inputs))
5 | }
```

Combinăția elimină categoria de bug-uri în care două clase Tailwind conflictuale (p-2 și p-4, de exemplu) ar fi aplicate amândouă într-o ordine nedeterminată.

3.2.0.4 Lucide React: iconițe.

Toate iconițele din aplicație vin din pachetul `lucide-react`, ales prin `components.json` (`"iconLibrary": "lucide"`). În `lib/icons.ts`, expun tot setul ca un map, ceea ce permite construirea componentei `IconPicker`, un `Popover` cu căutare în interiorul căruia utilizatorul poate alege orice icon Lucide pentru o cameră sau un dispozitiv:

```
1 | import * as LucideIcons from 'lucide-react';
2 | export const ICON_MAP = LucideIcons as unknown as Record<string, any>;
3 | export const ICON_OPTIONS = Object.keys(ICON_MAP).filter(key => {
4 |   return /^[A-Z]/.test(key) && key !== 'Icon' && typeof ICON_MAP[key] !== 'undefined';
5 | });
```

3.2.0.5 Sonner: sistemul de toast.

Pentru notificările inline (succes, eroare, info), folosesc biblioteca `sonner`, accesată prin singleton-ul `toast` din `components/ui/sonner.tsx`. Apelurile sunt scurte și consistente:

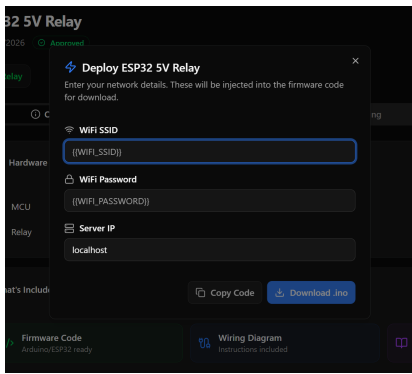
```
1 | toast.success("Layout set as default for all users");
2 | toast.error("Failed to sync device state", { description: err.message });
```

3.2.0.6 Framer Motion: animații.

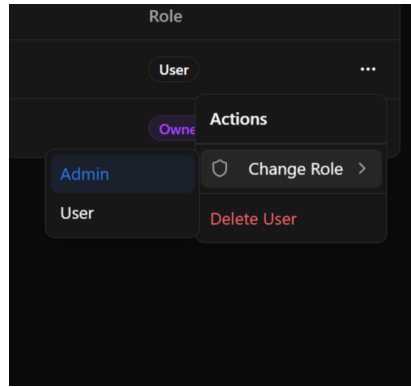
Animațiile (carduri care intră cu fade-in, folderul care se deschide cu `AnimatePresence`, etc.) sunt gestionate de `framer-motion`. În cardurile cu drag-and-drop folosesc `@dnd-kit/core`, `@dnd-kit/sortable` și `@dnd-kit/utilities`, toate trei făcând parte din aceeași familie de pachete.

3.2.0.7 Restul pieselor.

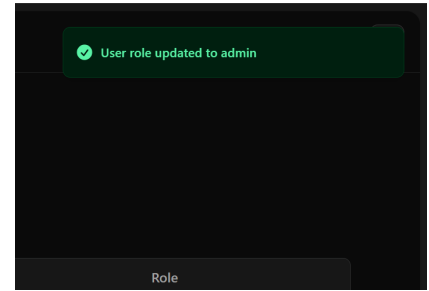
Pe lângă cele de mai sus, mai există câteva dependențe utilitare pe care le menționez în trecere: `date-fns` pentru manipulare de date și ore, `cmdk` pentru paleta de comandă, `prismjs` cu `react-simple-code-editor` pentru editor-ul de cod din device builder, `react-markdown` împreună cu `remark-gfm` și `rehype-highlight` pentru randarea documentației Markdown, iar `elkjs` este disponibilă pentru viitoarele layout-uri automate ale grafurilor (este în `dependencies`, dar nu o folosesc activ încă).



(a) Dialog de descărcare a firmware-ului



(b) Dropdown de schimbare a rolului



(c) Notificare toast

Figura 3.2: Primitive shadcn/ui în interfața HomeForge: (a) dialogul prin care, din colecția de dispozitive (*Device Collection*), utilizatorul introduce credențialele Wi-Fi și adresa IP a serverului pentru a descărca sketch-ul de firmware personalizat; (b) meniul *dropdown* cu submeniu pentru schimbarea rolului unui utilizator, din pagina de administrare; (c) notificarea *toast* oferită de biblioteca Sonner, afișată la finalizarea unei acțiuni.

3.3 Layout-ul aplicației: sidebar, breadcrumbs, header

Întreaga zonă autenticată folosește un layout comun `app/dashboard/layout.tsx`, care montează patru piese fixe: `AppSidebar`, butonul de `SidebarTrigger`, breadcrumb-urile dinamice și `ModeToggle`-ul pentru tema light/dark.

3.3.1 Sidebar-ul adaptiv

`components/app-sidebar.tsx` construiește meniul lateral pornind de la rolul utilizatorului curent. Lista de bază (vizibilă pentru toate rolurile) include intrările *Dashboard*, *Device Builder*, *Device Collection*, *Topology*, *Solar* și *Settings*. Pentru rolurile *admin* și *owner* se adaugă un submeniu „Admin Panel” cu link-uri către *Rooms*, *Users*, *Device Types* și *Debug*:

```

1  const navMain = React.useMemo(() => {
2    const items = [
3      { title: "Dashboard", url: "/dashboard", icon: LayoutDashboard, isActive: true },
4      { title: "Device Builder", url: "/dashboard/device-builder", icon: Cpu },
5      { title: "Device Collection", url: "/dashboard/device-collection", icon: Package },
6      { title: "Topology", url: "/dashboard/topology", icon: Network },
7      { title: "Solar", url: "/dashboard/solar", icon: Sun },
8      { title: "Settings", url: "/dashboard/settings", icon: Settings },
9    ];
10   if (isAdmin) {
11     items.push({
12       title: "Admin Panel", url: "/dashboard/admin", icon: Shield,
13       items: [
14         { title: "Rooms", url: "/dashboard/admin/rooms", icon: Home },
15         { title: "Users", url: "/dashboard/admin/users", icon: Users },
16         { title: "Device Types", url: "/dashboard/admin/device-types", icon: Cpu },
17         { title: "Debug", url: "/dashboard/admin/debug", icon: Bug },
18       ]
19     });
20   }
21   return items;
22 }, [isAdmin]);

```

Componenta de bază (`Sidebar`, `SidebarHeader`, `SidebarFooter`, `SidebarContent`) provine din shadcn/ui și expune un `SidebarProvider` care gestionează starea „colapsat / extins”. Componenta este responsivă. Pe ecrane mobile devine un `Sheet` care alunecă din lateral,

iar pe desktop poate fi redusă la o coloană de iconițe. Header-ul include logo-ul aplicației (componenta `HomeForgeLogo`, un SVG inline), iar footer-ul găzduiește componenta `NavUser` (avatar, nume, meniu cu opțiunea *Sign out*). Pe ecrane mobile, click-ul pe orice intrare apelează `useSidebar().setOpenMobile(false)`, închizând panoul lateral imediat după navigare, un comportament natural pentru utilizatorii care interacționează prin atingere. Figura 3.3 arată sidebar-ul cu submeniul *Admin Panel* extins.

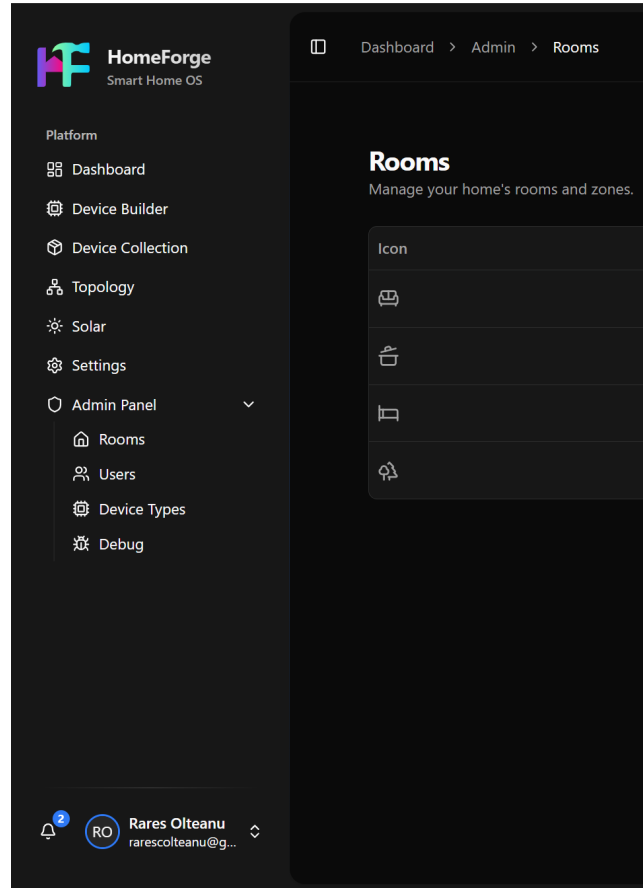


Figura 3.3: Sidebar-ul HomeForge cu sub-meniul *Admin Panel* activ

3.3.2 Breadcrumb-uri dinamice

Componenta `DynamicBreadcrumbs` citește path-ul curent prin `usePathname()` din `next/navigation` și îl segmentează automat. Asta înseamnă că la `/dashboard/admin/device-types?filter=pending` apar trei breadcrumb-uri (*Dashboard > Admin > Device Types*) fără ca eu să fi configurat ceva explicit. Capitalizarea, hyphen-to-space și link-urile către nivelurile părinte se calculează în timpul randării.

3.3.3 Tema și culorile de accent

Tema light/dark este oferită de pachetul `next-themes`, care setează un atribut `class` pe `<html>`. Butonul `ModeToggle` din header comută între cele două.

Pe lângă temă, fiecare utilizator își poate alege o *culoare de accent*. Valorile disponibile sunt declarate în `components/user-provider.tsx`:

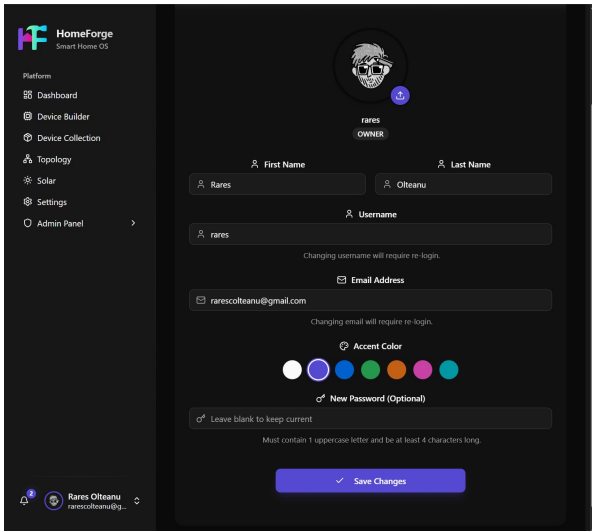
```
1 | export const ACCENT_COLORS = {
2 |   default: "oklch(0.205 0 0)",
3 |   violet:  "oklch(0.5 0.2 280)",
4 |   blue:    "oklch(0.5 0.2 250)",
```

```

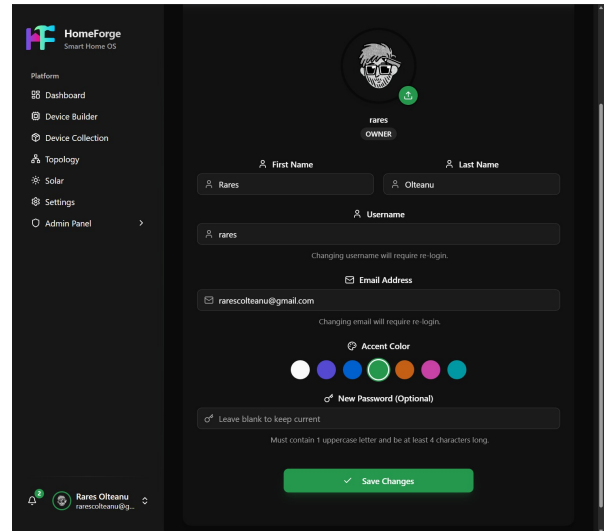
5 |   green:  "oklch(0.6 0.15 150)",
6 |   orange: "oklch(0.6 0.15 50)",
7 |   pink:   "oklch(0.6 0.2 340)",
8 |   cyan:   "oklch(0.6 0.15 200)",
9 | }

```

Sunt definite în spațiul de culoare OKLCH (preferabil față de RGB pentru consistență perceptivă), iar funcția `applyAccentColor` setează direct variabilele CSS `--primary` și `--ring` pe `document.documentElement`, fără reload. Preferința este persistată pe server (câmpul `accent_color` din `Profile`, vezi Capitolul 2) și replicată în `localStorage` pentru a evita „flash”-ul vizual la prima încărcare; un `useLayoutEffect` aplică culoarea înainte de paint.



(a) Accent violet



(b) Accent verde

Figura 3.4: Aceeași pagină *Settings* cu două *accent colors* diferite, ilustrând schimbarea instanțelor a culorii de accent prin variabilele CSS `--primary` și `--ring`, fără reîncărcarea paginii.

3.3.4 UserProvider: sursa unică de adevăr pe client

`components/user-provider.tsx` expune un Context React care:

- încarcă profilul curent la mount (`loadUser`): întâi din `localStorage` pentru a evita „flash of unauthenticated content”, apoi se actualizează cu răspunsul real al lui `fetchProfile()`;
- aplică culoarea de accent;
- gestionează logout-ul: golește cache-ul TanStack Query (`queryClient.clear()`), șterge token-urile, șterge layout-ul cache-uit local și redirecționează la `/login`.

Importanța pașilor de cleanup la logout nu este teoretică: fără `queryClient.clear()`, dacă un alt utilizator se loghează imediat după pe același browser, ar vedea pentru câteva secunde dispozitivele utilizatorului precedent.

3.4 Stratul de comunicare cu API-ul

Toate apelurile către backend sunt concentrate în fișierul `apiClient.js` (reprodus integral în Anexa B.1), scris în JavaScript pur. Alegerea de a nu folosi TypeScript aici a fost deliberată. Codul rămâne la nivelul unui apel `fetch` standard, ușor de urmărit de oricine cunoaște API-ul `fetch` al browserelor.

3.4.1 Determinarea URL-ului backend-ului

Frontend-ul nu cunoaște la build-time IP-ul gazdei pe care va rula. Funcția `getBackendUrl` folosește locația curentă a browserului și înlocuiește portul cu 8000:

```
1 export function getBackendUrl() {
2   if (typeof window !== 'undefined') {
3     return `${window.location.protocol}//${window.location.hostname}:8000`;
4   }
5   return 'http://localhost:8000';
6 }
```

Practic, dacă un utilizator accesează aplicația de pe `http://192.168.1.10:3000`, cererile vor merge automat către `http://192.168.1.10:8000`, fără configurări suplimentare.

3.4.2 Gestiunea token-urilor JWT

La login, token-ul de acces și cel de refresh sunt stocate în `localStorage`. Toate cererile autentificate trec prin `fetchWithAuth`, care:

1. adaugă antetul `Authorization: Bearer <token>`;
2. dacă răspunsul este 401, apelează `refreshAccessToken()` și reîncearcă cererea o singură dată;
3. dacă reîmprospătarea eșuează, șterge token-urile și aruncă o eroare cu `status = 401`, care este capturată de `dashboard/layout.tsx` și care redirecționează către `/login`.

```
1 async function fetchWithAuth(url, options = {}) {
2   const headers = { ...options.headers, ...getAuthHeaders() };
3   let res = await fetch(url, { ...options, headers });
4   if (res.status === 401) {
5     const newToken = await refreshAccessToken();
6     const newHeaders = { ...options.headers, Authorization: `Bearer ${newToken}` };
7     res = await fetch(url, { ...options, headers: newHeaders });
8   }
9   return res;
10 }
```

3.4.3 Tratarea uniformă a erorilor

Funcția `handleApiError` parsează formatul standard al erorilor returnate de Django REST Framework. Aceasta gestionează trei cazuri: un câmp `detail` (de obicei la erori de permisiune), un câmp `non_field_errors` (la validări la nivel de serializer) și restul, în care fiecare cheie este un câmp specific cu o listă de mesaje. Mesajele sunt concatenate și aruncate ca un singur `Error`, pe care UI-ul îl afișează prin `toast.error`.

3.5 Gestiunea stării: TanStack Query

HomeForge folosește *TanStack Query* (denumit anterior *React Query*) pentru toate datele la nivel de aplicație. `QueryProvider` este montat în root-ul aplicației (`components/query-provider.tsx`) și expune un singur `QueryClient`. Fiecare componentă apoi consumă două hook-uri: `useQuery` pentru fetch-uri (lista de dispozitive, profilul, notificările) și `useMutation` pentru scrieri (toggle, update, delete). Două decizii de configurare merită evidențiate. Lista de dispozitive folosește `refetchInterval: 3000`, adică polling la 3 secunde, în locul unei conexiuni WebSocket. Contorul de notificări necitite folosește `refetchInterval: 30000`, adică 30 de secunde: suficient pentru un badge, fără a încărca inutil backend-ul.

3.6 Dashboard-ul principal

Pagina `app/dashboard/page.tsx` orchestrează trei surse de date (dispozitive, tipuri de dispozitive și camere) și le combină într-un singur grid. Există cinci moduri de grupare expuse într-un `<Select>`:

- `all`: toate dispozitivele, în ordinea aleasă de utilizator (mod cu drag-and-drop);
- `room`: grupare pe cameră;
- `type`: grupare pe tipul de dispozitiv;
- `status`: grupare pe `Online / Offline / Error`;
- `name`: sortare alfabetică simplă.

La nivel de cod, modul curent (`viewMode`) este oglindit într-o variabilă `deviceOrder` pe care `useDashboardLayout` o sincronizează cu serverul prin endpoint-ul `PATCH /api/device-order/`. Preferința utilizatorului supraviețuiește între dispozitive.

3.6.1 Hook-ul `useDashboardLayout`

Logica de layout este izolată într-un singur hook (`hooks/useDashboardLayout.ts`, Anexa B.3). Acesta:

1. descarcă layout-ul curent prin `useQuery({ queryKey: ['dashboardLayout'], queryFn: fetchDashboardLayout, staleTime: Infinity })`;
2. dacă API-ul eșuează (mod offline), cade pe `localStorage` ca rezervă;
3. *reconciliază* layout-ul cu lista actuală de dispozitive: elimină referințele către cele șterse, adaugă cele noi, dizolvă folderele rămase cu mai puțin de 2 elemente;
4. expune funcții de mutație: `moveItem`, `createFolder`, `addToFolder`, `removeFromFolder`, `renameFolder`, `dissolveFolder`, `resetLayout`, `revertToShared`;
5. salvează modificările pe server cu `debounce` de 800 ms și *flush* imediat la ieșirea din modul de editare („Done”).

```
1  const scheduleSave = useCallback((newLayout: DashboardLayout) => {
2    saveLayout(newLayout); // localStorage instantaneu
3    if (saveTimerRef.current) clearTimeout(saveTimerRef.current);
4    hasPendingSaveRef.current = true;
5    saveTimerRef.current = setTimeout(() => {
6      saveToApi(newLayout); // API după 800 ms
7    }, 800);
8  }, [saveToApi]);
```

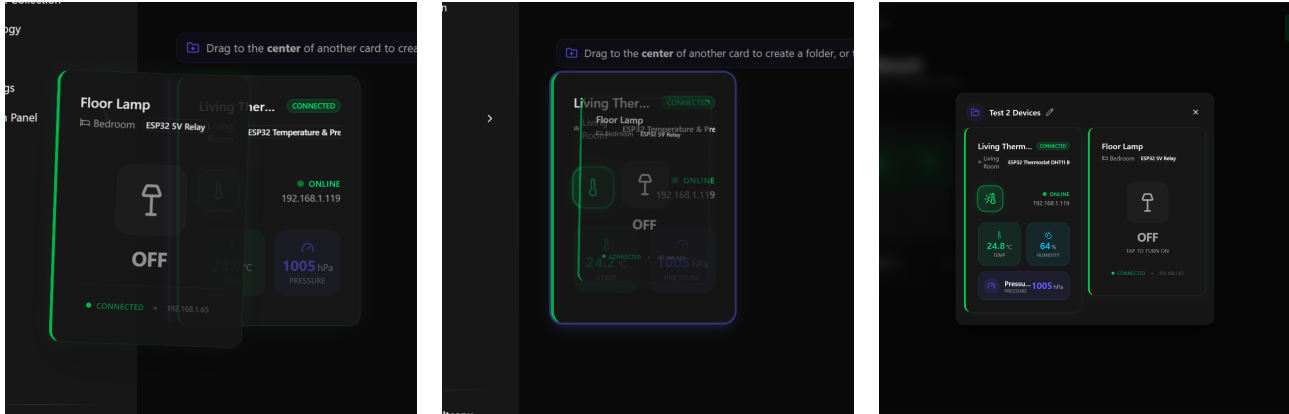
3.6.2 Drag-and-drop și foldere

Componenta `DraggableDeviceGrid` folosește biblioteca `@dnd-kit/core` pentru drag-and-drop. Fiecare card este învelit într-un `SortableGridItem` cu trei zone de drop:

- zona *stângă*: dispozitivul tras este inserat înaintea țintei;
- zona *dreaptă*: este inserat după;

- zona *mijloc*: dispozitivele se combină într-un folder nou (sau dispozitivul este adăugat în folderul țintă).

Constrângerile semantice (un folder are între 2 și 4 dispozitive, fiecare `deviceId` apare cel mult o dată) sunt verificate atât pe client cât și pe backend. Cele trei zone de drop sunt ilustrate în Figura 3.5.



(a) Drop lateral (reordonare)

(b) Drop central (combinare)

(c) Folderul rezultat

Figura 3.5: Crearea unui folder prin drag-and-drop în modul de editare al dashboard-ului: (a) cardul tras lateral declanșează indicatorul de reordonare (stânga/dreapta); (b) suprapunerea a două carduri activează zona de mijloc, care le combină într-un folder nou; (c) folderul rezultat, cu butonul-creion pentru redenumire.

3.7 Cardurile inteligente: SmartDeviceCard

Componenta `SmartDeviceCard` (reprodusă integral în Anexa B.2) este probabil cea mai densă din întregul frontend. Rezolvă în același loc trei probleme grele: randarea dinamică a unui număr variabil de widget-uri, optimistic UI și combaterea *glitch*-urilor de stare la apăsări rapide.

3.7.1 Randarea condusă de `card_template`

Fiecare tip de dispozitiv definește o listă de `controls`, în care fiecare element are un `widget_type`. Componenta apelează funcția internă `renderWidget(control, index)`, care selectează implementarea vizuală corespunzătoare: `Switch` pentru `TOGGLE`, `Slider` pentru `SLIDER` sau widget-uri specializate definite în `SensorWidgets.tsx` (`TemperatureWidget`, `HumidityWidget` etc.). Fiecare widget primește valoarea curentă prin câmpul `variable_mapping`, mecanism care permite aceleași componente să servească orice tip de dispozitiv definit prin device builder.

Layout-ul cardului are două variante: *row* (widget-uri pe linie, util pentru dispozitive simple) și *square* (widget-uri pătrate, util pentru senzori). Layout-ul de grid se citește dintr-un câmp `layout_config` cu chei `w` (numărul de coloane) și `gap`; componenta convertește `layout_config.w` sau `layout_config.columns` în clase Tailwind (`grid-cols-1/2/3`).

Există și un caz special: `isSingleToggleDevice(controls)`. Dacă un dispozitiv are exact un singur control de tip `TOGGLE` și nimic altceva (un releu pentru un bec, de exemplu), cardul devine *tap-to-toggle*: întreaga suprafață a cardului acționează ca un switch, fără un widget separat. Detaliul îmbunătățește vizibil ergonomia pe mobil.

3.7.2 Pattern-ul *Optimistic UI*

Această subsecțiune descrie mecanismul central de gestiune a stării interactive. La apăsarea unui toggle, componenta:

1. construiește noua stare locală `newState = {...currentState, [variable]: value}`;
2. o salvează imediat în `useState`, deci UI-ul se actualizează în 0 milisecunde;
3. programează un `setTimeout` de 50 ms (pentru toggle, ca să consolideze apăsările repetate) sau 500 ms (pentru slider, ca să nu trimită zeci de cereri per gest), un *debounce* clasic;
4. după timer, declanșează mutația TanStack Query, care apelează `updateDeviceState(id, newState)`.

Mutația utilizează patru dintre hook-urile expuse de `useMutation`: `mutationFn`, `onMutate`, `onError` și `onSettled`. Hook-ul `onSuccess` este omis deliberat, din motive explicate mai jos. Scheletul implementării, în formă condensată:

```
1  const updateMutation = useMutation({
2    mutationFn: ({ id, newState }) => updateDeviceState(id, newState),
3    onMutate: async ({ id, newState }) => {
4      await queryClient.cancelQueries({ queryKey: ['devices'] });
5      const previousDevice = queryClient.getQueryData(['device', id]);
6      queryClient.setQueryData(['device', id], (old) => ({ ...old, current_state: newState }));
7      return { previousDevice }; // pasat la onError pentru rollback
8    },
9    onError: (_e, _v, ctx) => { /* rollback la ctx.previousDevice + toast.error */ },
10   onSettled: () => { /* setIsRefetching(true); invalidate cu delay; timeout siguranță 5s */ },
11 });
```

Câteva detalii subtile pe care le-am implementat după mai multe iterații:

- `onMutate` salvează starea anterioară într-un *context* care, în caz de eroare, este folosit pentru *rollback*. Fără aceasta, dacă dispozitivul este offline, butonul ar rămâne blocat în starea optimistă incorectă.
- Intenționat *nu* folosesc `onSuccess` pentru a suprascrie starea locală: serverul răspunde imediat cu HTTP 202 și o stare anticipată, dar starea reală vine asincron, de pe MQTT. Dacă aș suprascrie pe `onSuccess`, butonul ar putea „sări” înapoi câteva secunde mai târziu.
- `onSettled` marchează `isRefetching = true`, ceea ce face ca UI-ul să afișeze un `Loader2` suprapus. Există un timeout de siguranță (5 secunde) care anulează această stare dacă backend-ul nu mai răspunde, fiindcă nu vrem ca utilizatorul să rămână blocat cu un *spinner* infinit.
- Există un `useEffect` care detectează când starea de pe server „a ajuns din urmă” starea optimistă (toate cheile modificate sunt egale) și, în acel moment, oprește `isRefetching`, iar vizual butonul rămâne în stare „sincronizare” doar atât timp cât chiar este o discrepantă reală între ce vrem și ce raportează firmware-ul.

Cronologia completă a acestor pași, de la apăsare până la confirmarea de pe MQTT, este redată în Figura 3.6.

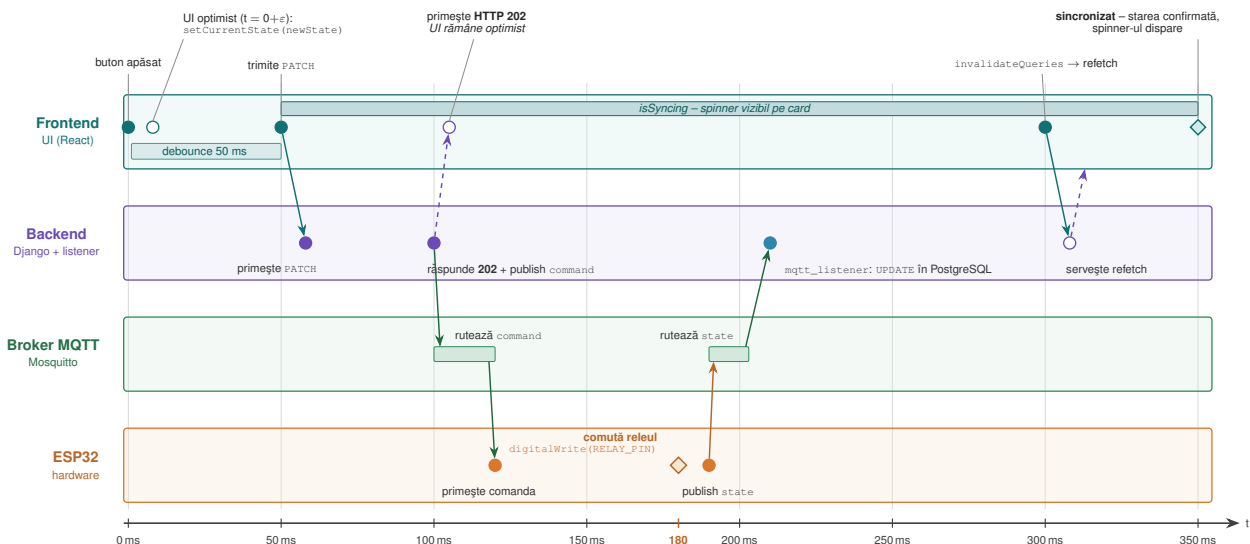


Figura 3.6: Diagrama temporală a unei apăsări de toggle în interfața HomeForge

3.8 Centrul de notificări

`notification-center.tsx` este componenta din colțul dreapta-sus al headerului. Este un Popover care afișează lista alertelor curente:

- folosește două `useQuery`: unul pentru contorul de necitite, cu `refetchInterval: 30000`, și unul pentru lista completă, activat doar cât timp popover-ul e deschis (`enabled: open`);
- iconițele și culorile sunt alese în funcție de `notification_type` (`device_offline`, `device_type_pending`, etc.);
- `action_url`-ul venit de la backend este normalizat printr-o funcție `getActionUrl` care adaugă prefixul `/dashboard` și redirecționează URL-urile către individuale (`/admin/device-types/22`) către pagina-listă cu filtru (`/dashboard/admin/device-types?filter=pending`).

3.9 Vizualizarea topologiei

Pagina `/dashboard/topology/` se bazează pe biblioteca `@xyflow/react` (React Flow v12). Datele vin direct de la endpoint-ul `GET /api/topology/`, în formatul cerut nativ de React Flow (`nodes[]` și `edges[]`). Logica frontend este în `TopologyCanvas.tsx` și are două componente:

1. Un algoritm propriu de layout ierarhic care plasează gateway-ul în stânga și dispozitivele într-o coloană verticală în dreapta, cu `childXOffset = 450` și `childSpacingY = 180` pixeli. Renunțarea la un algoritm generic precum ELK sau Dagre este motivată de profilul tipic al rețelei vizualizate: pentru un număr de dispozitive cuprins între 5 și 30, o dispunere în coloană oferă cea mai bună lizibilitate.
2. Un mecanism de păstrare a selecției: utilizatorul poate selecta unul sau mai multe noduri, iar la re-randarea automată (pagina de topologie își reîmprospătează datele printr-un interval propriu de 5 secunde), un `useRef` numit `selectionRef` reține ce a fost selectat și restaurează starea pe noile noduri.

Nodurile sunt randate de patru componente custom în `topology/nodes/`: `BuilderStyleNode`, `GlassDeviceNode`, `TopologyBuilderNode`, `UnifiDeviceNode`. `TopologyBuilderNode` este cea folosită în mod implicit; celelalte sunt variante stilistice între care utilizatorul poate alege.

Începând cu versiunea care include modulul solar, `TopologyBuilderNode` discriminează între categorii de noduri pe baza câmpurilor `type` și `is_solar`. Sistemele fotovoltaice apar ca noduri distincte cu o iconiță soare (`Sun` din `lucide-react`), pe o paletă ambră, în timp ce dispozitivele MCU păstrează stilul deja existent. Detecția cade înapoi pe analiza câmpurilor `device_type` și `icon` (`fa-sun`, `fa-solar-panel`) pentru a acoperi sistemele importate înainte de adăugarea câmpului explicit `is_solar`.

3.10 Pagina de monitorizare solară

Modulul fotovoltaic dispune de o pagină dedicată la `/dashboard/solar/`, accesibilă tuturor rolurilor autentificate. Aceasta consumă API-ul descris în Capitolul 2 (în special `GET /api/solar/systems/<id>/overview/`) și este alcătuită din componente specializate, izolate sub `components/solar/` (Figura 3.7).

3.10.1 Cicluri de *polling* și gestionarea stării

Pagina folosește `useQuery` pentru a interoga endpoint-ul `overview` la fiecare 5 secunde (`refetchInterval: 5000`), cu pauză automată când tab-ul browserului trece în fundal. Această frecvență este sub-limita cache-ului aplicativ de 8 secunde de pe server, deci traficul efectiv către vendor rămâne sub o cerere la fiecare 8 secunde, chiar dacă mai mulți clienți vizualizează pagina simultan. Statele tranzitorii (lipsa unui sistem înregistrat, sistem offline, sistem dezactivat) sunt acoperite de componente dedicate (`SolarEmptyState`, banner pentru offline derivat din `status.message`), astfel încât interfața să rămână informativă fără a afișa erori brute.

3.10.2 Diagrama de flux de putere

`PowerFlowDiagram.tsx` construiește o reprezentare animată SVG, inspirată de interfețele Fronius SolarWeb și Tesla. Patru noduri periferice (PV, rețea, consum casnic, baterie) sunt distribuite în jurul unui nod central reprezentând invertorul. Direcțiile fluxurilor sunt deduse din semnele câmpurilor `power.solarW`, `power.gridW` și `power.batteryW` prin funcția `deriveFlows` din `components/solar/solar-utils.ts`; separat, câmpul `power.loadW` decide direct dacă latura de consum casnic e activă. Nodul de baterie este afișat doar atunci când câmpul `battery.present` este adevărat. Pragul de inactivitate este de 1 W per latură, valori sub acest prag determinând estomparea nodurilor corespunzătoare și o animație discretă de tip „respirație” (`solar-idle-breathe`) care comunică starea de repaus fără a sugera mișcare. Toate dimensiunile sunt parametrizate prin constantele `NODE_R` și `HUB_R`.

3.10.3 Graficul de putere în timp real

Componenta `LivePowerChart.tsx` acumulează în memoria sesiunii eșantioanele primite de la `overview`, oferind o vedere de tip serie de timp pe orizont scurt. Deoarece API-ul Fronius nu expune un endpoint dedicat de istoric agregat fiabil pe toate modelele (în special pe GEN24, unde contoarele cumulative pot fi indisponibile), graficul afișează exclusiv intervalul „de când utilizatorul a deschis pagina”, iar o legendă explicită îl semnalează („*Data collected since this*

page was opened”). Graficul este construit peste **recharts**, integrat printr-un wrapper **shaden** (`components/ui/chart.tsx`) cu suport pentru tooltip-uri și legende personalizate.

3.10.4 Cardurile cu indicatori

`SolarStatCards.tsx` prezintă valorile instantanee numerice: producție PV, consum, schimb cu rețeaua, putere baterie, stare de încărcare (SOC), procente de auto-consum și autonomie energetică. Câmpurile lipsă (`null`) sunt randate ca o liniuță, evitând afișarea unor valori 0 înșelătoare. Indicatorii cumulativi (energie produsă astăzi / anul curent / total) au fost eliminați din varianta curentă. Valorile primite de la API-ul Fronius pe modele GEN24 s-au dovedit nepopulate, frecvent `null` sau resetate la zero. Rămânerea exclusiv la valorile instantanee asigură consistența indiferent de modelul de invertor.

3.10.5 Dialogul administrator și starea „gol”

Pentru rolurile cu privilegii de administrare, butonul „Add solar system” deschide componenta `SolarSystemDialog.tsx`, un `Dialog` cu câmpurile *name*, *API URL* și *provider*. Validarea conectivității se realizează server-side: backend-ul efectuează un apel `discover()` înainte de a accepta înregistrarea, iar mesajele de eroare (URL inaccesibil, Solar API dezactivat pe invertor) sunt surfasate către utilizator printr-un `toast.error`. Componenta `SolarEmptyState.tsx` este afișată atunci când nu există încă niciun sistem înregistrat, oferind un buton CTA pentru utilizatorii admin și un mesaj pasiv pentru utilizatorii obișnuiți.

3.10.6 Pasul „Solar” din setup wizard

La înregistrarea inițială a platformei, asistentul de configurare include un pas opțional `SolarStep.tsx`, inserat în `SetupWizard.tsx` între *devices* și *complete*. Pasul oferă două opțiuni: *Skip* (continuă fără înregistrare) sau introducerea unui sistem Fronius prin câmpurile nume / URL / provider, urmate de același apel `createSolarSystem` folosit și de dialogul administrator. La final, ecranul de confirmare prezintă un rezumat care include statutul *solar connected*, dacă pasul a fost completat.

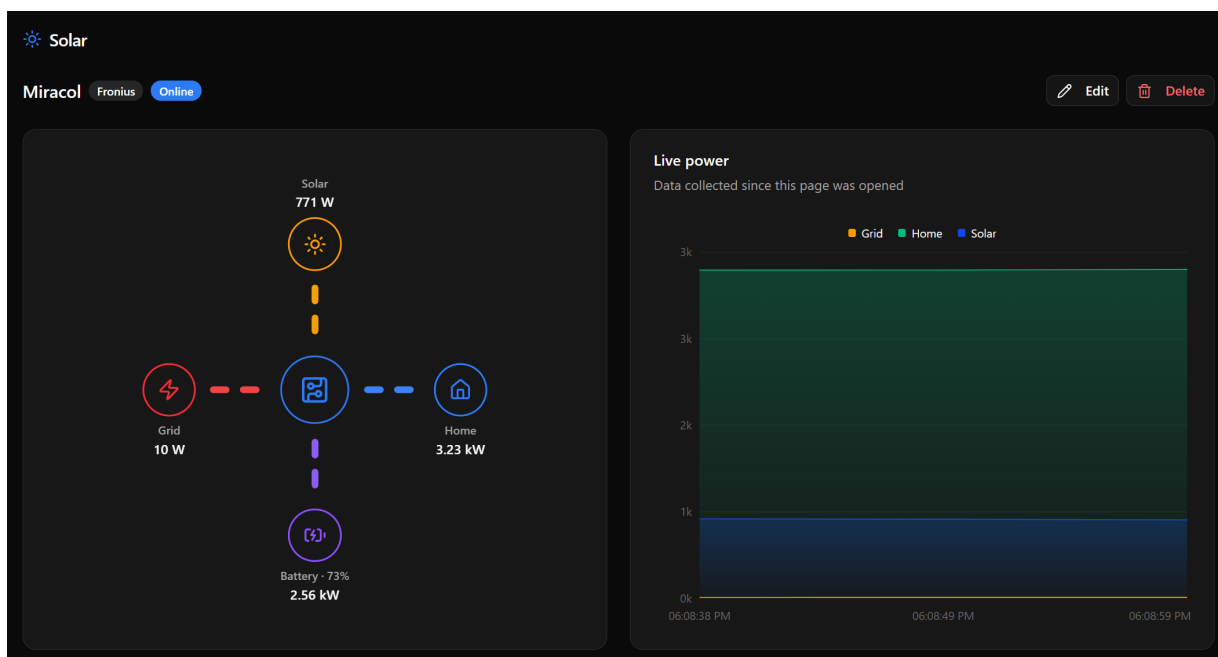


Figura 3.7: Pagina *Solar*: diagramă de flux de putere, indicatori și grafic în timp real

3.11 Editorul vizual de tipuri de dispozitive (*device builder*)

Una dintre cele mai interesante pagini este `device-builder/DeviceUICreator.tsx` (Anexa B.4), un editor vizual cu două panouri (Figura 3.8):

- în stânga, un *node graph* (bazat tot pe React Flow) unde utilizatorul adaugă microcontrolere și senzori, conectându-i între ei. Fiecare nod are un ID unic care va fi referențiat din widget-urile vizuale;
- în dreapta, lista *widget-urilor* (toggle, slider, sensor) care vor compune cardul dispozitivului. Fiecare widget este legat de un nod prin câmpul `variable_mapping`.

În jurul editorului există trei panouri auxiliare:

- `FirmwareCodeEditor.tsx`: editor de cod Arduino bazat pe `prismjs` și `react-simple-code-editor`;
- `WiringDiagramEditor.tsx`: încărcare de schemă electrică (image) și descriere textuală;
- `DocumentationEditor.tsx`: editor Markdown cu randare `react-markdown` (cu `remark-gfm` și `rehype-highlight`).

La finalul fluxului, utilizatorul apasă *Propose* și aplicația apelează `POST /api/device-types/propose/` cu o structură JSON compusă din:

- `name`: denumirea tipului;
- `definition.structure[]`: noduri și conexiuni (microcontroler, senzori, pini, fire);
- `card_template.controls[]`: widget-urile vizuale;
- `firmware_code`, `wiring_diagram_text`, `documentation`: text;
- `wiring_diagram_base64`, `documentation_images_base64`: imagini encodeate.

Validatorul de backend (`CustomDeviceTypeSerializer.validate`) verifică explicit că fiecare `variable_mapping` din widget-uri corespunde unui ID definit în `structure[]`. Astfel, este imposibil să propui un tip de dispozitiv „rupt”.

3.11.1 Paleta de componente: *supported* vs. *future*

Pentru ca interfața să reflecte fidel ce widget-uri sunt cu adevărat procesate end-to-end (firmware → MQTT → backend → card), paleta este împărțită în două liste hardcodate în `DeviceUICreator.tsx`, ținute manual sincronizate cu setul suportat de backend descris în Capitolul 2 (endpoint-ul `GET /api/widget-types/` există, dar nu e consumat încă din acest editor). Tipurile *supported* pot fi plasate liber pe canvas, în timp ce tipurile *future* (`MOTION`, `LIGHT`, `CO2`) sunt randate ca plăci estompate, marcate cu un badge „Soon” și inactive la *drag-and-drop* sau *click*, generând un mesaj `toast` explicativ. Aceeași logică se aplică și în paleta de componente (microcontroler, senzori, releu) din panoul de topologie: doar tipurile cu suport firmware complet sunt selectabile. Senzorul de presiune (BMP180) a fost recent promovat din *future* în *supported*, odată cu integrarea citirilor de presiune barometrică în firmware-ul ESP32.

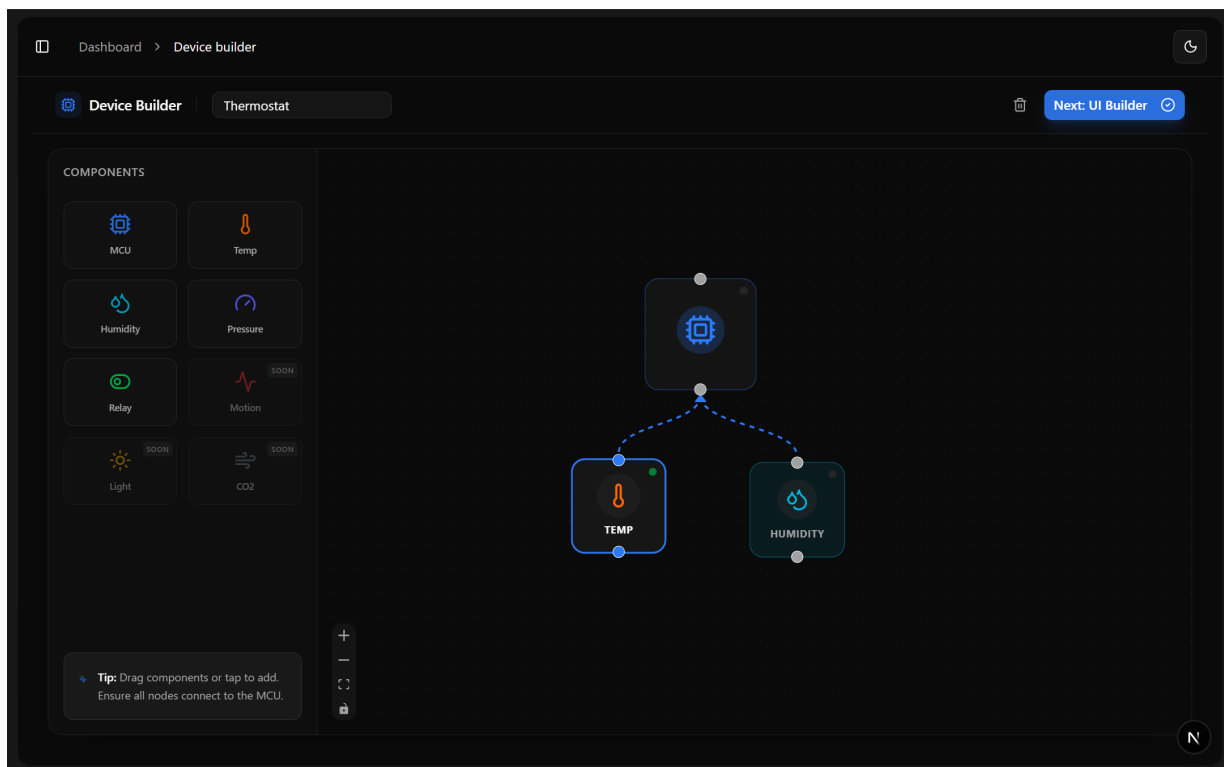


Figura 3.8: Device builder-ul HomeForge

3.12 Setup wizard-ul

La prima vizită pe o instanță proaspătă, `SetupWizard.tsx` ghidează utilizatorul printr-un flux în 6 pași: *welcome* → *account* → *rooms* → *devices* → *solar* → *complete*. Pașii sunt componente separate în `components/setup/steps/` și comunică prin `onNext` / `onBack` cu containerul. La pasul *account*, formularul trimite explicit `role: 'owner'` către `POST /register/`, valabil pentru primul cont creat pe o instanță proaspătă. La pasul *devices*, utilizatorul poate importa tipurile predefinite prin endpoint-ul `POST /api/device-types/import-defaults/`. Pasul *solar* este complet opțional (utilizatorii fără sistem fotovoltaic îl sar prin butonul *Skip*); detaliile sunt descrise în secțiunea dedicată mai sus.

Indicatorul de progres este o linie întreruptă în segmente egale (câte unul per pas, fără cele de început și final), iar fiecare segment primește clase Tailwind diferite pe baza indexului pasului curent. Animațiile de tranziție folosesc `animate-in fade-in duration-300`, o clasă utilitară oferită de pluginul `tw-animate-css`.

3.13 Onboarding-ul: OnboardingChecklist și PageTooltip

Asistentul de configurare inițial (*setup wizard*) acoperă doar primii pași de instalare. Pentru utilizatorii care accesează aplicația ulterior, este oferit un mecanism de onboarding suplimentar, mai puțin intruziv, format din două componente complementare situate în `components/onboarding/` (Figura 3.9).

3.13.0.1 Checklist-ul de pe dashboard.

Componenta `OnboardingChecklist.tsx` este un card afișat în partea superioară a dashboard-ului, vizibil până la bifarea tuturor pașilor din listă: crearea contului, adăugarea primului dispozitiv, explorarea Device Builder-ului, verificarea topologiei rețelei și personalizarea profilului. Pentru rolurile `admin/owner` se adaugă și definirea camerelor și importul tipurilor de dispozitive. Verificarea condițiilor utilizează contoarele primite prin `props`, cu fallback către apelurile `fetchRooms()`, `fetchDeviceTypes()` și `fetchDevices()`; pașii satisfăcuți sunt marcați automat ca *completați*. Pașii suplimentari sunt persistați în `localStorage` sub cheia `onboarding-checklist-completed`, iar butonul *Dismiss* setează flag-ul `onboarding-checklist-dismissed = true`.

3.13.0.2 Tooltip-urile de pagină.

Componenta `PageTooltip.tsx` este reutilizabilă și se inserează o singură dată pe fiecare pagină principală. Vizibilitatea este controlată prin `localStorage` (cheia `visited_pages`): tooltip-ul se afișează exclusiv la prima vizită, cu dispariție automată după aproximativ 8 secunde (mai exact, un `setTimeout` la 8500 ms de la încărcarea paginii, tooltip-ul devenind vizibil la 500 ms) sau imediat, prin butonul de închidere. Mesajele sunt specifice fiecărei pagini. Pe dashboard, de exemplu, textul afișat este: *„This is your control center. Add devices, organize them into rooms, and monitor everything in real time.”*

(a) Setup wizard, pasul „account”

(b) Setup wizard, pasul „devices”

(c) Setup wizard, ecranul de finalizare

(d) Checklist-ul de pe dashboard

Figura 3.9: Fluxul de onboarding al platformei: pașii *account* (a) și *devices* (b) din setup wizard, ecranul de finalizare care confirmă resursele configurate (c) și checklist-ul *Getting Started* afișat ulterior pe dashboard (d).

3.14 Pagina de setări

`app/dashboard/settings/page.tsx` grupează într-un singur formular controlat patru zone funcționale: profilul de bază (nume, email, username, parolă nouă cu confirmare), avatarul (`FileReader` pentru preview, `FormData` la salvare; backend-ul generează un nume UUID și șterge prin semnal `pre_save` avatarul vechi), paleta de *accent colors* (cele 7 valori OKLCH din `ACCENT_COLORS`, aplicate instant prin `updateAccentColor`, iar efectul vizual este ilustrat în Figura 3.4) și un panou read-only cu rolul curent și informații despre cont. Salvarea apelează `updateProfile` din `apiClient.js` (un `PUT /api/me/` multipart), iar la succes invocă `refreshUser()` ca să-și reîmprospăteze starea contextului de utilizator. Fără acest pas, o

schimbare de nume nu s-ar reflecta imediat în sidebar.

3.15 Colecția de dispozitive a comunității

Pagina `app/dashboard/device-collection/page.tsx` este locul în care utilizatorii explorează tipurile de dispozitive deja aprobate. Spre deosebire de *device builder* (care creează un tip nou), colecția le *expune* pe cele existente: fiecare tip apare ca un card (**Card** din `shadcn/ui`) cu numele, badge-uri pentru senzorii din componență și o miniatură a topologiei hardware (Figura 3.10). Sus se află un câmp de căutare (**Input** cu filtrare locală prin `useMemo`); pentru rolul `admin`, apare și un buton de import care încarcă un tip dintr-un fișier JSON (`importDeviceTypesFromFile`). Utilizatorii obișnuiți propun în schimb tipul spre aprobare. La click pe un card, aplicația navighează către pagina de detaliu `device-collection/[id]`, unde se găsesc codul firmware, schema electrică, documentația și dialogul de descărcare a sketch-ului personalizat (prezentat în Figura 3.2). Datele provin dintr-un singur `useQuery` către `GET /api/device-types/`; filtrarea la tipurile aprobate se face pe client, nu de către endpoint.

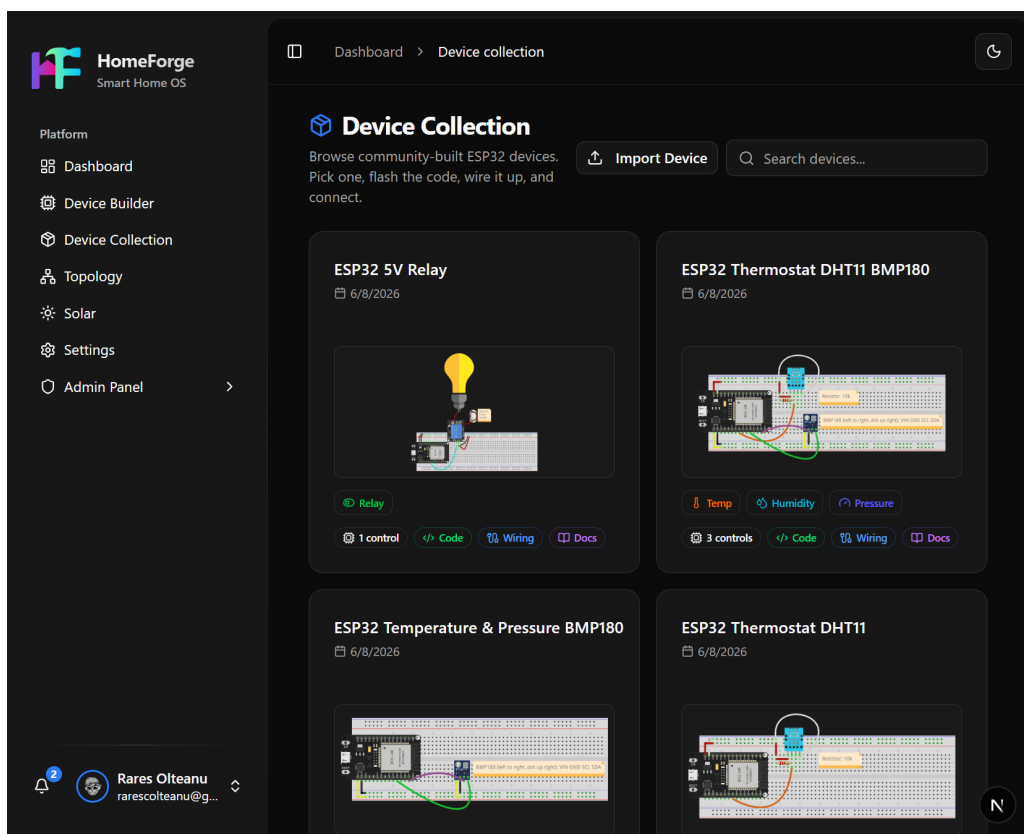


Figura 3.10: Pagina *Device Collection*: grila tipurilor de dispozitive aprobate, propuse de comunitate, fiecare card afișând badge-uri pentru senzori și o miniatură a topologiei hardware.

3.16 Adăugarea unui dispozitiv: AddDeviceDialog

Adăugarea unui dispozitiv este una dintre cele mai frecvente acțiuni ale utilizatorului. De aceea am implementat-o ca dialog multi-pas: `AddDeviceDialog.tsx`. Componenta este un **Dialog** construit din primitive `shadcn/ui` peste `Radix`, cu trei pași: (1) selectarea tipului de dispozitiv dintr-o listă filtrată după `approved=true`, (2) completarea formularului cu nume, adresă IP (validată local), cameră (prin `Select`) și o iconiță aleasă prin componenta `IconPicker`, (3) ecranul

de confirmare și transmiterea cererii `registerDevice`. Datele necesare sunt încărcate printr-un singur apel `Promise.all([fetchDeviceTypes(), fetchRooms()])` la deschidere; după înregistrarea cu succes, `onDeviceAdded` declanșează un `window.location.reload()`, o soluție simplă care poate fi înlocuită în versiuni ulterioare cu `queryClient.invalidateQueries`. Componenta `IconPicker` este un `Popover` care expune întregul set de iconițe Lucide (peste 1500 de simboluri) împreună cu un câmp de căutare; filtrarea se face local prin `useMemo`, limitată la primele 200 de rezultate pentru a păstra timpul de randare scăzut.

3.17 Panoul de administrare

Pentru utilizatorii cu rol `admin` sau `owner`, secțiunea `/dashboard/admin/` expune cinci pagini de management. Toate verifică încă o dată rolul (`useUser()` + role check). Există și protecții la nivel de UI: butoanele destructive sunt mereu confirmate printr-un `AlertDialog`.

3.17.1 Hub-ul: `/dashboard/admin/`

Pagina-hub (`app/dashboard/admin/page.tsx`) este o grilă de patru carduri, fiecare ducând la o sub-secțiune. Sub grila de module există un mic card „System Status” cu două indicatoare verzi pentru *API Services* și *Database Connected*, momentan statice, dar gândite să fie populate într-o iterație viitoare de o cerere către `GET /api/system-status/`.

3.17.2 Aprobarea tipurilor de dispozitive: `approvals/`

`app/dashboard/admin/approvals/page.tsx` reproduce vizualizarea de tip device builder în interiorul interfeței de aprobare: lista propunerilor *Pending*, cu un link separat către tipurile respinse, iar pentru fiecare propunere: un mini-grafic React Flow care arată topologia electrică, un preview al cardului UI rezultat (un `SmartDeviceCard` read-only) și butoanele *Approve / Deny* (acesta din urmă obligă completarea unui motiv prin `Textarea`, validat și pe server). La aprobare, `useMutation` apelează `approveDeviceType(id)` și invalidează cache-ul; backend-ul creează notificarea „Device Type Approved” pentru contul care a dat aprobarea (vezi Capitolul 2), nu pentru cel care a propus tipul. Interfața de aprobare este arătată în Figura 3.11.

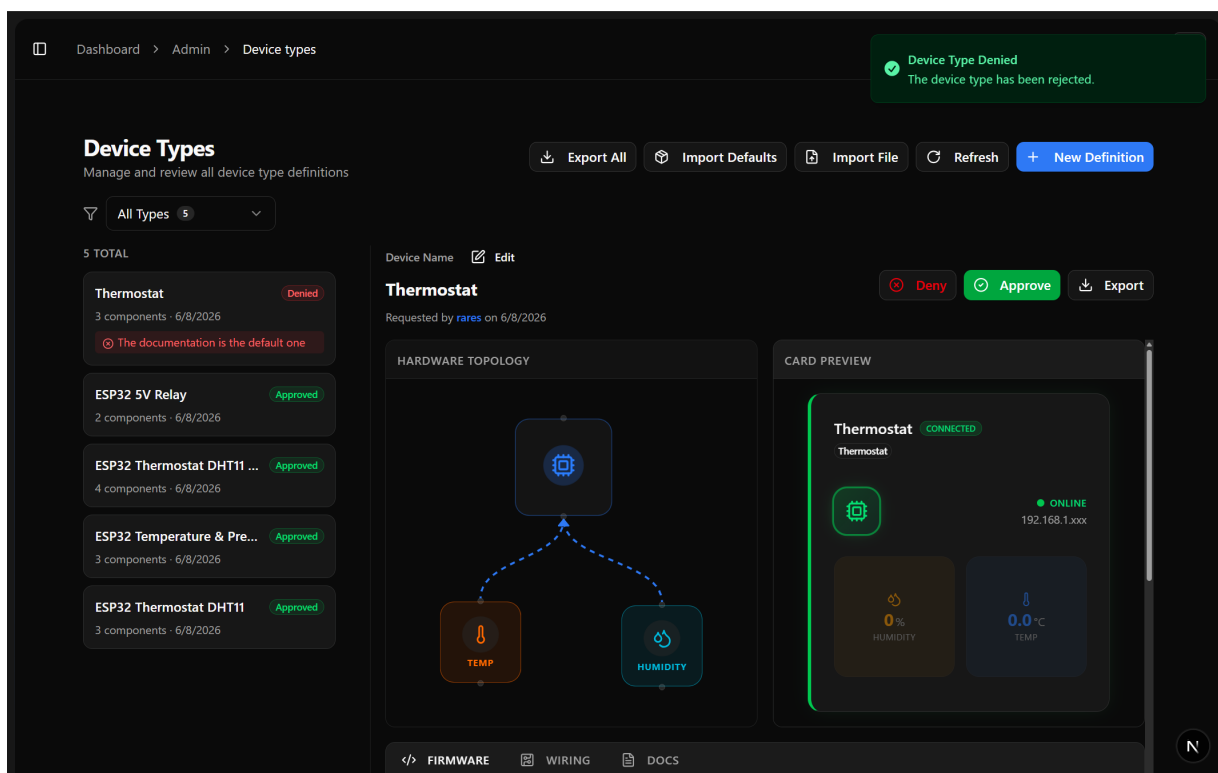


Figura 3.11: Pagina de aprobare a tipurilor de dispozitive propuse de utilizatori

3.17.3 Gestiunea utilizatorilor: users/

`app/dashboard/admin/users/page.tsx` afișează un tabel `Table` (shadcn/ui) cu toți utilizatorii din sistem. Coloanele includ avatar, username, email, rol și un meniu de acțiuni (`DropDownMenu` cu `DropDownMenuSub` pentru schimbarea rolului). Filtrul de top oferă o căutare textuală live (controlată local) și un select pentru filtrare după rol. Url-ul acceptă un `?highlight=<id>` care evidențiază temporar un user, folosit din notificări pentru deep-linking.

3.17.4 Camerele, tipurile de dispozitive, debug-ul

Restul sub-paginilor au design similar. `admin/rooms/` efectuează operațiuni CRUD pe camere, folosind `IconPicker` pentru iconițe (ștergerea unei camere lasă dispozitivele în starea „neasignată” prin `on_delete=SET_NULL`). `admin/debug/` este o pagină de *debugging* care permite administratorului să suprascrie manual starea unui dispozitiv (`Textarea` cu validare JSON live), să forțeze statusul `online/offline/error` și să trimită comenzi de probă, utilă pentru iterația pe interfață fără hardware conectat.

Pagina `admin/device-types/` merită o discuție separată. Aceasta listează toate tipurile (aprobate, în așteptare, respinse) cu suport pentru import/export JSON și ștergere în bulk a celor respinse. Două aspecte au necesitat tratament special. Primul: pentru un administrator, endpoint-ul `GET /device-types/` returnează *toate* tipurile (inclusiv pending și denied), iar listele dedicate `/admin/device-types/pending/` și `/admin/device-types/denied/` duceau anterior la apariția unui tip recent propus în două secțiuni simultan. Soluția implementată este o de-duplicare după `id`, în care înregistrările sunt fuzionate (păstrând câmpurile grele (`firmware_code`, `wiring_diagram_base64`, `documentation`) din lista completă), iar statusul autoritar provine din listele dedicate; contoarele afișate pe badge-uri sunt recalculat pe setul de-duplicat. Al doilea: panoul de revizuire detaliată afișează simultan topologia hardware, previzualizarea cardului și trei artefacte textuale (*firmware*, *wiring*, *docs*). Versiunea inițială

le stivuia vertical pe toate trei, forțând o derulare excesivă; varianta curentă le grupează într-un **Tabs** shadow cu trei tab-uri și un corp scrollabil cu înălțime fixă (`h-[440px]`), oferind o vizualizare compactă și completă.

Capitolul 4

Firmware-ul ESP32 și integrarea hardware

4.1 Alegerea platformei: de ce ESP32

Tot firmware-ul livrat cu HomeForge este ținut pe familia Espressif ESP32. Decizia se bazează pe patru argumente practice, nu pe vreo afinitate de brand:

1. **Cost.** Modulele ESP32-WROOM-32 (DevKit V1, Figura 4.1) costă aproximativ 5 euro la achiziție unitară, un termen acceptabil pentru un proiect didactic în care studentul probabil va arde măcar o placă învățând.
2. **Wi-Fi integrat.** Spre deosebire de Arduino UNO, ESP32 include nativ Wi-Fi (802.11 b/g/n), Bluetooth Low Energy și un dual-core la 240 MHz. Nu este nevoie de niciun *shield* adițional pentru conectarea la LAN.
3. **Compatibilitate cu Arduino IDE.** Toolchain-ul ESP-IDF este pentru cei care vor să meargă în profunzime; pentru un demonstrator didactic este suficient framework-ul Arduino-ESP32, cu funcții familiare (`setup()`, `loop()`).
4. **Comunitate vastă.** Biblioteci precum `PubSubClient` (pentru MQTT), `ArduinoJson` (pentru parsing/serializare JSON) și `DHT` sunt utilizate de zeci de mii de proiecte și au documentație abundentă.

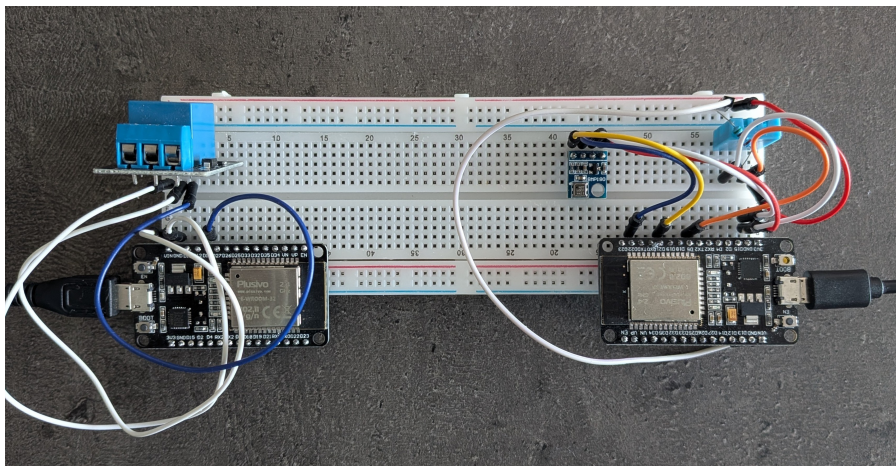


Figura 4.1: Modulul ESP32 DevKit V1 utilizat în prototipurile HomeForge

4.2 Anatomia unui sketch HomeForge

Toate variantele de firmware folosite de HomeForge sunt distribuite *inline* în fișierul `fixture backend/api/fixtures/default_device_types.json`, sub câmpul `firmware_code` al fiecărui tip de dispozitiv. Această structurare elimină dependența de un arbore de fișiere separat și

permite ca distribuția unui set complet de tipuri (cod sursă, schemă electrică în Markdown și imagini codificate `base64`) să se reducă la o singură resursă JSON. Pentru a face explicațiile cât mai concrete, exemplele de mai jos provin din varianta *ESP32 5V Relay*. Este singurul template care implementează ambele sensuri: fluxul de telemetrie (publicare de stare) și fluxul invers de comenzi (subscribe + callback). Astfel integrez ambele direcții de comunicație MQTT într-un singur fișier sursă. Pentru fluxurile pur senzoriale (publicare ciclică, fără recepție de comenzi), referințele vor merge către varianta *ESP32 Thermostat DHT11 BMP180*.

4.2.1 Variabile de configurare (placeholder-uri)

Primele linii ale fiecărui sketch conțin trei constante marcate cu placeholder-uri Mustache-like:

```
1 | const char* wifi_ssid = "{WIFI_SSID}";
2 | const char* wifi_password = "{WIFI_PASSWORD}";
3 | const char* server_ip = "{SERVER_IP}";
4 | const int mqtt_port = 1883;
```

Aceste valori sunt înlocuite în clar de către utilizator înainte de compilare. `wifi_ssid` și `wifi_password` sunt strict obligatorii: backend-ul refuză să accepte un `firmware_code` care nu conține referințele lor. În schimb, `server_ip` a devenit *optional*: dispozitivul poate descoperi singur brokerul prin mDNS, mecanism descris în secțiunea 4.3. Validatorul tratează valorile goale ("" sau placeholder-ul neînlocuit) ca un semnal pentru activarea descoperirii automate, iar instalarea funcționează în acest caz fără ca utilizatorul să caute manual adresa serverului HomeForge.

4.2.2 Identitatea dispozitivului: deviceMac

La pornire, după conectarea la Wi-Fi, sketch-ul citește adresa MAC a interfeței Wi-Fi a cip-ului, o normalizează și se anunță în rețeaua locală prin mDNS:

```
1 | void setup_wifi() {
2 |     delay(10);
3 |     Serial.printf("\nConnecting to %s", wifi_ssid);
4 |     WiFi.mode(WIFI_STA);
5 |     WiFi.begin(wifi_ssid, wifi_password);
6 |     while (WiFi.status() != WL_CONNECTED) {
7 |         delay(500);
8 |         Serial.print(".");
9 |     }
10 |     deviceIp = WiFi.localIP().toString();
11 |     deviceMac = WiFi.macAddress();
12 |     deviceMac.replace(":", "");
13 |
14 |     commandTopic = String("homeforge/devices/") + deviceMac + "/command";
15 |     stateTopic = String("homeforge/devices/") + deviceMac + "/state";
16 |
17 |     String hostname = "homeforge-" + deviceMac;
18 |     if (MDNS.begin(hostname.c_str())) {
19 |         Serial.printf("mDNS responder started: %s.local\n", hostname.c_str());
20 |     }
21 | }
```

La firmware-urile pur senzoriale, linia `commandTopic` lipsește (nu există nimic la care să se aboneze), restul fiind identic. Patru detalii merită semnalate:

- `WiFi.mode(WIFI_STA)` forțează modul *station-only*, oprind orice activitate de soft-AP rămasă din încercări anterioare;
- adresa MAC este formatată implicit de Arduino-ESP32 cu separatori (:); se elimină pentru ca topicul MQTT să fie un singur token alfanumeric, același format pe care îl așteaptă `mqtt_listener` pe partea de Django;

- topicul `state` este construit pe toate firmware-urile; topicul `command` este construit doar de variantele care expun un actuator (*ESP32 5V Relay*), restul fiind pur senzoriale și neavând nevoie să se aboneze la nimic;
- adresa MAC servește ca identificator unic la nivel de platformă: un dispozitiv care își schimbă IP-ul își păstrează identitatea către backend.

4.2.3 Conectarea la broker și subscribe

Funcția `reconnect()` este apelată din `loop()` ori de câte ori clientul `PubSubClient` pierde legătura. În varianta *ESP32 5V Relay*, funcția înglobează și re-descoperirea brokerului atunci când adresa cache-uită nu mai răspunde:

```

1  void reconnect() {
2      while (!client.connected()) {
3          if (strlen(mqttServer) == 0) {
4              if (!discoverBroker()) { delay(3000); return; }
5              client.setServer(mqttServer, mqtt_port);
6          }
7          Serial.printf("Connecting to MQTT broker %s ...", mqttServer);
8          String clientId = "ESP32-Relay-" + deviceMac;
9          if (client.connect(clientId.c_str())) {
10             Serial.println(" connected!");
11             client.subscribe(commandTopic.c_str()); // doar pe firmware-urile cu actuator
12             publishState();
13         } else {
14             Serial.printf(" failed (rc=%d). Retrying in 5s...\n", client.state());
15             delay(5000);
16             return;
17         }
18     }
19 }

```

`clientId` este unic per placă, construit din adresa MAC fără separatori și un prefix care identifică tipul de firmware: *ESP32-Relay-*, *ESP32-DHT11-*, *ESP32-BMP180-* sau *ESP32-Sensor-* (acesta din urmă pentru varianta combinată DHT11 + BMP180). Aceste prefixe sunt utile la depanare: un `mosquitto_sub` pe brokerul HomeForge permite identificarea imediată a tipului de placă care s-a conectat. Imediat după conectare, dispozitivul publică starea curentă prin `publishState()`. Acest mesaj joacă rolul unui *check-in*: este suficient pentru ca daemon-ul `mqtt_listener` să facă auto-binding-ul descris în Capitolul 2. Firmware-urile pur senzoriale au aceeași structură, dar omit linia `client.subscribe(...)`: neavând actuator, nu se abonează la niciun topic de comandă.

4.2.4 Telemetria: publishState()

Funcția care construiește mesajul de stare folosește `ArduinoJson` pentru a serializa un document static. Pentru varianta *ESP32 5V Relay*, mesajul publicat conține valoarea actuatorului (`relay_1`), un eventual *echo* către cheia dinamică folosită în comanda care a generat starea curentă și câmpurile auxiliare `ip` / `mac`:

```

1  void publishState() {
2      if (!client.connected()) return;
3
4      StaticJsonDocument<256> doc;
5      doc["ip"] = deviceIp;
6      doc["mac"] = deviceMac;
7      doc[VAR_RELAY] = relayState;
8      if (lastControlKey.length() > 0) {
9          doc[lastControlKey] = relayState; // trimite înapoi cheia dinamică
10     }
11
12     char buffer[256];
13     serializeJson(doc, buffer);

```

```

14 | client.publish(stateTopic.c_str(), buffer);
15 | }

```

Mecanismul *echo* acoperă cazul în care device builder-ul a generat o cheie dinamică pentru widget (de exemplu `switch-1780830762843`): firmware-ul reține ultima cheie primită în `lastControlKey` și o include în mesajul de stare alături de `relay_1`, astfel încât interfața să vadă confirmarea pe identificatorul exact pe care l-a folosit la trimitere. Termostatele senzoriale (*DHT11*, *BMP180*, *DHT11 + BMP180*) folosesc același tipar, dar populează câmpurile relevante pentru senzorii lor (`VAR_TEMP`, `VAR_HUMID`, `VAR_PRESSURE`, după caz), fără `VAR_RELAY` (nu au actuator). `StaticJsonDocument` cu dimensiune fixă alocă memoria în stivă, nu pe heap, ceea ce este recomandat pe microcontrolere unde fragmentarea heap-ului poate cauza eșecuri intermitente după ore de funcționare. Câmpurile `ip` și `mac` sunt incluse în payload chiar dacă brokerul le-ar putea infera din topic, fiind utile pentru auto-binding-ul descris în Capitolul 2, atunci când backend-ul nu cunoaște încă adresa MAC. Constantele de denumire a câmpurilor sunt declarate la începutul fiecărui sketch:

```

1 | const char* VAR_RELAY    = "relay_1";    // doar în ESP32 5V Relay
2 | const char* VAR_TEMP     = "temperature";
3 | const char* VAR_HUMID    = "humidity";
4 | const char* VAR_PRESSURE = "pressure";

```

Aceste nume sunt cele „standard”, pe care `mqtt_listener.map_standard_key()` le translatează către cheile dinamice ale widget-urilor.

4.2.5 Recepția comenzilor: `callback()`

Callback-ul `PubSubClient` este apelat de fiecare dată când brokerul publică un mesaj pe topicul la care firmware-ul s-a abonat. Implementarea concretă din *ESP32 5V Relay* iterează peste cheile documentului JSON și acceptă, pe lângă `relay_1`, orice identificator care începe cu prefixul `switch-` (folosit de device builder atunci când utilizatorul nu redenumeste manual controlul). Parser-ul tolerează trei reprezentări pentru valoarea booleană (boolean nativ, string `"true"`, `"on"`, `"1"` și întreg), pentru a fi compatibil cu diferitele stiluri de payload pe care le poate genera UI-ul:

```

1 | void callback(char* topic, byte* payload, unsigned int length) {
2 |     String message;
3 |     for (unsigned int i = 0; i < length; i++) {
4 |         message += (char)payload[i];
5 |     }
6 |     Serial.print("Command received: ");
7 |     Serial.println(message);
8 |
9 |     StaticJsonDocument<256> doc;
10 |    if (deserializeJson(doc, message)) return;
11 |
12 |    JsonObject root = doc.as<JsonObject>();
13 |    for (JsonPair kv : root) {
14 |        String key = kv.key().c_str();
15 |        if (key == VAR_RELAY || key.startsWith("switch-")) {
16 |            lastControlKey = key;
17 |            if (kv.value().is<bool>()) {
18 |                relayState = kv.value().as<bool>();
19 |            } else if (kv.value().is<const char*>()) {
20 |                String val = kv.value().as<String>();
21 |                val.toLowerCase();
22 |                relayState = (val == "true" || val == "on" || val == "1");
23 |            } else if (kv.value().is<int>()) {
24 |                relayState = kv.value().as<int>() == 1;
25 |            }
26 |            break;
27 |        }
28 |    }
29 |
30 |    digitalWrite(RELAY_PIN, relayState ? HIGH : LOW);

```

```

31 |
32 | preferences.begin("state", false);
33 | preferences.putBool("relay", relayState);
34 | preferences.end();
35 |
36 | publishState();
37 | }

```

Trei detalii merită observate. Primul: variabila `lastControlKey` reține identificatorul exact cu care a venit ultima comandă, pentru ca `publishState()` să poată face *echo* înapoi pe aceeași cheie dinamică (mecanism prezentat în subsecțiunea anterioară). Al doilea: după acționarea releului, starea este persistată în memoria nevolatilă a ESP32 (`Preferences`, namespace `state`), astfel încât după o reală cădere de tensiune dispozitivul revine în aceeași stare la pornire. Al treilea: `publishState()` este apelat imediat după execuție, închizând bucla de feedback: backend-ul primește confirmarea că operația a fost realizată și actualizează `current_state` în baza de date, iar frontend-ul preia noua valoare la următoarea iterație de `refreshInterval` (3 secunde).

4.2.6 Bucla principală

`loop()` este structurată în trei pași: servirea serverului HTTP de fallback (`server.handleClient()` pentru endpoint-ul `/config`), asigurarea conexiunii MQTT prin `reconnect()` și o secvență ciclică de eșantionare la fiecare câteva secunde. Pentru *ESP32 5V Relay*, partea ciclică se limitează la republicarea stării actuatorului (nu există senzori de citit); pentru firmware-urile de termostat, citirea senzorilor este izolată într-o funcție `readSensors()` apelată din bucla principală. Schema buclei pentru varianta *ESP32 Thermostat DHT11 BMP180*:

```

1 | void loop() {
2 |     server.handleClient();
3 |     if (!client.connected()) {
4 |         reconnect();
5 |     }
6 |     client.loop();
7 |
8 |     unsigned long now = millis();
9 |     if (now - lastMsg > interval) {
10 |         lastMsg = now;
11 |         readSensors();
12 |         publishState();
13 |     }
14 | }
15 |
16 | void readSensors() {
17 |     float newT = dht.readTemperature();
18 |     float newH = dht.readHumidity();
19 |     float newP = bmp.readPressure();
20 |
21 |     if (!isnan(newT) && !isnan(newH)) {
22 |         humidity = newH;
23 |         // Aplică corecția de toleranță DHT11.
24 |         temperature = (DHT11_ERROR < 0) ? newT + DHT11_ERROR : newT - DHT11_ERROR;
25 |         if (!isnan(newP)) pressure = newP / 100.0; // Pa în hPa
26 |     } else {
27 |         Serial.println("Failed to read from DHT11/BMP180 sensor!");
28 |     }
29 | }

```

Constanta `interval = 5000` (5 secunde) este sincronizată cu pragul din `mqtt.listener.check_offline_devices()` (30 de secunde): primim 6 oportunități de heartbeat ratate înainte ca dispozitivul să fie declarat offline. Pe DHT11, citirile prea frecvente nu sunt fiabile (senzorul are nevoie de cel puțin o secundă între eșantioane), deci intervalul de 5 secunde oferă o marjă confortabilă. Verificarea `!isnan(newT) && !isnan(newH)` reține valorile vechi atunci când o citire eșuează intermitent, iar constanta `DHT11_ERROR` aplică o

corecție de offset pentru a compensa eroarea sistematică observată la lotul de DHT11 folosit. Presiunea barometrică este convertită din Pascali (unitate brută a BMP180) în hectopascali (hPa), unitatea uzuală în meteorologie.

4.3 Descoperirea automată a brokerului prin mDNS

Una dintre operațiunile mai puțin elegante în implementarea inițială era cerința ca utilizatorul să cunoască adresa IP a serverului HomeForge înainte de compilare. Pentru rețelele în care IP-ul serverului este atribuit dinamic prin DHCP, această cerință devenea o sursă recurentă de probleme: o singură reassignare DHCP forța re-flash-area tuturor dispozitivelor. Versiunea curentă rezolvă acest aspect prin introducerea unui mecanism de descoperire automată bazat pe Multicast DNS (mDNS) și DNS-Based Service Discovery (DNS-SD), conform RFC 6762 [7] și RFC 6763 [8].

4.3.1 Principiul de funcționare

mDNS permite rezolvarea numelor DNS pe segmentul local de rețea fără a depinde de un server DNS centralizat, prin trimiterea de pachete UDP către adresa de multicast 224.0.0.251 (IPv4), pe portul 5353. DNS-SD extinde acest mecanism cu o convenție pentru descoperirea serviciilor: un client interesat de un serviciu de tipul „MQTT peste TCP” interoghează numele rezervat `_mqtt._tcp.local`, iar furnizorii de serviciu răspund cu o înregistrare SRV care indică gazda și portul efectiv. HomeForge folosește această convenție prin daemon-ul Avahi care rulează în containerul backend (acolo unde topologia de rețea o permite, vezi Capitolul 5), publicând în mod continuu un record `_mqtt._tcp` cu portul 1883.

4.3.2 Ierarhia celor trei niveluri de descoperire

Firmware-ul ESP32 încearcă să identifice brokerul printr-o secvență de trei mecanisme, în ordinea priorității:

1. **Override manual prin `server_ip`.** Dacă utilizatorul a substituit explicit adresa serverului la momentul compilării, dispozitivul se conectează direct, fără descoperire. Modul este util pentru topologii fixe sau pentru sisteme unde mDNS nu este disponibil.
2. **Cache local în memorie nevolatilă (NVS).** La fiecare conectare reușită, firmware-ul salvează adresa brokerului în NVS folosind biblioteca `Preferences`. La un `reboot` ulterior, reconectarea este aproape instantanee fiindcă cache-ul este consultat înainte de a iniția o nouă interogare multicast.
3. **Interogare mDNS / DNS-SD.** Atunci când `server_ip` este gol și cache-ul lipsește sau este invalid, firmware-ul folosește biblioteca `ESPmDNS` pentru a interoga `_mqtt._tcp` pe segmentul de rețea. Funcția `discoverBroker()` încearcă până la trei interogări consecutive (cu timeout scurt) și acceptă primul răspuns valid:

```
1  bool discoverBroker() {
2      Serial.println("Discovering HomeForge broker via mDNS...");
3      for (int attempt = 0; attempt < 3; attempt++) {
4          int n = MDNS.queryService(MDNS_SERVICE, MDNS_PROTO);
5          if (n > 0) {
6              IPAddress ip = MDNS.address(0);
7              String found = ip.toString();
8              if (found != "0.0.0.0") {
9                  strncpy(mqttServer, found.c_str(), sizeof(mqttServer) - 1);
10                 saveBroker(found);
11             }
12         }
13     }
14 }
```

```

11     Serial.printf("Broker found: %s\n", mqttServer);
12     return true;
13 }
14 }
15     delay(1000);
16 }
17     Serial.println("Broker not found (will retry / use /config fallback).");
18     return false;
19 }

```

`mqttServer` este un buffer de caractere global (umplut prin `strncpy`), iar `saveBroker(found)` apelează API-ul `Preferences` pentru a persista adresa descoperită în memoria nevolatilă a ESP32, pentru reutilizare la următorul boot.

Dacă toate cele trei niveluri eșuează (situație rară, dar plauzibilă atunci când serverul nu rulează încă sau este pe altă subrețea), dispozitivul activează un al patrulea mecanism de rezervă: pornește un server HTTP pe portul 80 și acceptă o cerere POST `/config` cu adresa brokerului. Utilizatorul deschide în browser pagina dispozitivului și introduce adresa manual, fără a fi necesar un reflash.

4.3.3 Anunțarea propriei identități prin mDNS

Pe lângă consumarea înregistrărilor mDNS, fiecare dispozitiv își publică și propria identitate, sub un nume derivat din adresa MAC: `esp32-<MAC>.local`. Astfel, administratorul rețelei locale poate accesa dispozitivul prin nume DNS, fără a căuta IP-ul în interfața de administrare a router-ului. Apelul `MDNS.begin(hostname)` se face imediat după conectarea la Wi-Fi, în secvența standard de inițializare. Setul predefinit conține patru tipuri de dispozitive ESP32, toate stocate inline în `backend/api/fixtures/default_device_types.json` și importate automat la apelul POST `/api/device-types/import-defaults/` în prima sesiune cu administratorul. Fiecare intrare conține integral codul firmware Arduino, schema electrică (text Markdown + imagine `base64`), documentația și definiția vizuală a cardului, totul într-un singur obiect JSON, fără referințe către fișiere externe.

Cele patru variante împart o singură categorie funcțională clară. **ESP32 5V Relay** este singurul template care expune un actuator: un releu controlat de `RELAY PIN = GPIO 14`, redat în interfață printr-un widget `TOGGLE` a cărui variabilă (`relay_1` sau un identificator generat dinamic care începe cu prefixul `switch-`) este transmisă bidirecțional, atât în publicările de stare cât și în comenzile primite de la backend. Celelalte trei variante sunt strict senzoriale: publică telemetrie ciclică, fără a se abona la topicul de comandă și fără a implementa un `callback` de recepție.

ESP32 Thermostat DHT11 folosește un senzor DHT11 (temperatură + umiditate) conectat la `DHTPIN = GPIO 5` și expune două widget-uri: `TEMPERATURE` și `HUMIDITY`. **ESP32 Temperature & Pressure BMP180** folosește un senzor BMP180 pe magistrală I2C (`SDA = GPIO 21`, `SCL = GPIO 22`, valorile implicite Wire ale ESP32), publicând temperatura măsurată de BMP-ul însuși și presiunea barometrică în două widget-uri `TEMPERATURE` și `PRESSURE`. **ESP32 Thermostat DHT11 BMP180** combină cele două: DHT11 pe GPIO 5 și BMP180 pe magistrala I2C standard, publicând trei valori (temperatură, umiditate și presiune) în trei widget-uri corespunzătoare.

Cheile cu care fiecare firmware publică datele pe broker sunt constantele `VAR_TEMP = "temperature"`, `VAR_HUMID = "humidity"`, `VAR_PRESSURE = "pressure"` și `VAR_RELAY = "relay_1"`. Daemon-ul `mqtt.listener` de pe partea de backend mapează aceste nume canonice către identificatorii dinamici generați de device builder pentru fiecare widget al cardului, astfel încât reorganizarea widget-urilor în UI nu impune reflash-area firmware-ului.

Pentru fiecare variantă, schema electrică este distribuită ca imagine `base64` în câmpul `wiring_diagram_base64` al obiectului JSON, însoțită opțional de o descriere textuală în

wiring_diagram_text. Schema electrică minimală pentru varianta *ESP32 5V Relay* este sintetizată în Tabelul 4.1, iar montajele pe breadboard ale tuturor celor patru variante sunt arătate în Figura 4.2:

Modul	Pin	ESP32	Descriere
Releu	VCC	5V	Alimentare 5 V
Releu	GND	GND	Masă
Releu	IN	GPIO 14	Semnal de control (<code>RELAY_PIN</code>)

Tabela 4.1: Conexiunile pinilor pentru tipul *ESP32 5V Relay*

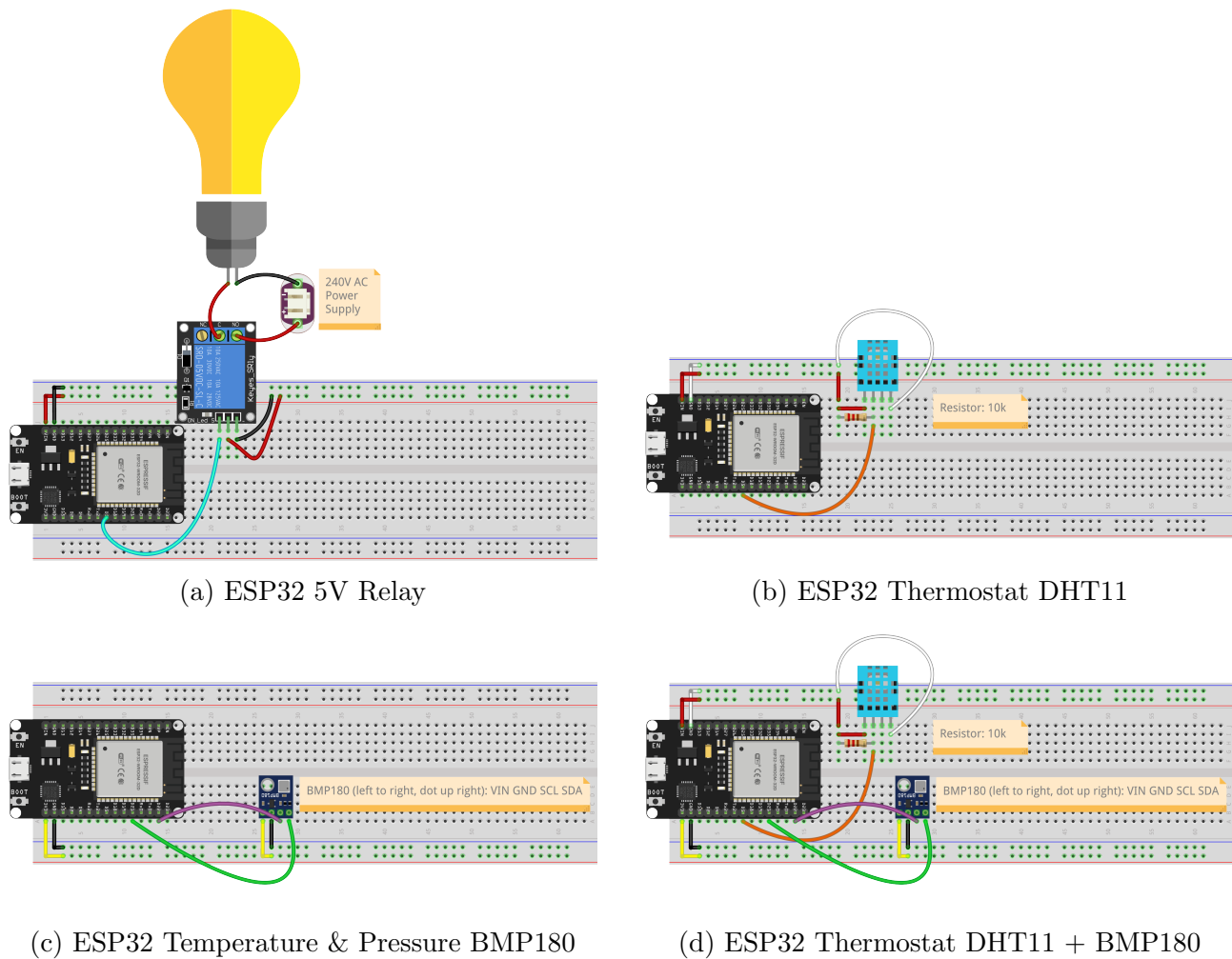


Figura 4.2: Schemele de conexiuni (breadboard) ale celor patru tipuri de dispozitive predefinite: releul 5V comandând un consumator (a), termostatul cu DHT11 (b), modulul de temperatură și presiune BMP180 pe I2C (c) și varianta combinată DHT11 + BMP180 (d).

4.4 Bibliotecile Arduino utilizate

Pentru a compila firmware-urile, mediul Arduino IDE folosește mai multe biblioteci. O parte sunt incluse automat cu pachetul ESP32 Arduino Core de la Espressif: `WiFi.h` pentru conectarea Wi-Fi, `ESPmDNS.h` pentru descoperirea automată a brokerului, `Preferences.h` pentru cache-ul persistent al adresei brokerului în memoria nevolatilă, `WebServer.h` pentru

endpoint-ul de fallback `/config` și `Wire.h` pentru comunicarea I2C. Restul se instalează prin *Library Manager*: `PubSubClient` (Nick O’Leary, client MQTT cu suport pentru QoS 0 și 1), `ArduinoJson` (Benoit Blanchon, parsing și serializare JSON cu documente alocate static), `DHT sensor library` (Adafruit, folosită în firmware-urile cu DHT11) și `Adafruit BMP085 Library` (Adafruit, compatibilă cu BMP180). Pachetul ESP32 Arduino Core se adaugă în Arduino IDE prin *File* → *Preferences* → *Additional Boards Manager URLs*, cu URL-ul oficial https://espressif.github.io/arduino-esp32/package_esp32_index.json.

4.5 Limitări ale implementării curente

Pentru claritate, enumăr aici ce nu este implementat în firmware-ul curent. Nu folosesc task-uri FreeRTOS create explicit: bucla `loop()` împreună cu `PubSubClient.loop()` acoperă volumul actual de logică. Nu există un `Captive Portal` pentru configurarea Wi-Fi la prima pornire, SSID-ul și parola fiind înlocuite direct în cod prin placeholder-uri. Comunicarea MQTT nu folosește TLS, ceea ce este acceptabil într-un LAN privat, dar pentru un deployment de producție ar trebui adăugate certificate și `WiFiClientSecure`. Actualizarea OTA nu este implementată, re-flash-area făcându-se prin USB-UART. Niciuna dintre aceste limitări nu împiedică utilizarea curentă a platformei și sunt menționate ca direcții posibile de extindere.

Capitolul 5

Deployment, rulare și testare

5.1 Modurile de rulare ale platformei

Pentru fluxul de zi cu zi, HomeForge poate fi pornit în trei moduri, toate incluse explicit în repository. Primul este un mod *nativ* pentru dezvoltare locală (scriptul `dev.sh`, fără containere). Al doilea este un mod *containerizat de dezvoltare* (Docker Compose cu bridge networking, potrivit pentru Docker Desktop pe Windows sau macOS). Al treilea este un mod *containerizat de producție* (Docker Compose cu `network_mode: host`, gândit pentru un host Linux nativ pe care se află și dispozitivele, tipic un Raspberry Pi 4 în aceeași rețea cu plăcile ESP32). Distincția dintre ultimele două nu este cosmetică. Ea decide dacă mecanismul de descoperire mDNS descris în Capitolul 4 ajunge sau nu la dispozitive.

5.1.1 Modul de dezvoltare locală

Scriptul `./dev.sh` pornește în paralel backend-ul și frontend-ul, fără container. Are trei subcomenzi:

```
1 | ./dev.sh           # pornește atât backend cât și frontend
2 | ./dev.sh backend  # doar backend (Django pe :8000)
3 | ./dev.sh frontend # doar frontend (Next.js pe :3000)
```

La pornirea backend-ului, scriptul activează un `venv` local dacă există și rulează `python manage.py runserver 0.0.0.0:8000`. Frontend-ul este pornit prin `npm run dev`, cu *hot module reload*. Avantajul principal al acestui mod este viteza ciclului de iterație: o modificare în Python sau TypeScript este reflectată imediat, fără build de container. În schimb, dezvoltatorul trebuie să asigure manual rularea brokerului Mosquitto pe `localhost:1883` (instalat ca serviciu de sistem pe Linux/macOS sau prin pachetul oficial pe Windows).

5.1.2 Modul containerizat de dezvoltare (*bridge networking*)

Pentru a evita configurarea manuală a serviciilor de sistem, repository-ul include un `docker-compose.yml` de bază care orchestrează trei servicii: `db` (PostgreSQL), `backend` (Django împreună cu Mosquitto și Avahi în același container) și `frontend` (Next.js). Comanda uzuală de pornire este:

```
1 | docker compose up -d --build
```

Această variantă folosește *bridge networking* (modul implicit Docker) și expune porturile critice ale containerelor pe gazda fizică: 8000 pentru API, 3000 pentru interfața web, 1883 pentru brokerul MQTT și 5432 pentru PostgreSQL. Este modul recomandat pentru dezvoltare pe Docker Desktop (Windows și macOS), pentru două motive practice. Mai întâi, este singurul mod care funcționează corect pe aceste sisteme. Docker Desktop rulează containerele într-o mașină virtuală Linux dedicată, iar `network_mode: host` se referă acolo la interfața de loopback a VM-ului (192.168.65.x sau 172.x.x.x), nu la rețeaua fizică a gazdei Windows/macOS. Ca atare, traficul multicast mDNS de la ESP32-uri nu ajunge până la containerul backend prin

VM. În al doilea rând, păstrarea porturilor publicate explicit permite testarea locală a portului 8000 (API) și 3000 (UI) prin browser fără configurări suplimentare.

Demonstrația practică a acestei lucrări a fost realizată pe un laptop cu Windows 11 și Docker Desktop, configurație în care firmware-ul ESP32 nu poate folosi nivelul mDNS de descoperire automată. În această situație, plăcile sunt provizionate cu adresa IP explicită a gazdei Windows substituită în placeholder-ul `server_ip` înainte de flash, iar conexiunea se realizează prin portul 1883 publicat de container.

5.1.3 Modul de producție pe Linux nativ (*host networking*)

Pentru deployment-ul țintă (un Raspberry Pi 4 sau alt mini-server Linux care partajează aceeași rețea fizică cu dispozitivele), repository-ul include un fișier `overlay backend/deploy/docker-compose.prod.yml` care suprapune setări suplimentare peste compoziția de bază. Comanda completă este:

```
1 | docker compose -f docker-compose.yml \
2 |   -f backend/deploy/docker-compose.prod.yml \
3 |   up -d --build
```

Cele două schimbări principale aduse de overlay-ul de producție sunt:

- containerul `backend` primește `network_mode: host`, ceea ce îl plasează direct pe rețeaua fizică a gazdei Linux. În această configurație, Avahi anunță `_mqtt._tcp` pe adresa LAN reală (de exemplu `192.168.1.x`), iar pachetele multicast 5353 ajung la toate dispozitivele de pe segment, astfel încât descoperirea automată descrisă în Capitolul 4 devine funcțională;
- containerul `db` este expus în continuare prin port forwarding, iar backend-ul accesează PostgreSQL prin `DB_HOST=127.0.0.1` (deoarece, odată ieșit de pe rețeaua bridge a Compose-ului, numele de serviciu `db` nu mai este rezolvabil).

Această configurație a fost validată pe un Raspberry Pi 4 într-o rețea reală, prin scenariul standard al unei plăci ESP32 livrate cu `server_ip` gol. Tranziția între cele două moduri se reduce strict la prezența sau absența fișierului `overlay` în comanda `docker compose`. Nu e nevoie de nicio reconstruire a imaginilor. Fezabilitatea rulării Docker pe noduri Edge de tipul Raspberry Pi, raportată în literatura de specialitate [14], este astfel confirmată empiric și de prezenta lucrare.

5.2 Imaginea Docker a backend-ului

Dockerfile-ul backend-ului (Dockerfile) pornește de la `python:3.12-slim` și instalează în straturi pachete native necesare:

```
1 | FROM python:3.12-slim
2 | ENV PYTHONUNBUFFERED=1
3 | WORKDIR /app
4 | RUN apt-get update && apt-get install -y --no-install-recommends \
5 |   build-essential libpq-dev mosquitto mosquitto-clients \
6 |   avahi-daemon avahi-utils dbus libnss-mdns \
7 |   net-tools iproute2 procps \
8 |   && rm -rf /var/lib/apt/lists/*
9 |
10 | COPY requirements.txt .
11 | RUN pip install --no-cache-dir -r requirements.txt
12 | COPY . .
13 |
14 | RUN mkdir -p /var/run/mosquitto && \
15 |   chown mosquitto:mosquitto /var/run/mosquitto
```

```

16 RUN echo "listener 1883 0.0.0.0\allow_anonymous true" \
17 > /etc/mosquitto/conf.d/external.conf
18 RUN sed -i 's/#enable-dbus=yes/enable-dbus=yes/' /etc/avahi/avahi-daemon.conf
19
20 # Publică serviciul mDNS _mqtt._tcp pentru auto-discovery (Avahi îl preia
21 # automat din /etc/avahi/services/ la pornire).
22 COPY deploy/avahi/mqtt.service /etc/avahi/services/mqtt.service

```

Câteva observații:

- `mosquitto` și `avahi-daemon` sunt instalate direct în imaginea backend-ului. Această decizie favorizează simplitatea operațională: într-un context didactic, este preferabilă o configurație cu un singur container care expune întreaga infrastructură necesară hardware-ului (broker MQTT pentru plăcile ESP32 și mDNS pentru rezolvarea numelui `homeforge.local`), față de o arhitectură distribuită pe trei containere distincte.
- Configurația Mosquitto este scrisă din linia de comandă într-un fișier de override (`/etc/mosquitto/conf.d/external.conf`) cu două directive: `listener 1883 0.0.0.0` (acceptă conexiuni din afara container-ului) și `allow_anonymous true` (nu cere autentificare). Pentru un deployment în producție, ambele ar trebui reconsiderate, vezi secțiunea de discuție.
- `avahi-daemon` este pornit pentru a publica un record mDNS, astfel încât utilizatorul să poată accesa platforma la `http://homeforge.local` fără să caute IP-ul.

5.3 Scriptul de start `run.sh`

Containerul backend nu definește un `CMD/ENTRYPOINT` implicit; comanda de pornire e comentată în `docker-compose.yml`, iar orchestrarea reală a serviciilor e în scriptul `run.sh`, rulat manual în interiorul containerului (de exemplu prin `docker compose exec backend sh run.sh`, după ce `docker compose up` a construit imaginile). Scriptul folosește POSIX `sh` (nu `bash`) pentru compatibilitate cu imagini Alpine sau slim unde `/bin/sh` nu este Bash. Forma sa, condensată pentru claritate:

```

1  #!/usr/bin/env sh
2  set -eu
3
4  python -m pip install --upgrade pip
5  python -m pip install -r requirements.txt
6
7  # Aplică migrațiile bazei de date (preferă migrate.sh dacă bash este disponibil)
8  if [ -f "migrate.sh" ] && command -v bash >/dev/null 2>&1; then
9      bash migrate.sh
10  else
11      python manage.py migrate
12  fi
13
14  mkdir -p /run/dbus /var/run/dbus /run/avahi-daemon /app/media/avatars
15  rm -f /run/dbus/pid /var/run/dbus/pid /run/avahi-daemon/pid \
16      /run/mosquitto/mosquitto.pid
17
18  # Servicii opționale: detectate cu `command -v`; lipsa unuia generează warning
19  command -v dbus-daemon >/dev/null 2>&1 && dbus-daemon --system --fork
20  command -v avahi-daemon >/dev/null 2>&1 && avahi-daemon --daemonize --no-drop-root
21  command -v mosquitto >/dev/null 2>&1 && mosquitto -d -c /etc/mosquitto/mosquitto.conf
22
23  # Procesele Django auxiliare, în background
24  python manage.py mqtt_listener &
25  python manage.py poll_solar &
26
27  exec python manage.py runserver 0.0.0.0:8000

```

Pașii sunt secvențiali și pot fi împărțiți pe categorii:

1. upgrade-ul `pip` și reinstalarea dependențelor (utilă atunci când imaginea este folosită cu `volume-bind` pentru dezvoltare, când `site-packages`-ul container-ului poate fi suprascris);
2. aplicarea migrațiilor Django prin `migrate.sh` dacă acesta există și `bash` este disponibil, altfel fallback la `python manage.py migrate`; etapa pornește schema bazei de date la prima rulare;
3. curățarea `.pid`-urilor stale, fiindcă containerul nu are un sistem de inițializare propriu (cum ar fi `systemd`), iar serviciile lasă fișiere de PID care, la restart, pot bloca pornirea;
4. pornirea condiționată a serviciilor de sistem în ordinea `dbus` → `avahi` → `mosquitto` (prezența fiecăruia este verificată cu `command -v`; lipsa unuia generează un *warning*, fără a opri scriptul);
5. lansarea în fundal a celor două procese auxiliare Django: `mqt_listener` pentru recepția telemetriei MQTT și `poll_solar` pentru menținerea *liveness*-ului sistemelor solare în topologie;
6. pornirea serverului Django în prim-plan prin `exec`, ceea ce face ca PID-ul Django să devină procesul principal al containerului: atunci când serverul se oprește, întregul container se închide.

5.4 Imaginea Docker a frontend-ului

Dockerfile-ul frontend-ului (`Dockerfile`) este minimal:

```

1 FROM node:25-alpine
2 RUN apk add --no-cache bash
3 WORKDIR /app
4 COPY package.json .
5 COPY package-lock.json .
6 RUN npm install
7 COPY . .

```

Imaginea este o Alpine cu Node.js 25, cu `npm install` cache-uit ca strat separat (pentru a beneficia de re-build incremental atunci când se schimbă doar codul, nu și dependențele). `run.sh` de pe partea de frontend pornește `npm run dev`, la fel rulat manual în container (comanda din `docker-compose.yml` e comentată, la fel ca la backend), deci modul implicit este de dezvoltare; pentru o instalare de producție, comanda ar putea fi schimbată în `npm run build && npm run start`.

5.5 Migrațiile bazei de date

Django folosește un sistem propriu de migrații versionabile în `api/migrations/`. La fiecare modificare de `models.py`, comanda `python manage.py makemigrations` generează un fișier numerotat (`0001_initial.py`, `0002_...`, etc.), iar `python manage.py migrate` le aplică succesiv pe baza de date.

Scriptul `migrate.sh` (apelat din `run.sh`) împachetează acest flux: rulează `makemigrations` pentru toate aplicațiile, apoi `migrate` pe fiecare bază de date detectată din `settings.DATABASES`. Scriptul nu așteaptă explicit disponibilitatea PostgreSQL: se bazează pe `depends_on`: `db` din `docker-compose.yml` pentru ordinea de pornire a containerelor, nu pe o verificare activă a conexiunii. Decizia de a apela `makemigrations` la fiecare boot poate părea agresivă, dar în practică nu produce nimic dacă schema este sincronizată cu

modelele. În schimb, dacă un dezvoltator a uitat să comite un fișier de migrație, scriptul îl generează automat la prima rulare în Docker.

5.6 Pași de testare manuală

Pentru cele câteva funcționalități core ale platformei, am parcurs sistematic următoarele verificări manuale:

5.6.1 Test 1: înregistrarea primului utilizator devine owner

1. `docker compose up -d --build`, urmat de `docker compose exec backend python manage.py flush --noinput` pentru a goli baza de date;
2. în browser, deschidere `http://localhost:3000`: `SetupWizard` ar trebui să apară fiindcă `GET /api/system-status/` returnează `is_fresh: true`;
3. completarea formularului din pasul *account*, urmărind toast-urile pentru erori de validare (politica de parolă cere minim 4 caractere și o majusculă);
4. după `POST /api/register/`, login automat și navigare la `/dashboard/settings`: câmpul *role* arată `Owner`.

5.6.2 Test 2: propunere și aprobare de tip de dispozitiv

1. creare a unui al doilea cont, cu rol `user`;
2. în `/dashboard/device-builder`, adăugarea unui ESP32 și a unui DHT11, conectarea pinilor, adăugarea a două widget-uri `TEMPERATURE` și `HUMIDITY`, încărcarea unei imagini de schemă electrică, completarea documentației în Markdown;
3. apăsarea butonului *Propose*: verificarea că backend-ul răspunde 201 `Created` și că în `/dashboard/admin/approvals` (logat ca `owner`) apare propunerea;
4. aprobarea cu *Approve*: verificarea că în `/dashboard/device-types` apare noul tip și că în notificările contului care a aprobat (`owner`) apare mesajul *Device Type Approved*;
5. încercarea de a aproba cu un utilizator `user`: backend-ul răspunde 403 `Forbidden`.

5.6.3 Test 3: adăugarea unui dispozitiv real ESP32

1. flash-uirea unui ESP32 cu firmware-ul *ESP32 5V Relay* extras din fixture (codul Arduino se obține din câmpul `firmware_code` al tipului din `default_device_types.json`, sau direct din UI prin pagina *Device Collection*), după ce `{{WIFI_SSID}}` și `{{WIFI_PASSWORD}}` au fost completate (placeholder-ul `{{SERVER_IP}}` se lasă neschimbat pentru a folosi descoperirea automată prin mDNS, sau se înlocuiește cu IP-ul serverului pentru topologiile pe Docker Desktop);
2. monitorizarea pe Serial Monitor (115200 baud): așteptarea mesajului `Connecting to MQTT broker... connected!` și a primului `Published::`;
3. în `/dashboard`, apăsarea butonului *Add* și introducerea IP-ului ESP32-ului: `POST /api/devices/` returnează 201;

4. după 3 secunde, refresh-ul automat al listei marchează dispozitivul ca *online*, iar coloana `mac.address` a fost completată automat de `mqtt_listener`;
5. apăsarea toggle-ului de pe card: UI-ul se actualizează imediat (optimistic), iar pe Serial Monitor apare mesajul `Command received:` la mai puțin de 100 ms după click.

Rezultatul, cu dispozitivul *online* și cardul actualizat în dashboard, este arătat în Figura 5.1.

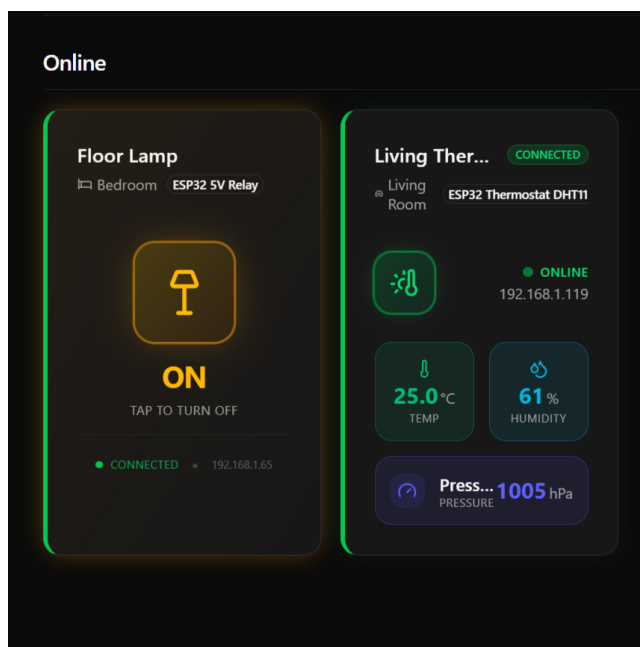


Figura 5.1: Testul 3: dispozitiv ESP32 real adăugat și controlat

5.6.4 Test 4: detecția dispozitivelor offline

1. alimentarea ESP32-ului pornită, dispozitivul este *online*;
2. dezalimentarea ESP32-ului;
3. urmărirea log-urilor backend-ului: după aproximativ 30 de secunde, `mqtt_listener.check_offline_devices()` marchează dispozitivul ca *offline* și apare mesajul *Device jname timed out. Marking OFFLINE.*;
4. în UI, cardul devine gri-decolorat și toggle-ul este dezactivat;
5. realimentarea ESP32: în maxim 5 secunde, primul `publishState()` aduce dispozitivul înapoi la *online*.

5.6.5 Test 5: persistența layout-ului dashboard-ului

1. în modul edit, rearanjarea cardurilor, crearea unui folder cu 3 dispozitive;
2. ieșirea din modul edit: `flushSave` forțează un `PUT /api/dashboard-layout/` imediat;
3. refresh hard de browser (`Ctrl+Shift+R`): layout-ul revine identic;
4. ștergerea cookie-urilor și `localStorage`-ului, urmată de re-login: layout-ul este încă acolo (pentru că este servit de pe server, nu doar din `localStorage`).

5.6.6 Sinteza rezultatelor

Tabelul 5.1 sumarizează observațiile pe cele cinci scenarii descrise mai sus. Demonstrația practică a fost executată pe un laptop Windows 11 cu Docker Desktop (modul *bridge* de dezvoltare), un ESP32 DevKit V1 conectat la rețeaua locală și browser-ul Chromium. Configurația de producție cu `network_mode: host` a fost validată separat pe un Raspberry Pi 4 conectat în aceeași rețea cu plăcile ESP32, pentru a verifica scenariul cu descoperire automată mDNS.

Nr.	Scenariu	Rezultat așteptat	Rezultat observat
1	Primul utilizator → Owner	<code>role = owner,</code> <code>is_superuser = True</code>	Confirmat: wizard → Settings arată badge <i>Owner</i>
2	Propunere → Approve	Tipul apare în <i>Device Types</i> , notificare la contul care a aprobat	Confirmat: notificarea ajunge în mai puțin de 1 s în <i>notification-center</i>
3	ESP32 real conectat	MAC bind automat, comandă → releu	Comandă executată în ~80 ms pe LAN (mediană pe 10 încercări)
4	Detectia offline	Status <i>offline</i> după 30 s	Confirmat: observat în ~32 s în medie
5	Layout persistent	Folder + ordine reluate la re-login	Confirmat pe Chromium, Firefox, Safari

Tabela 5.1: Rezultatele celor cinci scenarii de testare manuală

5.7 Discuție: limitări și considerente de securitate

Stack-ul actual este suficient pentru un mediu de laborator sau pentru o rețea casnică privată. Pentru un deployment în producție, există însă câteva îmbunătățiri necesare:

- **TLS pe broker.** `allow_anonymous true` este acceptabil doar într-un LAN izolat; pentru un mediu real, ar trebui generate certificate (`mosquitto_passwd`) și configurate ACL-uri (`aclfile`) per topic.
- **Politica de parolă.** Cerința actuală de minimum 4 caractere este deliberat permisivă pentru un mediu de demonstrație; o instalare de producție ar utiliza `MinimumLengthValidator(min_length=12)` împreună cu o bibliotecă de *breach detection*.
- **SECRET_KEY și DEBUG.** Fișierul `settings.py` conține valori statice pentru `SECRET_KEY` și `DEBUG = True`, ambele marcate explicit prin comentarii. Pentru deployment-ul în producție, este obligatorie transferarea lor în variabile de mediu și dezactivarea `DEBUG`.
- **CORS.** Setarea `CORS_ALLOW_ALL_ORIGINS = True` este orientată către simplitatea dezvoltării; în producție, lista originilor permise trebuie restrânsă la domeniul instanței.

Modificările enumerate au un impact redus asupra codului, însă diferențiază clar un mediu didactic de unul de producție. În contextul prezentei lucrări, configurația a fost menținută în varianta permisivă pentru a permite ca obiectul principal al expunerii să rămână arhitectura platformei.

Concluzii și direcții viitoare

Rezumatul lucrării

Lucrarea a descris proiectarea și implementarea platformei HomeForge, un sistem Smart Home cu execuție locală, conceput simultan ca aplicație funcțională și ca material didactic. Spre deosebire de soluțiile comerciale dependente de servicii cloud sau de platformele open-source mature precum Home Assistant (caracterizate de un nivel ridicat de abstractizare care reduce accesibilitatea componentelor interne pentru un dezvoltator în formare), HomeForge menține o stivă tehnologică redusă și un flux de date observabil end-to-end.

Obiectivele declarate în Introducere au fost atinse după cum urmează:

- **Firmware ESP32** modular, cu patru variante template (releu, termostat cu DHT11, termostat cu BMP180, variantă combinată) și un protocol de comunicație simplu peste MQTT, descris în Capitolul 4.
- **Backend Django** cu un API REST complet, autentificare bazată pe JWT, sistem ierarhic de roluri `owner/admin/user` și un proces MQTT dedicat care leagă stratul hardware de baza de date PostgreSQL (Capitolul 2).
- **Frontend Next.js** cu dashboard interactiv, mecanism de optimist UI pe carduri, layout persistat pe server, editor vizual pentru tipuri de dispozitive și vizualizare a topologiei prin React Flow (Capitolul 3).
- **Deployment containerizat** printr-o singură comandă `docker compose up`, completat cu un mod de dezvoltare nativ (Capitolul 5).

Contribuții proprii

La nivel cantitativ, contribuția proprie acoperă întreaga stivă: aproximativ 4900 de linii de Python pentru backend (`api/`), peste 20000 de linii TypeScript și JavaScript pentru frontend (`frontend/`) și cele patru proiecte de firmware ESP32 (între 300 și 350 de linii fiecare). Pe lângă volumul de cod, câteva elemente arhitecturale particulare reprezintă contribuții substanțiale ale acestei lucrări:

1. **Stocare fără schemă fixă peste RDBMS.** Starea fiecărui dispozitiv este stocată într-un `JSONField` mapat pe tipul nativ `JSONB` din PostgreSQL, în timp ce relațiile structurale (utilizator, cameră, tip de dispozitiv) rămân normalizate. Adăugarea unui nou tip de senzor nu necesită migrații SQL, iar mecanismele de control al accesului se sprijină în continuare pe integritatea referențială a schemei relaționale.
2. **Descoperire automată a brokerului prin mDNS / DNS-SD [7], [8].** Firmware-ul ESP32 folosește o ierarhie cu patru niveluri de descoperire (override manual prin `server_ip`, cache local în NVS, interogare `_mqtt._tcp.local` și endpoint web `/config`), eliminând necesitatea reflash-ării plăcilor la fiecare schimbare de adresă IP a serverului. În deployment-ul de producție pe Linux nativ, mecanismul oferă o experiență de tip *plug-and-play* pentru utilizator.

3. **Auto-binding MAC-IP.** Mecanismul prin care procesul `mqtt_listener` asociază automat un dispozitiv recent înregistrat (cunoscut inițial doar prin adresa IP) cu prima publicare MQTT, care conține adresa MAC. Astfel, utilizatorul nu trebuie să introducă manual adresa MAC, eliminând o sursă frecventă de erori.
4. **Mapare „chei standard → identificatori widget”.** Firmware-ul publică datele sub denumiri canonice (`temperature`, `humidity`, `pressure`, `relay_1`), iar procesul de ascultare le traduce în identificatorii dinamici generați de *device builder*. Drept consecință, reorganizarea widget-urilor pe card nu impune reflash-area firmware-ului.
5. **Integrare de sisteme externe de panouri solare.** Modulul `api/solar/` introduce un pattern adapter/registry care normalizează telemetria primită de la API-uri terțe într-o schemă unitară de tip „power-flow”. Prima implementare suportă oficial API-ul Fronius Solar V1 [11]; arhitectura este pregătită pentru adăugarea altor furnizori printr-o singură subclasă, fără modificări la nivelul interfeței sau al modelului de date.
6. **Optimistic UI cu rollback.** Implementarea cardurilor inteligente prin `onMutate`, `onError` și `onSettled` din TanStack Query, în corespondență cu răspunsul HTTP 202 Accepted din backend, oferă feedback vizual imediat și revine corect la starea anterioară în cazul în care dispozitivul nu confirmă comanda. Mecanismul `matchesOptimistic` elimină situațiile în care interfața ar rămâne blocată în starea de sincronizare.
7. **Device builder vizual.** Editorul bazat pe un graf de noduri și un panou de widget-uri, dublat de o validare server-side care impune ca fiecare `variable_mapping` să corespundă unui ID definit în `structure[]`, permite contribuția unor tipuri noi de dispozitive fără modificări la nivelul codului platformei. Paleta este sincronizată cu un endpoint dedicat (`/widget-types/`) care separă explicit widget-urile suportate end-to-end de cele rezervate pentru extensii ulterioare.
8. **Layout cu cascadă de fallback.** Mecanismul ierarhic *layout personal* → *layout partajat de administrator* → *implicit calculat client-side* permite administratorilor să stabilească o configurație inițială pentru utilizatorii noi, fără a restricționa personalizarea ulterioară a fiecăruia.
9. **Deployment dual (bridge dev / host prod).** Configurarea Docker Compose în două variante complementare (bridge networking pentru dezvoltare cross-platform pe Docker Desktop Windows/macOS și `network_mode: host` pentru producție pe Linux nativ) permite obținerea simultană a portabilității în mediul de dezvoltare și a funcționalității complete de auto-discovery mDNS în deployment-ul real. Validarea pe Raspberry Pi 4 a confirmat funcționarea ambelor moduri.

Limitări cunoscute

Versiunea actuală a platformei prezintă o serie de limitări care trebuie semnalate explicit.

Securitate.

După cum s-a detaliat în Capitolul 5, brokerul Mosquitto este configurat cu `allow_anonymous true`, comunicarea MQTT nu folosește TLS, iar variabilele Django sensibile (`SECRET_KEY`, `DEBUG`) au valori statice. Aceste setări sunt acceptabile pentru un mediu de laborator, dar nu și pentru o instanță expusă în internet.

Lipsa OTA și a captive portal-ului.

Reconfigurarea sau actualizarea unui ESP32 presupune conectarea fizică prin USB. O versiune ulterioară ar putea integra ArduinoOTA sau ESPHome pentru actualizări la distanță.

Scalabilitate.

Cache-urile sunt locale (LocMemCache din Django), brokerul Mosquitto rulează în același container cu Django, iar baza de date este unică. Limita practică a implementării curente este de ordinul a câteva sute de dispozitive simultan, ceea ce acoperă scenariile rezidențiale, dar nu și pe cele de scară superioară.

Absența automatizărilor.

HomeForge expune dispozitivele și permite controlul manual, dar nu include încă un motor de reguli de tip *trigger* → *condition* → *action*, cum oferă Home Assistant. Adăugarea acestuia este o direcție evidentă de continuare.

Testare automată.

Repository-ul conține fișierul `tests.py`, dar acoperirea testelor unitare și de integrare rămâne modestă. O eventuală evoluție către un proiect mentenabil pe termen lung impune extinderea suitei pentru a acoperi serializările principale, fluxurile RBAC și mecanismele din `mqtt_listener`.

Direcții viitoare

Pe baza limitărilor identificate, se pot evidenția șase direcții de dezvoltare ulterioară.

1. **TLS și autentificare MQTT.** Configurarea Mosquitto cu certificate generate automat, integrate într-un asistent de inițializare. Firmware-ul ar putea utiliza `WiFiClientSecure` cu un `rootCA` predefinit.
2. **Motor de automatizări.** Un model Django `Automation` cu câmpurile `trigger_topic`, `condition_expr`, `action_topic` și `action_payload`, evaluat de un proces care ascultă același flux MQTT ca `mqtt_listener`. Pe frontend, un editor vizual analog cu device builder-ul.
3. **Suport pentru protocoale suplimentare.** Adăugarea unui adaptor pentru Matter sau Zigbee2MQTT ar permite integrarea dispozitivelor comerciale, fără abandonarea principiului *local-first*. Adaptoarele ar publica pe aceleași topice `homeforge/devices/...`, astfel încât restul stivei să rămână nemodificat.
4. **Extinderea modului solar la noi furnizori.** Pattern-ul `adapter/registry` implementat în `api/solar/` este pregătit pentru integrarea unor furnizori suplimentari prin scrierea unei singure subclase `BaseSolarProvider`, fără modificări la nivelul interfeței. Candidați naturali pentru o etapă ulterioară sunt invertoarele SolarEdge (API `monitoringapi.solaredge.com`), Huawei FusionSolar (Northbound API) și Enphase Envoy (API local pe gateway), fiecare expunând date similare ca structură celor publicate de Fronius. Schema normalizată `power/battery/energy/ratios` a fost proiectată generic tocmai pentru a putea acomoda aceste surse fără modificarea contractului cu frontend-ul.

5. **Mod public read-only.** Posibilitatea expunerii unei pagini publice (semnată cu un token) care să prezinte un subset de dispozitive în regim read-only, utilă pentru monitorizare la distanță fără a expune întreaga aplicație.
6. **OTA și captive portal Wi-Fi pe ESP32.** Înlocuirea reflash-ării prin USB cu un sistem de actualizări over-the-air (ArduinoOTA sau ESPHome), respectiv adăugarea unui captive portal care să permită configurarea credențialelor Wi-Fi la prima pornire, fără modificări în codul sursă.
7. **Pipeline CI/CD.** Un flux GitHub Actions care să ruleze `python manage.py test`, `npm run build` și `eslint` la fiecare *pull request*, completat cu construirea imaginilor Docker la fiecare tag.

Învățări personale

Implementarea acestei platforme a oferit ocazia parcurgerii practice a etapelor de cuplare între o stivă web modernă și un strat embedded. Patru observații se desprind ca fiind relevante pentru proiecte similare. Prima: definirea precisă a contractelor de date este mai importantă decât alegerea unui protocol anume: dezbateră „REST vs. MQTT vs. WebSocket” devine secundară odată ce este clar ce informație circulă între componente. A doua: optimistic UI nu este o decorație vizuală, ci o strategie de gestiune a timpului perceput de utilizator, iar un mecanism de rollback robust este la fel de important ca redarea instantanee inițială. A treia: transparența arhitecturală generează un cost de implementare, dar simplifică considerabil etapa de depanare: observarea directă a traficului MQTT cu `mosquitto_sub` s-a dovedit utilă în mod repetat. A patra: o bază de date relațională extinsă cu JSONB reprezintă, în numeroase scenarii, o alternativă preferabilă unei baze NoSQL pure, oferind flexibilitate fără pierderea garanțiilor ACID sau a ergonomiei unui ORM matur.

Reflecție finală

HomeForge nu își propune să acopere integralitatea funcționalităților unei platforme Smart Home comerciale. Rolul său este intermediar: face legătura între noțiunile teoretice predate la cursurile de rețele, sisteme distribuite și baze de date și aplicarea lor concretă într-un sistem care interacționează cu mediul fizic. Considerând obiectivul didactic, lucrarea poate fi utilă oricărui student sau dezvoltator care dorește să își configureze un ESP32 propriu și să urmărească, linie cu linie, traseul unei valori senzoriale de la dispozitiv până la dashboard.

Codul sursă rămâne disponibil în repository-ul proiectului, deschis utilizării, modificării și extinderii ulterioare în contextul educațional și academic.

Bibliografie

- [1] M. Alaa, A. A. Zaidan, B. B. Zaidan, M. Talal și M. L. M. Kiah, „A review of smart home applications based on Internet of Things,” *Journal of Network and Computer Applications*, vol. 97, pp. 48–65, 2017.
- [2] D. Marikyan, S. Papagiannidis și E. Alamanos, „A systematic review of smart home literature: A user perspective,” *Technological Forecasting and Social Change*, vol. 138, pp. 139–154, 2019. DOI: 10.1016/j.techfore.2018.08.015
- [3] B. Mishra și A. Kertész, „The Use of MQTT in M2M and IoT Systems: A Survey,” *IEEE Access*, vol. 8, pp. 201 071–201 086, 2020. DOI: 10.1109/ACCESS.2020.3035849
- [4] M. Noura, M. Atiquzzaman și M. Gaedke, „Interoperability in Internet of Things: Taxonomies and Open Challenges,” *Mobile Networks and Applications*, vol. 24, nr. 3, pp. 796–809, 2019. DOI: 10.1007/s11036-018-1089-9
- [5] A. Dorri, S. S. Kanhere, R. Jurdak și P. Gauravaram, „Blockchain for IoT security and privacy: The case study of a smart home,” în *2017 IEEE international conference on pervasive computing and communications workshops (PerCom workshops)*, IEEE, 2017, pp. 618–623.
- [6] I. Sittón-Candanedo, R. S. Alonso, J. M. Corchado, S. Rodríguez-González și R. Casado-Vara, „A review of edge computing reference architectures and a new global edge proposal,” *Future Generation Computer Systems*, vol. 99, pp. 278–294, 2019. DOI: 10.1016/j.future.2019.04.016
- [7] S. Cheshire și M. Krochmal, *RFC 6762: Multicast DNS*, IETF Standards Track, feb. 2013. DOI: 10.17487/RFC6762 adresa: <https://datatracker.ietf.org/doc/html/rfc6762>
- [8] S. Cheshire și M. Krochmal, *RFC 6763: DNS-Based Service Discovery*, IETF Standards Track, feb. 2013. DOI: 10.17487/RFC6763 adresa: <https://datatracker.ietf.org/doc/html/rfc6763>
- [9] R. T. Fielding, „Architectural Styles and the Design of Network-based Software Architectures,” Teză de doct., University of California, Irvine, 2000. adresa: <https://roy.gbiv.com/pubs/dissertation/top.htm>
- [10] M. Jones, J. Bradley și N. Sakimura, *RFC 7519: JSON Web Token (JWT)*, IETF Standards Track, mai 2015. DOI: 10.17487/RFC7519 adresa: <https://datatracker.ietf.org/doc/html/rfc7519>
- [11] Fronius International GmbH, *Fronius Solar API V1: Operating Instructions*, Document 42,0410,2012, 2021. adresa: <https://www.fronius.com/~/downloads/Solar%5C%20Energy/Operating%5C%20Instructions/42,0410,2012.pdf>
- [12] PostgreSQL Global Development Group, *PostgreSQL 15 Documentation: Chapter 8.14, JSON Types*, Accessed: 2026-05, 2023. adresa: <https://www.postgresql.org/docs/15/datatype-json.html>
- [13] N. Naik, „Choice of effective messaging protocols for IoT systems: MQTT, CoAP, AMQP and HTTP,” în *2017 IEEE International Systems Engineering Symposium (ISSE)*, IEEE, Vienna, Austria, 2017, pp. 1–7. DOI: 10.1109/SysEng.2017.8088251

- [14] P. Bellavista și A. Zanni, „Feasibility of Fog Computing Deployment based on Docker Containerization over RaspberryPi,” în *Proceedings of the 18th International Conference on Distributed Computing and Networking (ICDCN '17)*, ACM, Hyderabad, India, 2017, pp. 1–10. DOI: 10.1145/3007748.3007777
- [15] C. Pautasso, O. Zimmermann și F. Leymann, „RESTful Web Services vs. “Big” Web Services: Making the Right Architectural Decision,” în *Proceedings of the 17th International Conference on World Wide Web (WWW '08)*, ACM, Beijing, China, 2008, pp. 805–814. DOI: 10.1145/1367497.1367606
- [16] J. Nielsen, *Response Times: The 3 Important Limits*, Nielsen Norman Group, Originally published in *Usability Engineering*, Morgan Kaufmann, 1993; reprinted online 2010., 1993. adresa: <https://www.nngroup.com/articles/response-times-3-important-limits/>

Anexa A

Backend

A.1 Modelele de date (api/models.py)

```
1 from django.db import models
2 from django.conf import settings
3 from django.dispatch import receiver
4 from django.db.models.signals import post_save, pre_save
5 import os
6 import uuid
7
8 def avatar_upload_path(instance, filename):
9     """Generate a unique path for the avatar using UUID."""
10    ext = filename.split('.')[-1]
11    filename = f"{uuid.uuid4()}.{ext}"
12    return os.path.join('avatars/', filename)
13
14 class Profile(models.Model):
15     """Extended profile for users: avatar image and role for dashboard permissions."""
16     ROLE_OWNER = 'owner'
17     ROLE_ADMIN = 'admin'
18     ROLE_USER = 'user'
19
20     ROLE_CHOICES = [
21         (ROLE_OWNER, 'Owner'),
22         (ROLE_ADMIN, 'Admin'),
23         (ROLE_USER, 'User'),
24     ]
25
26     user = models.OneToOneField(settings.AUTH_USER_MODEL, on_delete=models.CASCADE, related_name='profile')
27     avatar = models.ImageField(upload_to=avatar_upload_path, null=True, blank=True)
28     role = models.CharField(max_length=20, choices=ROLE_CHOICES, default=ROLE_USER)
29     accent_color = models.CharField(max_length=7, default='#3B82F6', blank=True)
30
31     def __str__(self):
32         return f"Profile({self.user.username})"
33
34 @receiver(pre_save, sender=Profile)
35 def delete_old_avatar_on_change(sender, instance, **kwargs):
36     """Delete the old avatar file from the filesystem when a new one is uploaded."""
37     if not instance.pk:
38         return False
39
40     try:
41         old_profile = Profile.objects.get(pk=instance.pk)
42         old_avatar = old_profile.avatar
43     except Profile.DoesNotExist:
44         return False
45
46     new_avatar = instance.avatar
47     if old_avatar and old_avatar != new_avatar:
48         if old_avatar and os.path.isfile(old_avatar.path):
49             os.remove(old_avatar.path)
50
51 @receiver(post_save, sender=settings.AUTH_USER_MODEL)
52 def create_user_profile(sender, instance, created, **kwargs):
53     if created:
54         Profile.objects.create(user=instance)
55
56 @receiver(post_save, sender=settings.AUTH_USER_MODEL)
57 def save_user_profile(sender, instance, **kwargs):
58     # Ensure profile exists and is saved whenever the user is saved
59     try:
60         instance.profile.save()
61     except Exception:
62         # If profile doesn't exist for some reason, create it
```

```

63     Profile.objects.get_or_create(user=instance)
64
65 class Room(models.Model):
66     name = models.CharField(max_length=100)
67     icon = models.CharField(max_length=50, default='fa-door-open', blank=True, help_text="FontAwesome icon class
        ↳ (e.g., fa-bed, fa-couch)")
68     user = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE, related_name='rooms')
69
70     class Meta:
71         unique_together = ['name', 'user']
72
73     def __str__(self):
74         return f"{self.name} ({self.user.username})"
75
76 class Device(models.Model):
77     STATUS_ONLINE = 'online'
78     STATUS_OFFLINE = 'offline'
79     STATUS_ERROR = 'error'
80     STATUS_CHOICES = [
81         (STATUS_ONLINE, 'Online'),
82         (STATUS_OFFLINE, 'Offline'),
83         (STATUS_ERROR, 'Error'),
84     ]
85
86     name = models.CharField(max_length=100)
87     ip_address = models.GenericIPAddressField(protocol='IPv4')
88     mac_address = models.CharField(max_length=17, blank=True, null=True, unique=True, help_text="MAC Address
        ↳ (automatic from device)")
89     status = models.CharField(max_length=10, choices=STATUS_CHOICES, default=STATUS_OFFLINE, db_index=True)
90     icon = models.CharField(max_length=50, default='fa-cube', blank=True, help_text="FontAwesome icon class
        ↳ (e.g., fa-lightbulb)")
91     device_type = models.ForeignKey('CustomDeviceType', on_delete=models.CASCADE, related_name='devices')
92     room = models.ForeignKey(Room, on_delete=models.SET_NULL, null=True, blank=True, related_name='devices')
93     user = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE, related_name='devices',
        ↳ db_index=True)
94     current_state = models.JSONField(default=dict, blank=True, help_text="Current state of device controls
        ↳ (e.g., {'relay_1': True})")
95     updated_at = models.DateTimeField(auto_now=True, help_text="Last time the device reported state")
96
97     class Meta:
98         ordering = ['-id']
99         indexes = [
100             models.Index(fields=['user', 'status'], name='device_user_status_idx'),
101             models.Index(fields=['room'], name='device_room_idx'),
102             models.Index(fields=['device_type'], name='device_type_idx'),
103         ]
104
105     def __str__(self):
106         return f"{self.name} - {self.ip_address}"
107
108 def wiring_diagram_upload_path(instance, filename):
109     """Legacy: kept for migration compatibility only. No longer used."""
110     return f"wiring/{filename}"
111
112 class CustomDeviceType(models.Model):
113     name = models.CharField(max_length=100, unique=True)
114     definition = models.JSONField(default=dict, blank=True)
115     approved = models.BooleanField(default=False, db_index=True)
116     rejection_reason = models.TextField(blank=True, null=True, help_text="Reason for rejection if denied Admin
        ↳ approval")
117     proposed_by = models.ForeignKey(
118         settings.AUTH_USER_MODEL,
119         on_delete=models.SET_NULL,
120         null=True,
121         blank=True,
122         related_name='proposed_device_types',
123         help_text="User who proposed this device type"
124     )
125     firmware_code = models.TextField(blank=True, default='')
126     wiring_diagram_base64 = models.TextField(blank=True, default='', help_text="Base64 data URI of wiring
        ↳ diagram image, stored directly in DB")
127     wiring_diagram_text = models.TextField(blank=True, default='')
128     documentation = models.TextField(blank=True, default='')
129     documentation_images_base64 = models.JSONField(default=list, blank=True, help_text="Array of {filename,
        ↳ data} for documentation images, stored directly in DB")
130     created_at = models.DateTimeField(auto_now_add=True)
131
132     class Meta:

```

```

133         ordering = ['-created_at']
134         indexes = [
135             models.Index(fields=['approved', 'created_at'], name='devicetype_approved_idx'),
136         ]
137
138     def __str__(self):
139         return f"{self.name} ({'Approved' if self.approved else 'Pending'})"
140
141 class DeviceCardTemplate(models.Model):
142     """
143     Defines the UI layout for a specific device type card.
144     One-to-One with CustomDeviceType.
145     """
146     device_type = models.OneToOneField(CustomDeviceType, on_delete=models.CASCADE, related_name='card_template')
147     layout_config = models.JSONField(default=dict, blank=True, help_text="Frontend grid layout configuration (x, y, w, h)")
148
149     def __str__(self):
150         return f"Template for {self.device_type.name}"
151
152 class DeviceControl(models.Model):
153     """
154     Individual control widgets (sliders, toggles, sensors) for the device card.
155     """
156     # Interactive Controls
157     WIDGET_TOGGLE = 'TOGGLE'
158     WIDGET_SLIDER = 'SLIDER'
159     WIDGET_BUTTON = 'BUTTON'
160
161     # Display/Sensor Widgets
162     WIDGET_GAUGE = 'GAUGE'
163     WIDGET_TEMPERATURE = 'TEMPERATURE'
164     WIDGET_HUMIDITY = 'HUMIDITY'
165     WIDGET_MOTION = 'MOTION'
166     WIDGET_LIGHT = 'LIGHT'
167     WIDGET_CO2 = 'CO2'
168     WIDGET_PRESSURE = 'PRESSURE'
169     WIDGET_POWER = 'POWER'
170     WIDGET_BATTERY = 'BATTERY'
171     WIDGET_STATUS = 'STATUS'
172
173     # Widget types with full backend + firmware + fixture support today.
174     # Keep this in sync with the device builder palette the frontend exposes.
175     SUPPORTED_WIDGET_TYPES = [
176         WIDGET_TOGGLE,          # relay
177         WIDGET_SLIDER,
178         WIDGET_BUTTON,
179         WIDGET_GAUGE,
180         WIDGET_TEMPERATURE,
181         WIDGET_HUMIDITY,
182         WIDGET_PRESSURE,
183         WIDGET_POWER,
184         WIDGET_BATTERY,
185         WIDGET_STATUS,
186     ]
187
188     # Planned for future implementation, not yet wired to firmware.
189     FUTURE_WIDGET_TYPES = [
190         WIDGET_MOTION,
191         WIDGET_CO2,
192         WIDGET_LIGHT,
193     ]
194
195     WIDGET_CHOICES = [
196         # Interactive
197         (WIDGET_TOGGLE, 'Toggle Switch'),
198         (WIDGET_SLIDER, 'Slider'),
199         (WIDGET_BUTTON, 'Button'),
200         # Sensors/Display
201         (WIDGET_GAUGE, 'Gauge'),
202         (WIDGET_TEMPERATURE, 'Temperature Sensor'),
203         (WIDGET_HUMIDITY, 'Humidity Sensor'),
204         (WIDGET_PRESSURE, 'Pressure Sensor'),
205         (WIDGET_POWER, 'Power Meter'),
206         (WIDGET_BATTERY, 'Battery Level'),
207         (WIDGET_STATUS, 'Status Indicator'),
208         # Future implementations (kept valid, surfaced as "coming soon" in the UI)
209         (WIDGET_MOTION, 'Motion Sensor (Coming Soon)'),

```

```

210         (WIDGET_CO2, 'CO2 Sensor (Coming Soon)'),
211         (WIDGET_LIGHT, 'Light Sensor (Coming Soon)'),
212     ]
213
214     # Display variant options
215     VARIANT_ROW = 'row'
216     VARIANT_SQUARE = 'square'
217     VARIANT_COMPACT = 'compact'
218
219     VARIANT_CHOICES = [
220         (VARIANT_ROW, 'Row'),
221         (VARIANT_SQUARE, 'Square'),
222         (VARIANT_COMPACT, 'Compact'),
223     ]
224
225     # Size options
226     SIZE_SM = 'sm'
227     SIZE_MD = 'md'
228     SIZE_LG = 'lg'
229
230     SIZE_CHOICES = [
231         (SIZE_SM, 'Small'),
232         (SIZE_MD, 'Medium'),
233         (SIZE_LG, 'Large'),
234     ]
235
236     template = models.ForeignKey(DeviceCardTemplate, on_delete=models.CASCADE, related_name='controls')
237     widget_type = models.CharField(max_length=20, choices=WIDGET_CHOICES)
238     label = models.CharField(max_length=50)
239     variable_mapping = models.CharField(max_length=50, help_text="MQTT/API variable key (e.g., relay_1)")
240
241     # Slider/Gauge specific fields
242     min_value = models.FloatField(null=True, blank=True)
243     max_value = models.FloatField(null=True, blank=True)
244     step = models.FloatField(null=True, blank=True)
245
246     # New optional display fields
247     variant = models.CharField(max_length=20, choices=VARIANT_CHOICES, null=True, blank=True, help_text="Display
↳ variant: row, square, or compact")
248     size = models.CharField(max_length=10, choices=SIZE_CHOICES, null=True, blank=True, help_text="Widget size:
↳ sm, md, or lg")
249     unit = models.CharField(max_length=20, null=True, blank=True, help_text="Display unit (e.g., degC, %, lux,
↳ ppm)")
250
251     class Meta:
252         # Ensure a variable name is unique within a single device type template
253         unique_together = ('template', 'variable_mapping')
254
255     def __str__(self):
256         return f"{self.label} ({self.widget_type})"
257
258     class Notification(models.Model):
259         """
260         Notification model for user alerts and system messages.
261         Supports different notification types with flexible metadata.
262         """
263         # Notification Types
264         TYPE_DEVICE_TYPE_PENDING = 'device_type_pending' # Admin: new type awaiting approval
265         TYPE_DEVICE_TYPE_APPROVED = 'device_type_approved' # User: your type was approved
266         TYPE_DEVICE_TYPE_DENIED = 'device_type_denied' # User: your type was denied
267         TYPE_DEVICE_OFFLINE = 'device_offline' # Device went offline
268         TYPE_DEVICE_ONLINE = 'device_online' # Device came online
269         TYPE_DEVICE_ERROR = 'device_error' # Device has an error
270         TYPE_SYSTEM = 'system' # General system notification
271         TYPE_INFO = 'info' # Informational message
272         TYPE_WARNING = 'warning' # Warning message
273         TYPE_ERROR = 'error' # Error message
274
275         TYPE_CHOICES = [
276             (TYPE_DEVICE_TYPE_PENDING, 'Device Type Pending'),
277             (TYPE_DEVICE_TYPE_APPROVED, 'Device Type Approved'),
278             (TYPE_DEVICE_TYPE_DENIED, 'Device Type Denied'),
279             (TYPE_DEVICE_OFFLINE, 'Device Offline'),
280             (TYPE_DEVICE_ONLINE, 'Device Online'),
281             (TYPE_DEVICE_ERROR, 'Device Error'),
282             (TYPE_SYSTEM, 'System'),
283             (TYPE_INFO, 'Info'),
284             (TYPE_WARNING, 'Warning'),

```

```

285         (TYPE_ERROR, 'Error'),
286     ]
287
288     # Priority levels
289     PRIORITY_LOW = 'low'
290     PRIORITY_NORMAL = 'normal'
291     PRIORITY_HIGH = 'high'
292     PRIORITY_URGENT = 'urgent'
293
294     PRIORITY_CHOICES = [
295         (PRIORITY_LOW, 'Low'),
296         (PRIORITY_NORMAL, 'Normal'),
297         (PRIORITY_HIGH, 'High'),
298         (PRIORITY_URGENT, 'Urgent'),
299     ]
300
301     # Core fields
302     user = models.ForeignKey(
303         settings.AUTH_USER_MODEL,
304         on_delete=models.CASCADE,
305         related_name='notifications',
306         help_text="The user who receives this notification"
307     )
308     notification_type = models.CharField(
309         max_length=30,
310         choices=TYPE_CHOICES,
311         default=TYPE_INFO,
312         db_index=True
313     )
314     title = models.CharField(max_length=200)
315     message = models.TextField()
316     priority = models.CharField(
317         max_length=10,
318         choices=PRIORITY_CHOICES,
319         default=PRIORITY_NORMAL
320     )
321
322     # Status
323     is_read = models.BooleanField(default=False, db_index=True)
324
325     # Optional reference to related objects (stored as JSON for flexibility)
326     # e.g., {"device_type_id": 5} or {"device_id": 12}
327     reference_data = models.JSONField(default=dict, blank=True)
328
329     # Link for action (optional)
330     action_url = models.CharField(max_length=500, blank=True, null=True)
331
332     # Timestamps
333     created_at = models.DateTimeField(auto_now_add=True)
334     read_at = models.DateTimeField(null=True, blank=True)
335
336     class Meta:
337         ordering = ['-created_at']
338         indexes = [
339             models.Index(fields=['user', 'is_read'], name='notif_user_read_idx'),
340             models.Index(fields=['user', 'notification_type'], name='notif_user_type_idx'),
341             models.Index(fields=['created_at'], name='notif_created_idx'),
342         ]
343
344     def __str__(self):
345         status = "Read" if self.is_read else "Unread"
346         return f"[{status}] {self.title} - {self.user.username}"
347
348     @property
349     def time_ago(self):
350         """Human-readable time since creation."""
351         from django.utils import timezone
352         from datetime import timedelta
353         diff = timezone.now() - self.created_at
354         if diff < timedelta(minutes=1):
355             return "Just now"
356         elif diff < timedelta(hours=1):
357             return f"{int(diff.total_seconds() / 60)}m ago"
358         elif diff < timedelta(days=1):
359             return f"{int(diff.total_seconds() / 3600)}h ago"
360         elif diff < timedelta(days=7):
361             return f"{diff.days}d ago"
362         return self.created_at.strftime("%b %d, %Y")

```

```

363
364 def mark_as_read(self):
365     """Mark this notification as read."""
366     if not self.is_read:
367         from django.utils import timezone
368         self.is_read = True
369         self.read_at = timezone.now()
370         self.save(update_fields=['is_read', 'read_at'])
371
372 @classmethod
373 def create_notification(cls, user, notification_type, title, message,
374                        priority=None, reference_data=None, action_url=None):
375     """
376     Helper method to create a notification.
377     """
378     return cls.objects.create(
379         user=user,
380         notification_type=notification_type,
381         title=title,
382         message=message,
383         priority=priority or cls.PRIORITY_NORMAL,
384         reference_data=reference_data or {},
385         action_url=action_url
386     )
387
388 @classmethod
389 def notify_admins(cls, notification_type, title, message,
390                  priority=None, reference_data=None, action_url=None):
391     """
392     Send a notification to all admin and owner users.
393     """
394     from django.contrib.auth import get_user_model
395     User = get_user_model()
396
397     admin_profiles = Profile.objects.filter(role__in=[Profile.ROLE_ADMIN, Profile.ROLE_OWNER])
398     admin_users = User.objects.filter(profile__in=admin_profiles)
399
400     notifications = []
401     for user in admin_users:
402         notifications.append(cls(
403             user=user,
404             notification_type=notification_type,
405             title=title,
406             message=message,
407             priority=priority or cls.PRIORITY_NORMAL,
408             reference_data=reference_data or {},
409             action_url=action_url
410         ))
411
412     return cls.objects.bulk_create(notifications)
413
414 class DashboardLayout(models.Model):
415     """
416     Stores dashboard grid layout for a user (personal) or system-wide (shared).
417     When user is NULL, this is the shared/default layout visible to all users
418     who don't have a personal layout.
419     """
420     user = models.OneToOneField(
421         settings.AUTH_USER_MODEL,
422         on_delete=models.CASCADE,
423         null=True,
424         blank=True,
425         unique=True,
426         related_name='dashboard_layout',
427         help_text="Owner of this layout. NULL = shared/default layout."
428     )
429     ORDER_ROOM = 'room'
430     ORDER_TYPE = 'type'
431     ORDER_STATUS = 'status'
432     ORDER_NAME = 'name'
433     ORDER_CUSTOM = 'custom'
434
435     ORDER_CHOICES = [
436         (ORDER_ROOM, 'Group by Room'),
437         (ORDER_TYPE, 'Group by Device Type'),
438         (ORDER_STATUS, 'Group by Status'),
439         (ORDER_NAME, 'Alphabetical by Name'),
440         (ORDER_CUSTOM, 'Custom Order'),

```

```

441 ]
442
443 layout = models.JSONField(
444     help_text="Dashboard layout JSON: {version, items[]}"
445 )
446 device_order = models.CharField(
447     max_length=20,
448     choices=ORDER_CHOICES,
449     default=ORDER_ROOM,
450     help_text="Device grouping/sorting preference for the dashboard."
451 )
452 updated_at = models.DateTimeField(auto_now=True)
453
454 class Meta:
455     ordering = ['-updated_at']
456     constraints = [
457         # Ensure only one shared layout (user=NULL) exists
458         models.UniqueConstraint(
459             fields=['user'],
460             condition=models.Q(user__isnull=True),
461             name='unique_shared_layout',
462         ),
463     ]
464
465 def __str__(self):
466     if self.user:
467         return f"Layout for {self.user.username}"
468     return "Shared Layout"
469
470 @property
471 def is_personal(self):
472     return self.user is not None
473
474 class SolarSystem(models.Model):
475     """
476     A registered external solar/energy data source (e.g. a Fronius inverter).
477
478     We store ONLY the base link to the vendor's raw-JSON API plus a provider
479     discriminator. The backend fetches and normalizes the vendor JSON into a
480     provider-agnostic schema (see api/solar/), so a single reusable power-flow
481     UI works across providers.
482     """
483     PROVIDER_FRONIUS = 'fronius'
484     PROVIDER_CHOICES = [
485         (PROVIDER_FRONIUS, 'Fronius Solar API V1'),
486     ]
487
488     name = models.CharField(max_length=100, help_text="Friendly name for the solar system")
489     base_url = models.URLField(
490         max_length=500,
491         help_text="Base link to the vendor API, e.g. http://fronius-server:9999"
492     )
493     provider = models.CharField(
494         max_length=30,
495         choices=PROVIDER_CHOICES,
496         default=PROVIDER_FRONIUS,
497         help_text="Vendor adapter used to normalize the raw JSON"
498     )
499     user = models.ForeignKey(
500         settings.AUTH_USER_MODEL,
501         on_delete=models.CASCADE,
502         related_name='solar_systems',
503         db_index=True
504     )
505     enabled = models.BooleanField(default=True, db_index=True)
506
507     # Non-authoritative cached discovery metadata (populated on register/refresh)
508     api_version = models.CharField(max_length=20, blank=True, default='')
509     capabilities = models.JSONField(
510         default=dict,
511         blank=True,
512         help_text="Cached capability flags, e.g. {'battery': true, 'meter': true}"
513     )
514     last_seen = models.DateTimeField(null=True, blank=True, help_text="Last successful fetch")
515
516     created_at = models.DateTimeField(auto_now_add=True)
517     updated_at = models.DateTimeField(auto_now=True)
518

```

```

519     class Meta:
520         ordering = ['-created_at']
521         unique_together = ['base_url', 'user']
522
523     def __str__(self):
524         return f"{self.name} ({self.provider})"

```

A.2 Serializare și validare (api/serializers.py)

```

1  from rest_framework import serializers
2  from rest_framework.validators import UniqueValidator
3  from django.contrib.auth.models import User
4  from django.contrib.auth.password_validation import validate_password
5  from .models import Profile, Device, Room, CustomDeviceType, DeviceCardTemplate, DeviceControl, Notification,
   ↪ DashboardLayout, SolarSystem
6
7  class DeviceControlSerializer(serializers.ModelSerializer):
8      class Meta:
9          model = DeviceControl
10         fields = ['id', 'widget_type', 'label', 'variable_mapping', 'min_value', 'max_value', 'step', 'variant',
   ↪ 'size', 'unit']
11         extra_kwargs = {
12             'min_value': {'required': False},
13             'max_value': {'required': False},
14             'step': {'required': False},
15             'variant': {'required': False},
16             'size': {'required': False},
17             'unit': {'required': False},
18         }
19
20     def validate(self, data):
21         """
22         Validate that SLIDER widgets have min/max/step values.
23         Sensor widgets do not require these fields.
24         """
25         widget_type = data.get('widget_type')
26
27         # SLIDER requires min/max/step
28         if widget_type == 'SLIDER':
29             if data.get('min_value') is None:
30                 raise serializers.ValidationError({'min_value': 'This field is required for SLIDER widgets.'})
31             if data.get('max_value') is None:
32                 raise serializers.ValidationError({'max_value': 'This field is required for SLIDER widgets.'})
33             if data.get('step') is None:
34                 raise serializers.ValidationError({'step': 'This field is required for SLIDER widgets.'})
35
36         return data
37
38     class DeviceCardTemplateSerializer(serializers.ModelSerializer):
39         controls = DeviceControlSerializer(many=True)
40
41     class Meta:
42         model = DeviceCardTemplate
43         fields = ['id', 'layout_config', 'controls']
44
45     class CustomDeviceTypeSerializer(serializers.ModelSerializer):
46         card_template = DeviceCardTemplateSerializer(required=False)
47         proposed_by_username = serializers.CharField(source='proposed_by.username', read_only=True)
48         wiring_diagram_base64 = serializers.CharField(required=False, allow_blank=True)
49         # Keep wiring_diagram_image as an alias that returns the same base64 value (backward compat for frontend)
50         wiring_diagram_image = serializers.CharField(source='wiring_diagram_base64', read_only=True)
51
52     class Meta:
53         model = CustomDeviceType
54         fields = ['id', 'name', 'definition', 'approved', 'rejection_reason', 'proposed_by',
   ↪ 'proposed_by_username', 'created_at', 'card_template', 'firmware_code', 'wiring_diagram_image',
   ↪ 'wiring_diagram_base64', 'wiring_diagram_text', 'documentation', 'documentation_images_base64']
55         read_only_fields = ['id', 'created_at', 'approved', 'rejection_reason', 'proposed_by',
   ↪ 'proposed_by_username', 'wiring_diagram_image']
56         extra_kwargs = {
57             'name': {
58                 'error_messages': {
59                     'unique': 'A device type with this name already exists.'
60                 }
61             },
62             'firmware_code': {'required': False},

```

```

63         'wiring_diagram_text': {'required': False},
64         'documentation': {'required': False},
65         'documentation_images_base64': {'required': False},
66     }
67
68     def validate_firmware_code(self, value):
69         if value and len(value) > 100000:
70             raise serializers.ValidationError("Firmware code must not exceed 100,000 characters.")
71         if value:
72             # WiFi credentials are always required (replaced per-device by the UI).
73             # server_ip is optional: firmware may auto-discover the broker via mDNS.
74             required_vars = ['wifi_ssid', 'wifi_password']
75             missing = [v for v in required_vars if v not in value]
76             if missing:
77                 raise serializers.ValidationError(
78                     "Firmware code must contain the variables: wifi_ssid, wifi_password. These are required for
79                     ↪ device connectivity."
80                 )
81             return value
82
83     def validate_documentation(self, value):
84         if value and len(value) > 50000:
85             raise serializers.ValidationError("Documentation must not exceed 50,000 characters.")
86         return value
87
88     def validate_wiring_diagram_text(self, value):
89         if value and len(value) > 50000:
90             raise serializers.ValidationError("Wiring diagram text must not exceed 50,000 characters.")
91         return value
92
93     def validate(self, data):
94         definition = data.get('definition', {})
95         card_template_data = data.get('card_template')
96
97         if card_template_data:
98             # Extract distinct IDs defined in the node builder structure
99             structure = definition.get('structure', [])
100             defined_ids = {node.get('id') for node in structure if 'id' in node}
101
102             # Check controls against defined IDs
103             controls_data = card_template_data.get('controls', [])
104             for control in controls_data:
105                 mapping = control.get('variable_mapping')
106                 if mapping and mapping not in defined_ids:
107                     raise serializers.ValidationError(
108                         f"Variable mapping '{mapping}' in controls does not match any ID in the device
109                         ↪ definition."
110                     )
111             return data
112
113     def create(self, validated_data):
114         card_template_data = validated_data.pop('card_template', None)
115
116         # Set proposed_by from request context
117         request = self.context.get('request')
118         if request and hasattr(request, 'user') and request.user.is_authenticated:
119             validated_data['proposed_by'] = request.user
120
121         # Create the Device Type
122         device_type = CustomDeviceType.objects.create(**validated_data)
123
124         # Create Template and Controls if provided
125         if card_template_data:
126             controls_data = card_template_data.pop('controls', [])
127             template = DeviceCardTemplate.objects.create(device_type=device_type, **card_template_data)
128
129             for control_data in controls_data:
130                 DeviceControl.objects.create(template=template, **control_data)
131
132         return device_type
133
134     def update(self, instance, validated_data):
135         card_template_data = validated_data.pop('card_template', None)
136
137         # Update definition and name
138         instance.name = validated_data.get('name', instance.name)
139         instance.definition = validated_data.get('definition', instance.definition)

```

```

139 instance.firmware_code = validated_data.get('firmware_code', instance.firmware_code)
140 instance.wiring_diagram_text = validated_data.get('wiring_diagram_text', instance.wiring_diagram_text)
141 instance.documentation = validated_data.get('documentation', instance.documentation)
142
143 # Handle wiring diagram base64
144 if 'wiring_diagram_base64' in validated_data:
145     instance.wiring_diagram_base64 = validated_data['wiring_diagram_base64']
146
147 # Handle documentation images base64
148 if 'documentation_images_base64' in validated_data:
149     instance.documentation_images_base64 = validated_data['documentation_images_base64']
150
151 instance.save()
152
153 if card_template_data:
154     controls_data = card_template_data.pop('controls', [])
155
156     # Update or Create Template
157     template, created = DeviceCardTemplate.objects.get_or_create(
158         device_type=instance,
159         defaults=card_template_data
160     )
161
162     if not created:
163         template.layout_config = card_template_data.get('layout_config', template.layout_config)
164         template.save()
165
166     # For simplicity, clear old controls and re-create (full sync)
167     template.controls.all().delete()
168     for control_data in controls_data:
169         DeviceControl.objects.create(template=template, **control_data)
170
171     return instance
172
173 class RoomSerializer(serializers.ModelSerializer):
174     class Meta:
175         model = Room
176         fields = ['id', 'name', 'icon']
177
178     def validate_name(self, value):
179         """Ensure no duplicate room names for the same user."""
180         request = self.context.get('request')
181         if request and request.user:
182             qs = Room.objects.filter(name__iexact=value, user=request.user)
183             if self.instance:
184                 qs = qs.exclude(pk=self.instance.pk)
185             if qs.exists():
186                 raise serializers.ValidationError("A room with this name already exists.")
187         return value
188
189 class DeviceSerializer(serializers.ModelSerializer):
190     room_name = serializers.CharField(source='room.name', read_only=True)
191     room_id = serializers.PrimaryKeyRelatedField(read_only=True, source='room')
192     device_type_name = serializers.CharField(source='device_type.name', read_only=True)
193
194     class Meta:
195         model = Device
196         fields = ['id', 'name', 'ip_address', 'status', 'icon', 'device_type', 'device_type_name', 'room',
197             ↪ 'room_name', 'room_id', 'current_state']
198
199     def validate_ip_address(self, value):
200         """
201         Check that the IP address is unique across all devices.
202         """
203         qs = Device.objects.filter(ip_address=value)
204         # Exclude current device if updating
205         if self.instance:
206             qs = qs.exclude(pk=self.instance.pk)
207
208         if qs.exists():
209             raise serializers.ValidationError(f"A device with IP address {value} already exists.")
210         return value
211
212     def validate_device_type(self, value):
213         if not value.approved:
214             raise serializers.ValidationError("Device type must be approved.")
215         return value

```

```

216 class ProfileSerializer(serializers.ModelSerializer):
217     avatar = serializers.SerializerMethodField()
218
219     class Meta:
220         model = Profile
221         fields = ('avatar', 'role', 'accent_color')
222
223     def get_avatar(self, obj):
224         if obj.avatar:
225             request = self.context.get('request')
226             if request:
227                 return request.build_absolute_uri(obj.avatar.url)
228             return obj.avatar.url
229         return None
230
231 class UserSerializer(serializers.ModelSerializer):
232     profile = ProfileSerializer(read_only=True)
233
234     class Meta:
235         model = User
236         fields = ['id', 'username', 'email', 'first_name', 'last_name', 'profile']
237
238 class RegisterSerializer(serializers.ModelSerializer):
239     password = serializers.CharField(write_only=True, validators=[validate_password])
240     role = serializers.ChoiceField(choices=Profile.ROLE_CHOICES, default=Profile.ROLE_USER, required=False)
241
242     email = serializers.EmailField(
243         required=True,
244         validators=[UniqueValidator(queryset=User.objects.all(), message="This email is already registered.")])
245
246     class Meta:
247         model = User
248         fields = ['username', 'email', 'password', 'first_name', 'last_name', 'role']
249         extra_kwargs = {
250             'username': {
251                 'error_messages': {
252                     'unique': 'This username is already taken.'
253                 }
254             }
255         }
256
257     def create(self, validated_data):
258         # Check if this is the first user
259         is_first_user = not User.objects.exists()
260
261         # Use create_user to hash the password automatically
262         role = validated_data.pop('role', Profile.ROLE_USER)
263
264         # If first user, force role to OWNER and give staff/superuser status
265         if is_first_user:
266             role = Profile.ROLE_OWNER
267
268         user = User.objects.create_user(
269             username=validated_data['username'],
270             email=validated_data.get('email', ''),
271             password=validated_data['password'],
272             first_name=validated_data.get('first_name', ''),
273             last_name=validated_data.get('last_name', ''),
274         )
275
276         if is_first_user:
277             user.is_staff = True
278             user.is_superuser = True
279             user.save()
280
281         # Ensure profile exists and set role
282         profile, _ = Profile.objects.get_or_create(user=user)
283         profile.role = role
284         profile.save()
285         return user
286
287 class NotificationSerializer(serializers.ModelSerializer):
288     """Serializer for Notification model."""
289
290     # Human-readable type and priority
291     notification_type_display = serializers.CharField(source='get_notification_type_display', read_only=True)
292     priority_display = serializers.CharField(source='get_priority_display', read_only=True)
293

```

```

294
295 # Time since creation (for display)
296 time_ago = serializers.SerializerMethodField()
297
298 class Meta:
299     model = Notification
300     fields = [
301         'id',
302         'notification_type',
303         'notification_type_display',
304         'title',
305         'message',
306         'priority',
307         'priority_display',
308         'is_read',
309         'reference_data',
310         'action_url',
311         'created_at',
312         'read_at',
313         'time_ago',
314     ]
315     read_only_fields = ['id', 'created_at', 'read_at', 'time_ago']
316
317 def get_time_ago(self, obj):
318     """Calculate human-readable time since notification was created."""
319     from django.utils import timezone
320     from datetime import timedelta
321
322     now = timezone.now()
323     diff = now - obj.created_at
324
325     if diff < timedelta(minutes=1):
326         return "Just now"
327     elif diff < timedelta(hours=1):
328         minutes = int(diff.total_seconds() / 60)
329         return f"{minutes}m ago"
330     elif diff < timedelta(days=1):
331         hours = int(diff.total_seconds() / 3600)
332         return f"{hours}h ago"
333     elif diff < timedelta(days=7):
334         days = diff.days
335         return f"{days}d ago"
336     else:
337         return obj.created_at.strftime("%b %d, %Y")
338
339 class NotificationCreateSerializer(serializers.ModelSerializer):
340     """Serializer for creating notifications (admin use)."""
341
342     # Allow specifying user by ID or username
343     user_id = serializers.IntegerField(write_only=True, required=False)
344     username = serializers.CharField(write_only=True, required=False)
345
346     class Meta:
347         model = Notification
348         fields = [
349             'user_id',
350             'username',
351             'notification_type',
352             'title',
353             'message',
354             'priority',
355             'reference_data',
356             'action_url',
357         ]
358
359     def validate(self, data):
360         user_id = data.pop('user_id', None)
361         username = data.pop('username', None)
362
363         if not user_id and not username:
364             raise serializers.ValidationError("Either 'user_id' or 'username' must be provided.")
365
366         try:
367             if user_id:
368                 data['user'] = User.objects.get(pk=user_id)
369             else:
370                 data['user'] = User.objects.get(username=username)
371         except User.DoesNotExist:

```

```

372         raise serializers.ValidationError("User not found.")
373
374     return data
375
376 class DashboardLayoutSerializer(serializers.Serializer):
377     """
378     Serializer for dashboard layout validation.
379     Validates the layout JSON structure: version, items (devices + folders).
380     Also accepts an optional device_order preference.
381     """
382     layout = serializers.JSONField()
383     device_order = serializers.ChoiceField(
384         choices=DashboardLayout.ORDER_CHOICES,
385         required=False,
386         default=None,
387         help_text="Device grouping/sorting preference.",
388     )
389
390     def _validate_layout_structure(self, layout):
391         """Validate the top-level layout structure."""
392         if not isinstance(layout, dict):
393             raise serializers.ValidationError({"layout": "Layout must be a JSON object."})
394
395         # version must be 1
396         version = layout.get('version')
397         if version != 1:
398             raise serializers.ValidationError({"layout": "layout.version must equal 1."})
399
400         # items must be a non-empty array with <= 100 entries
401         items = layout.get('items')
402         if not isinstance(items, list) or len(items) == 0:
403             raise serializers.ValidationError({"layout": "layout.items must be a non-empty array."})
404         if len(items) > 100:
405             raise serializers.ValidationError({"layout": "layout.items must have at most 100 entries."})
406
407         return items
408
409     def _collect_device_ids_and_validate_items(self, items):
410         """Validate each item and collect all device IDs."""
411         all_device_ids = []
412         folder_ids = set()
413
414         for i, item in enumerate(items):
415             if not isinstance(item, dict):
416                 raise serializers.ValidationError(
417                     {"layout": f"items[{i}] must be a JSON object."}
418                 )
419
420             item_type = item.get('type')
421             if item_type not in ('device', 'folder'):
422                 raise serializers.ValidationError(
423                     {"layout": f"items[{i}].type must be 'device' or 'folder'."}
424                 )
425
426             if item_type == 'device':
427                 device_id = item.get('deviceId')
428                 if not isinstance(device_id, int):
429                     raise serializers.ValidationError(
430                         {"layout": f"items[{i}].deviceId must be an integer."}
431                     )
432                 all_device_ids.append(device_id)
433
434             elif item_type == 'folder':
435                 # Validate folderId
436                 folder_id = item.get('folderId')
437                 if not isinstance(folder_id, str) or not folder_id.strip():
438                     raise serializers.ValidationError(
439                         {"layout": f"items[{i}].folderId must be a non-empty string."}
440                     )
441                 if folder_id in folder_ids:
442                     raise serializers.ValidationError(
443                         {"layout": f"Duplicate folderId '{folder_id}'."}
444                     )
445                 folder_ids.add(folder_id)
446
447             # Validate name
448             name = item.get('name')
449             if not isinstance(name, str) or not name.strip():

```

```

450         raise serializers.ValidationError(
451             {"layout": f"items[{i}].name must be a non-empty string."}
452         )
453     if len(name) > 50:
454         raise serializers.ValidationError(
455             {"layout": f"items[{i}].name must be at most 50 characters."}
456         )
457
458     # Validate deviceIds
459     device_ids = item.get('deviceIds')
460     if not isinstance(device_ids, list) or not (2 <= len(device_ids) <= 4):
461         raise serializers.ValidationError(
462             {"layout": f"items[{i}].deviceIds must be an array of 2-4 integers."}
463         )
464     for j, did in enumerate(device_ids):
465         if not isinstance(did, int):
466             raise serializers.ValidationError(
467                 {"layout": f"items[{i}].deviceIds[{j}] must be an integer."}
468             )
469     all_device_ids.extend(device_ids)
470
471     return all_device_ids
472
473 def _check_duplicate_device_ids(self, all_device_ids):
474     """Ensure no device ID appears more than once."""
475     seen = set()
476     for did in all_device_ids:
477         if did in seen:
478             raise serializers.ValidationError(
479                 {"layout": f"Duplicate deviceId {did}. Each device may appear at most once."}
480             )
481         seen.add(did)
482
483 def _check_devices_exist(self, all_device_ids, user, skip_ownership=False):
484     """Ensure all referenced device IDs exist and are accessible to the user."""
485     if not all_device_ids:
486         return
487     qs = Device.objects.filter(id__in=all_device_ids)
488     if not skip_ownership:
489         qs = qs.filter(user=user)
490     existing_ids = set(qs.values_list('id', flat=True))
491     missing = set(all_device_ids) - existing_ids
492     if missing:
493         raise serializers.ValidationError(
494             {"layout": f"Device IDs not found or not accessible: {sorted(missing)}"}
495         )
496
497 def validate_layout(self, value):
498     """Full layout validation pipeline."""
499     items = self._validate_layout_structure(value)
500     all_device_ids = self._collect_device_ids_and_validate_items(items)
501     self._check_duplicate_device_ids(all_device_ids)
502
503     # Device existence check requires user from context
504     user = self.context.get('user')
505     skip_ownership = self.context.get('skip_ownership', False)
506     if user:
507         self._check_devices_exist(all_device_ids, user, skip_ownership=skip_ownership)
508
509     return value
510
511 class SolarSystemSerializer(serializers.ModelSerializer):
512     """Serializer for registered solar/energy data sources."""
513
514     class Meta:
515         model = SolarSystem
516         fields = [
517             'id',
518             'name',
519             'base_url',
520             'provider',
521             'enabled',
522             'api_version',
523             'capabilities',
524             'last_seen',
525             'created_at',
526             'updated_at',
527         ]

```

```

528         read_only_fields = ['id', 'api_version', 'capabilities', 'last_seen', 'created_at', 'updated_at']
529
530     def validate_name(self, value):
531         if not value.strip():
532             raise serializers.ValidationError("Name cannot be empty.")
533         return value.strip()
534
535     def validate_base_url(self, value):
536         value = value.strip().rstrip('/')
537         if not value:
538             raise serializers.ValidationError("Base URL cannot be empty.")
539         return value

```

A.3 Permisuni RBAC (api/permissions.py)

```

1  from rest_framework import permissions
2  from .models import Profile
3
4  class IsOwner(permissions.BasePermission):
5      """
6      Allows access only to users with the 'owner' role.
7      """
8      def has_permission(self, request, view):
9          return (
10              request.user and
11              request.user.is_authenticated and
12              hasattr(request.user, 'profile') and
13              request.user.profile.role == Profile.ROLE_OWNER
14          )
15
16  class IsAdmin(permissions.BasePermission):
17      """
18      Allows access only to users with 'owner' or 'admin' roles.
19      """
20      def has_permission(self, request, view):
21          if not (request.user and request.user.is_authenticated and hasattr(request.user, 'profile')):
22              return False
23          return request.user.profile.role in [Profile.ROLE_OWNER, Profile.ROLE_ADMIN]
24
25  class IsOwnerOrReadOnly(permissions.BasePermission):
26      """
27      Custom permission to only allow owners of an object to edit it.
28      """
29      def has_object_permission(self, request, view, obj):
30          # Read permissions are allowed to any request,
31          # so we'll always allow GET, HEAD or OPTIONS requests.
32          if request.method in permissions.SAFE_METHODS:
33              return True
34
35          # Write permissions are only allowed to the owner of the snippet.
36          # Assumes the model instance has an `owner` or `user` attribute.
37          return getattr(obj, 'user', None) == request.user or getattr(obj, 'owner', None) == request.user

```

A.4 Daemonul MQTT (api/management/commands/mqtt_listener.py)

```

1  import json
2  import logging
3  import time
4  from django.core.management.base import BaseCommand
5  import paho.mqtt.client as mqtt
6  from api.models import Device
7  from django.utils import timezone
8  from datetime import timedelta
9
10 logger = logging.getLogger(__name__)
11
12 class Command(BaseCommand):
13     help = 'Starts the MQTT Listener to process device updates and monitor offline status'
14
15     def handle(self, *args, **options):
16         client = mqtt.Client()
17         client.on_connect = self.on_connect
18         client.on_message = self.on_message

```

```

19
20
21     try:
22         self.stdout.write('Connecting to MQTT Broker at localhost:1883...')
23         client.connect("localhost", 1883, 60)
24         client.loop_start() # Run MQTT loop in background thread
25
26         self.stdout.write(self.style.SUCCESS('MQTT Listener running. Monitoring device heartbeats...'))
27
28         while True:
29             self.check_offline_devices()
30             time.sleep(10) # Check every 10 seconds
31
32     except KeyboardInterrupt:
33         client.loop_stop()
34         self.stdout.write(self.style.SUCCESS('Stopped MQTT Listener'))
35     except Exception as e:
36         self.stdout.write(self.style.ERROR(f'Error: {e}'))
37
38 def check_offline_devices(self):
39     """
40     Check for devices that haven't updated in the last 30 seconds and mark them offline.
41     """
42     threshold = timezone.now() - timedelta(seconds=30) # 30 seconds timeout (firmware sends every 5s)
43
44     # Find devices that are marked online but haven't updated recently
45     offline_devices = Device.objects.filter(
46         status=Device.STATUS_ONLINE,
47         updated_at__lt=threshold
48     )
49
50     for device in offline_devices:
51         self.stdout.write(self.style.WARNING(f"Device {device.name} timed out. Marking OFFLINE."))
52         device.status = Device.STATUS_OFFLINE
53         device.save(update_fields=['status'])
54
55 def on_connect(self, client, userdata, flags, rc):
56     if rc == 0:
57         self.stdout.write(self.style.SUCCESS('Connected to MQTT Broker! Subscribing to
58         ↳ homeforge/devices/+state'))
59         client.subscribe("homeforge/devices/+state")
60     else:
61         self.stdout.write(self.style.ERROR(f'Connection failed with code {rc}'))
62
63 def on_message(self, client, userdata, msg):
64     try:
65         topic = msg.topic
66         payload_str = msg.payload.decode()
67         self.stdout.write(f'Received message on {topic}: {payload_str}')
68
69         # Topic format: homeforge/devices/{MAC}/state
70         parts = topic.split('/')
71         if len(parts) < 4:
72             return
73
74         device_mac_from_topic = parts[2]
75
76     try:
77         data = json.loads(payload_str)
78     except json.JSONDecodeError:
79         self.stdout.write(self.style.WARNING(f'Invalid JSON from device {device_mac_from_topic}'))
80         return
81
82     # Intelligent Device Matching Logic
83     # 1. Try to find device by MAC address (most reliable)
84     device = Device.objects.filter(mac_address=device_mac_from_topic).first()
85
86     # 2. If not found by MAC, try to find by IP address reported in payload (auto-discovery/binding)
87     if not device and "ip" in data:
88         reported_ip = data["ip"]
89         device = Device.objects.filter(ip_address=reported_ip).first()
90         if device:
91             self.stdout.write(self.style.SUCCESS(f'Auto-binding MAC {device_mac_from_topic} to Device
92             ↳ {device.name} (IP: {reported_ip})'))
93             device.mac_address = device_mac_from_topic
94             device.save(update_fields=['mac_address'])
95
96     if not device:
97         self.stdout.write(self.style.WARNING(f'Device with MAC {device_mac_from_topic} or IP
98         ↳ {data.get('ip')} not found in DB.'))

```

```

95         return
96
97     # Update current_state with new data (merge)
98     current_state = device.current_state or {}
99
100    # Filter out system keys from state blob if desired, or keep them for display
101    # We generally want to merge sensor data: temp, humidity, etc.
102
103    # Auto-Mapping Logic
104
105    remapped_data = {}
106    for key, value in data.items():
107        # Try to map "standard" keys to "dynamic" keys first
108        mapped_key = self.map_standard_key(device, key)
109
110        if mapped_key:
111            # If we found a mapping (e.g. relay_1 -> switch-123), use that
112            remapped_data[mapped_key] = value
113            self.stdout.write(f"Mapped incoming '{key}' -> '{mapped_key}'")
114
115            # Also REMOVE the old standard key if it exists in current_state to avoid duplicates
116            if key in current_state and key != mapped_key:
117                try:
118                    del current_state[key]
119                except KeyError:
120                    pass
121
122            # If no mapping found, check if it's already a known key
123            elif key in current_state:
124                remapped_data[key] = value
125
126            # Otherwise keep as new unmapped data
127            else:
128                remapped_data[key] = value
129
130    current_state.update(remapped_data)
131    device.current_state = current_state
132
133    # Mark as Online
134    if device.status != 'online':
135        device.status = 'online'
136        self.stdout.write(self.style.SUCCESS(f"Device {device.name} is now ONLINE"))
137
138    device.save()
139    self.stdout.write(self.style.SUCCESS(f"Updated Device {device.name} state"))
140
141    except Exception as e:
142        self.stdout.write(self.style.ERROR(f"Error processing message: {e}"))
143
144    def map_standard_key(self, device, key):
145        """
146        Map standard firmware keys (temperature, humidity, relay_1) to dynamic widget IDs.
147        """
148        # Load the device's type definition
149        try:
150            # We need to access the related CardTemplate -> Controls
151
152            # Get the CustomDeviceType
153            dev_type = device.device_type
154            if not hasattr(dev_type, 'card_template'):
155                return None
156
157            template = dev_type.card_template
158            controls = template.controls.all() # Fetch all controls
159
160        except Exception:
161            return None
162
163        # MAPPING RULES: firmware_key -> Widget Type
164        target_types = []
165
166        if key == "temperature":
167            target_types = ['TEMPERATURE', 'GAUGE']
168        elif key == "humidity":
169            target_types = ['HUMIDITY', 'GAUGE']
170        elif key == "pressure":
171            target_types = ['PRESSURE', 'GAUGE']
172        elif key == "relay_1" or key.startswith("switch-"):

```

```

173         target_types = ['TOGGLE', 'BUTTON']
174
175     if not target_types:
176         return None
177
178     # Find the first control that matches one of the target types
179     for control in controls:
180         if control.widget_type in target_types:
181             return control.variable_mapping
182
183     return None

```

A.5 Clientul MQTT de publicare (api/mqtt_client.py)

```

1  import paho.mqtt.client as mqtt
2  import json
3  import logging
4  from django.conf import settings
5
6  logger = logging.getLogger(__name__)
7
8  MQTT_BROKER = "localhost"
9  MQTT_PORT = 1883
10 MQTT_TOPIC_PREFIX = "homeforge/devices"
11
12 class MQTTClient:
13     _instance = None
14
15     def __new__(cls):
16         if cls._instance is None:
17             cls._instance = super(MQTTClient, cls).__new__(cls)
18             cls._instance.client = mqtt.Client()
19             cls._instance.connected = False
20             return cls._instance
21
22     def connect(self):
23         if not self.connected:
24             try:
25                 self.client.connect(MQTT_BROKER, MQTT_PORT, 60)
26                 self.client.loop_start()
27                 self.connected = True
28                 logger.info("Connected to MQTT Broker")
29             except Exception as e:
30                 logger.error(f"Failed to connect to MQTT Broker: {e}")
31
32     def publish(self, device_id, command_data):
33         """
34         Publish a command to a device.
35         Topic: homeforge/devices/{device_id}/command
36         """
37         if not self.connected:
38             self.connect()
39
40         topic = f"{MQTT_TOPIC_PREFIX}/{device_id}/command"
41         payload = json.dumps(command_data)
42         self.client.publish(topic, payload)
43         logger.info(f"Published to {topic}: {payload}")
44
45 mqtt_client = MQTTClient()

```

A.6 Adaptorul Fronius pentru sisteme solare (api/solar/providers/fronius.py)

```

1  """
2  Fronius Solar API V1 adapter.
3
4  Maps the Fronius realtime JSON (notably ``GetPowerFlowRealtimeData.fcgi``) onto
5  the normalized schema defined in :mod:`api.solar.providers.base`.
6
7  Fronius responses share a common envelope::
8
9  {

```

```

10         "Head": {"Status": {"Code": 0, "Reason": "", "UserMessage": ""},
11                 "Timestamp": "..."},
12         "Body": {"Data": {...}}
13     }
14
15     ``Status.Code == 0`` means success; any other value is an error. Datapoints that
16     are unavailable are returned as ``null`` (e.g. energy counters on GEN24).
17     """
18     from __future__ import annotations
19
20     from typing import Any, Dict, List, Optional
21
22     from ..client import SolarApiError, fetch_json
23     from .base import BaseSolarProvider
24
25     # Fronius default API base path (confirmed via GetAPIVersion.cgi.BaseURL).
26     _API_BASE = 'solar_api/v1/'
27     _POWER_FLOW = 'GetPowerFlowRealtimeData.fcgi'
28     _API_VERSION = 'solar_api/GetAPIVersion.cgi'
29
30     def _to_float(value: Any) -> Optional[float]:
31         """Coerce a vendor number to float, preserving null for missing data."""
32         if value is None:
33             return None
34         try:
35             return float(value)
36         except (TypeError, ValueError):
37             return None
38
39     class FroniusProvider(BaseSolarProvider):
40         key = 'fronius'
41
42         # envelope helpers
43
44         @staticmethod
45         def _unwrap(payload: Dict[str, Any]) -> Dict[str, Any]:
46             """Validate the Fronius Head/Body envelope and return Body.Data."""
47             head = payload.get('Head', {}) or {}
48             status = head.get('Status', {}) or {}
49             code = status.get('Code')
50             if code not in (0, None):
51                 message = status.get('UserMessage') or status.get('Reason') or 'Fronius API error'
52                 raise SolarApiError(message, vendor_code=code)
53             return (payload.get('Body', {}) or {}).get('Data', {}) or {}
54
55         # public API
56
57         def discover(self) -> Dict[str, Any]:
58             payload = fetch_json(self.base_url, _API_VERSION)
59             api_version = str(payload.get('APIVersion', '')) if isinstance(payload, dict) else ''
60
61             # Probe the power-flow feed to learn capabilities (battery/meter).
62             overview = self.get_overview()
63             return {
64                 'api_version': api_version,
65                 'capabilities': overview.get('capabilities', {}),
66             }
67
68         def get_overview(self) -> Dict[str, Any]:
69             payload = fetch_json(self.base_url, _API_BASE + _POWER_FLOW)
70
71             head = payload.get('Head', {}) or {}
72             status = (head.get('Status', {}) or {})
73             timestamp = head.get('Timestamp')
74
75             data = self._unwrap(payload)
76             site = data.get('Site', {}) or {}
77             inverters = data.get('Inverters', {}) or {}
78
79             p_pv = _to_float(site.get('P_PV')) or 0.0
80             p_grid = _to_float(site.get('P_Grid'))
81             p_load = _to_float(site.get('P_Load'))
82             p_akku = _to_float(site.get('P_Akku'))
83
84             # Battery presence: P_Akku present or any inverter reports SOC.
85             has_battery = p_akku is not None or any(
86                 (inv or {}).get('SOC') is not None for inv in inverters.values()
87             )

```

```

88     has_meter = p_grid is not None
89
90     soc = None
91     battery_mode = None
92     for inv in inverters.values():
93         inv = inv or {}
94         if inv.get('SOC') is not None:
95             soc = _to_float(inv.get('SOC'))
96             battery_mode = inv.get('Battery_Mode')
97             break
98
99     mode = self._normalize_mode(site.get('Mode'), has_battery, has_meter)
100
101     # Fronius P_Load is reported negative for consumption; normalize to
102     # positive-consumption so the frontend invariant is simple.
103     load_w = abs(p_load) if p_load is not None else None
104
105     # Fronius P_Akku uses the opposite sign convention of our schema
106     # (Fronius: - = charge, + = discharge; see Solar API manual, GetPowerFlowRealtimeData).
107     # Flip it so batteryW follows the normalized contract (+ = charging, - = discharging).
108     battery_w = -p_akku if p_akku is not None else None
109
110     return {
111         'provider': self.key,
112         'online': True,
113         'mode': mode,
114         'timestamp': timestamp,
115         'power': {
116             'solarW': p_pv,
117             'gridW': p_grid,
118             'loadW': load_w,
119             'batteryW': battery_w,
120         },
121         'battery': {
122             'present': has_battery,
123             'socPct': soc,
124             'mode': battery_mode,
125             'standby': site.get('BatteryStandby'),
126         },
127         'energy': {
128             'todayWh': _to_float(site.get('E_Day')),
129             'yearWh': _to_float(site.get('E_Year')),
130             'totalWh': _to_float(site.get('E_Total')),
131         },
132         'ratios': {
133             'selfConsumptionPct': _to_float(site.get('rel_SelfConsumption')),
134             'autonomyPct': _to_float(site.get('rel_Autonomy')),
135         },
136         'capabilities': {
137             'battery': has_battery,
138             'meter': has_meter,
139         },
140         'status': {
141             'code': status.get('Code', 0),
142             'message': status.get('Reason') or 'OK',
143         },
144     }
145
146     def get_inverters(self) -> List[Dict[str, Any]]:
147         payload = fetch_json(
148             self.base_url,
149             _API_BASE + 'GetInverterRealtimeData.cgi',
150             params={'Scope': 'System'},
151         )
152         data = self._unwrap(payload)
153         # Body.Data is keyed by channel (PAC, DAY_ENERGY, ...), each a map of
154         # DeviceId -> {Value, Unit}. Pivot into a per-inverter list.
155         result: Dict[str, Dict[str, Any]] = {}
156         for channel, block in data.items():
157             values = (block or {}).get('Values', {}) or {}
158             unit = (block or {}).get('Unit')
159             for device_id, value in values.items():
160                 entry = result.setdefault(str(device_id), {'deviceId': str(device_id)})
161                 entry[channel] = {'value': _to_float(value), 'unit': unit}
162         return list(result.values())
163
164     # helpers
165

```

```
166 | def _normalize_mode(self, raw_mode: Any, has_battery: bool, has_meter: bool) -> str:
167 |     if not has_meter:
168 |         return self.MODE_PRODUCE_ONLY
169 |     if has_battery:
170 |         return self.MODE_BIDIRECTIONAL
171 |     if raw_mode:
172 |         return self.MODE_METER
173 |     return self.MODE_UNKNOWN
```


Anexa B

Frontend

B.1 Comunicarea cu API-ul și gestiunea JWT (lib/apiClient.js)

```
1 // apiClient.js - small helper for auth + profile usage
2
3 export function getBackendUrl() {
4   if (typeof window !== 'undefined') {
5     return `${window.location.protocol}//${window.location.hostname}:8000`;
6   }
7   return 'http://localhost:8000';
8 }
9
10 function getApiBase() {
11   return `${getBackendUrl()}/api`;
12 }
13
14 // Helper to handle API errors globally matching backend format
15 async function handleApiError(res, defaultMsg) {
16   let errorData;
17   try {
18     errorData = await res.json();
19   } catch (e) {
20     throw new Error(defaultMsg || `Request failed with status ${res.status}`);
21   }
22
23   if (errorData) {
24     // 1. Check for specific "detail" (Permission errors, etc.)
25     if (typeof errorData.detail === 'string') {
26       throw new Error(errorData.detail);
27     }
28
29     // 2. Check for "non_field_errors"
30     if (Array.isArray(errorData.non_field_errors)) {
31       throw new Error(errorData.non_field_errors.join(' '));
32     }
33
34     // 3. Field errors
35     const parts = [];
36     for (const [key, value] of Object.entries(errorData)) {
37       // Just extract the messages, ignore the field names for a cleaner look
38       const messages = Array.isArray(value) ? value : [value];
39       parts.push(...messages);
40     }
41
42     if (parts.length > 0) {
43       // Join with newline
44       throw new Error(parts.join('\n'));
45     }
46   }
47
48   throw new Error(defaultMsg || `Request failed with status ${res.status}`);
49 }
50
51 export function getAvatarUrl(path) {
52   if (!path) return undefined;
53   const backendUrl = getBackendUrl();
54   const url = path.startsWith('http') ? path : `${backendUrl}${path}`;
55   return `${url}${url.includes('?') ? '&' : '?'}t=${Date.now()}`;
56 }
57
58 export function getMediaUrl(path) {
59   if (!path) return undefined;
60   if (path.startsWith('http') || path.startsWith('data:') || path.startsWith('blob:')) return path;
61   // Detect raw base64 strings (not a URL path - typically very long, no slashes at start)
62   if (!path.startsWith('/') && path.length > 200) {
```

```

63     return `data:image/png;base64,${path}`;
64 }
65 return `${getBackendUrl()}${path}`;
66 }
67
68 export async function registerUser({ username, email, password, first_name, last_name, role }) {
69     const res = await fetch(`${getApiBase()}/register/`, {
70         method: 'POST',
71         headers: { 'Content-Type': 'application/json' },
72         body: JSON.stringify({ username, email, password, first_name, last_name, role }),
73     });
74     if (!res.ok) await handleApiError(res, 'Registration failed');
75     return res.json();
76 }
77
78 export async function login({ username, password }) {
79     console.log('Login attempt:', { username, password });
80     const res = await fetch(`${getApiBase()}/login/`, {
81         method: 'POST',
82         headers: { 'Content-Type': 'application/json' },
83         body: JSON.stringify({ username: username.trim(), password }),
84     });
85     if (!res.ok) await handleApiError(res, 'Login failed');
86
87     const data = await res.json();
88     localStorage.setItem('access', data.access);
89     localStorage.setItem('refresh', data.refresh);
90     return data;
91 }
92
93 export function logout() {
94     localStorage.removeItem('access');
95     localStorage.removeItem('refresh');
96     if (typeof window !== 'undefined') {
97         window.location.href = '/login';
98     }
99 }
100
101 export function getAuthHeaders() {
102     const token = localStorage.getItem('access');
103     return token ? { Authorization: `Bearer ${token}` } : {};
104 }
105
106 async function fetchWithAuth(url, options = {}) {
107     const headers = { ...options.headers, ...getAuthHeaders() };
108
109     let res = await fetch(url, { ...options, headers });
110
111     if (res.status === 401) {
112         try {
113             const newToken = await refreshAccessToken();
114             const newHeaders = { ...options.headers, Authorization: `Bearer ${newToken}` };
115             res = await fetch(url, { ...options, headers: newHeaders });
116         } catch (error) {
117             throw error;
118         }
119     }
120     return res;
121 }
122
123 export async function fetchProfile() {
124     const res = await fetchWithAuth(`${getApiBase()}/me/`);
125     if (!res.ok) await handleApiError(res, 'Failed to fetch profile');
126     return res.json();
127 }
128
129 export async function updateProfile({ first_name, last_name, username, email, password, role, accent_color,
130     ↪ avatarFile }) {
131     const form = new FormData();
132     if (first_name !== undefined) form.append('first_name', first_name);
133     if (last_name !== undefined) form.append('last_name', last_name);
134     if (username !== undefined) form.append('username', username);
135     if (email !== undefined) form.append('email', email);
136     if (password !== undefined && password.trim() !== '') form.append('password', password);
137     if (role !== undefined) form.append('role', role);
138     if (accent_color !== undefined) form.append('accent_color', accent_color);
139     if (avatarFile) {
140         form.append('avatar', avatarFile);

```

```

140     }
141
142     const res = await fetchWithAuth(`${getApiBase()}/me/`, {
143       method: 'PUT',
144       body: form,
145     });
146     if (!res.ok) await handleApiError(res, 'Update failed');
147     return res.json();
148   }
149
150   export async function refreshAccessToken() {
151     const refresh = localStorage.getItem('refresh');
152     if (!refresh) {
153       const e = new Error('No refresh token available');
154       e.status = 401;
155       throw e;
156     }
157
158     const res = await fetch(`${getApiBase()}/token/refresh/`, {
159       method: 'POST',
160       headers: { 'Content-Type': 'application/json' },
161       body: JSON.stringify({ refresh }),
162     });
163
164     if (!res.ok) {
165       localStorage.removeItem('access');
166       localStorage.removeItem('refresh');
167       const e = new Error('Refresh failed');
168       e.status = 401;
169       throw e;
170     }
171
172     const data = await res.json();
173     localStorage.setItem('access', data.access);
174     return data.access;
175   }
176
177   // Rooms
178
179   export async function fetchRooms() {
180     const res = await fetchWithAuth(`${getApiBase()}/rooms/`);
181     if (!res.ok) await handleApiError(res, 'Failed to fetch rooms');
182     return res.json();
183   }
184
185   export async function createRoom(data) {
186     const res = await fetchWithAuth(`${getApiBase()}/rooms/`, {
187       method: 'POST',
188       headers: { 'Content-Type': 'application/json' },
189       body: JSON.stringify(data),
190     });
191     if (!res.ok) await handleApiError(res, 'Failed to create room');
192     return res.json();
193   }
194
195   export async function updateRoom(id, data) {
196     const res = await fetchWithAuth(`${getApiBase()}/rooms/${id}/`, {
197       method: 'PUT',
198       headers: { 'Content-Type': 'application/json' },
199       body: JSON.stringify(data),
200     });
201     if (!res.ok) await handleApiError(res, 'Failed to update room');
202     return res.json();
203   }
204
205   export async function deleteRoom(id) {
206     const res = await fetchWithAuth(`${getApiBase()}/rooms/${id}/`, {
207       method: 'DELETE',
208     });
209     if (!res.ok) await handleApiError(res, 'Failed to delete room');
210     return true;
211   }
212
213   // Solar
214
215   export async function fetchSolarSystems() {
216     const res = await fetchWithAuth(`${getApiBase()}/solar/systems/`);
217     if (!res.ok) await handleApiError(res, 'Failed to fetch solar systems');

```

```

218     return res.json();
219 }
220
221 export async function createSolarSystem(data) {
222     const res = await fetchWithAuth(`${getApiBase()}/solar/systems/`, {
223         method: 'POST',
224         headers: { 'Content-Type': 'application/json' },
225         body: JSON.stringify(data),
226     });
227     if (!res.ok) await handleApiError(res, 'Failed to create solar system');
228     return res.json();
229 }
230
231 export async function updateSolarSystem(id, data) {
232     const res = await fetchWithAuth(`${getApiBase()}/solar/systems/${id}/`, {
233         method: 'PUT',
234         headers: { 'Content-Type': 'application/json' },
235         body: JSON.stringify(data),
236     });
237     if (!res.ok) await handleApiError(res, 'Failed to update solar system');
238     return res.json();
239 }
240
241 export async function deleteSolarSystem(id) {
242     const res = await fetchWithAuth(`${getApiBase()}/solar/systems/${id}/`, {
243         method: 'DELETE',
244     });
245     if (!res.ok) await handleApiError(res, 'Failed to delete solar system');
246     return true;
247 }
248
249 export async function fetchSolarOverview(id) {
250     const res = await fetchWithAuth(`${getApiBase()}/solar/systems/${id}/overview/`);
251     if (!res.ok) await handleApiError(res, 'Failed to fetch solar overview');
252     return res.json();
253 }
254
255 // Users (Admin)
256
257 export async function fetchUsers() {
258     const res = await fetchWithAuth(`${getApiBase()}/users/`);
259     if (!res.ok) await handleApiError(res, 'Failed to fetch users');
260     return res.json();
261 }
262
263 export async function updateUserAdmin(id, data) {
264     const res = await fetchWithAuth(`${getApiBase()}/users/${id}/`, {
265         method: 'PUT',
266         headers: { 'Content-Type': 'application/json' },
267         body: JSON.stringify(data),
268     });
269     if (!res.ok) await handleApiError(res, 'Failed to update user');
270     return res.json();
271 }
272
273 export async function deleteUser(id) {
274     const res = await fetchWithAuth(`${getApiBase()}/users/${id}/`, {
275         method: 'DELETE',
276     });
277     if (!res.ok) await handleApiError(res, 'Failed to delete user');
278     return true;
279 }
280
281 // Device Types
282
283 export async function fetchDeviceTypes() {
284     const res = await fetchWithAuth(`${getApiBase()}/device-types/`);
285     if (!res.ok) await handleApiError(res, 'Failed to fetch device types');
286     return res.json();
287 }
288
289 export async function createDeviceType(data) {
290     const res = await fetchWithAuth(`${getApiBase()}/device-types/`, {
291         method: 'POST',
292         headers: { 'Content-Type': 'application/json' },
293         body: JSON.stringify(data),
294     });
295     if (!res.ok) await handleApiError(res, 'Failed to create device type');

```

```

296     return res.json();
297   }
298
299   export async function fetchPendingDeviceTypes() {
300     const res = await fetchWithAuth(`${getApiBase()}/admin/device-types/pending/`);
301     if (!res.ok) await handleApiError(res, 'Failed to fetch pending device types');
302     return res.json();
303   }
304
305   export async function approveDeviceType(id) {
306     const res = await fetchWithAuth(`${getApiBase()}/admin/device-types/${id}/approve/`, {
307       method: 'POST',
308       headers: { 'Content-Type': 'application/json' },
309     });
310     if (!res.ok) await handleApiError(res, 'Failed to approve device type');
311     return res.json();
312   }
313
314   export async function denyDeviceType(id, reason) {
315     const res = await fetchWithAuth(`${getApiBase()}/admin/device-types/${id}/deny/`, {
316       method: 'POST',
317       headers: { 'Content-Type': 'application/json' },
318       body: JSON.stringify({ reason }),
319     });
320     if (!res.ok) await handleApiError(res, 'Failed to deny device type');
321     return res.json();
322   }
323
324   export async function deleteDeviceType(id) {
325     const res = await fetchWithAuth(`${getApiBase()}/device-types/${id}/`, {
326       method: 'DELETE',
327     });
328     if (!res.ok) await handleApiError(res, 'Failed to delete device type');
329     return true;
330   }
331
332   // Denied Device Types (Admin)
333
334   export async function fetchDeniedDeviceTypes() {
335     const res = await fetchWithAuth(`${getApiBase()}/admin/device-types/denied/`);
336     if (!res.ok) await handleApiError(res, 'Failed to fetch denied device types');
337     return res.json();
338   }
339
340   export async function deleteDeniedDeviceType(id) {
341     const res = await fetchWithAuth(`${getApiBase()}/admin/device-types/denied/${id}/`, {
342       method: 'DELETE',
343     });
344     if (!res.ok) await handleApiError(res, 'Failed to delete denied device type');
345     return res.json();
346   }
347
348   export async function bulkDeleteDeniedDeviceTypes(ids = null) {
349     const options = {
350       method: 'DELETE',
351       headers: { 'Content-Type': 'application/json' },
352     };
353
354     // If ids provided, send them in body; otherwise delete all
355     if (ids && ids.length > 0) {
356       options.body = JSON.stringify({ ids });
357     }
358
359     const res = await fetchWithAuth(`${getApiBase()}/admin/device-types/denied/delete/`, options);
360     if (!res.ok) await handleApiError(res, 'Failed to delete denied device types');
361     return res.json();
362   }
363
364   // Admin Device Type Editing
365
366   export async function fetchAdminDeviceType(id) {
367     const res = await fetchWithAuth(`${getApiBase()}/admin/device-types/${id}/`);
368     if (!res.ok) await handleApiError(res, 'Failed to fetch device type');
369     return res.json();
370   }
371
372   export async function updateAdminDeviceType(id, data, partial = false) {
373     const res = await fetchWithAuth(`${getApiBase()}/admin/device-types/${id}/`, {

```

```

374     method: partial ? 'PATCH' : 'PUT',
375     headers: { 'Content-Type': 'application/json' },
376     body: JSON.stringify(data),
377   });
378   if (!res.ok) await handleApiError(res, 'Failed to update device type');
379   return res.json();
380 }
381
382 // User Device Type Proposal
383
384 export async function proposeDeviceType(data) {
385   const res = await fetchWithAuth(`${getApiBase()}/device-types/propose/`, {
386     method: 'POST',
387     headers: { 'Content-Type': 'application/json' },
388     body: JSON.stringify(data),
389   });
390   if (!res.ok) await handleApiError(res, 'Failed to propose device type');
391   return res.json();
392 }
393
394 // Devices
395
396 export async function fetchDevices() {
397   const res = await fetchWithAuth(`${getApiBase()}/devices/`);
398   if (!res.ok) await handleApiError(res, 'Failed to fetch devices');
399   return res.json();
400 }
401
402 export async function registerDevice(data) {
403   const res = await fetchWithAuth(`${getApiBase()}/devices/`, {
404     method: 'POST',
405     headers: { 'Content-Type': 'application/json' },
406     body: JSON.stringify(data),
407   });
408   if (!res.ok) await handleApiError(res, 'Failed to register device');
409   return res.json();
410 }
411
412 export async function updateDeviceState(id, data) {
413   const res = await fetchWithAuth(`${getApiBase()}/devices/${id}/state/`, {
414     method: 'PATCH',
415     headers: { 'Content-Type': 'application/json' },
416     body: JSON.stringify(data),
417   });
418   if (!res.ok) await handleApiError(res, 'Failed to update device state');
419   return res.json();
420 }
421
422 export async function updateDevice(id, data) {
423   const res = await fetchWithAuth(`${getApiBase()}/devices/${id}/`, {
424     method: 'PATCH',
425     headers: { 'Content-Type': 'application/json' },
426     body: JSON.stringify(data),
427   });
428   if (!res.ok) await handleApiError(res, 'Failed to update device');
429   return res.json();
430 }
431
432 export async function deleteDevice(id) {
433   const res = await fetchWithAuth(`${getApiBase()}/devices/${id}/`, {
434     method: 'DELETE',
435   });
436   if (!res.ok) await handleApiError(res, 'Failed to delete device');
437   return true;
438 }
439
440 export async function fetchTopology() {
441   const res = await fetchWithAuth(`${getApiBase()}/topology/`);
442   if (!res.ok) await handleApiError(res, 'Failed to fetch topology');
443   return res.json();
444 }
445
446 // Notifications API
447
448 export async function fetchNotifications(params = {}) {
449   const queryParams = new URLSearchParams();
450   if (params.is_read !== undefined) queryParams.append('is_read', params.is_read);
451   if (params.type) queryParams.append('type', params.type);

```

```

452     if (params.priority) queryParams.append('priority', params.priority);
453
454     const queryString = queryParams.toString();
455     const url = `${getApiBase()}/notifications/${queryString ? `?${queryString}` : ''}`;
456
457     const res = await fetchWithAuth(url);
458     if (!res.ok) await handleApiError(res, 'Failed to fetch notifications');
459     return res.json();
460 }
461
462 export async function fetchUnreadNotificationCount() {
463     const res = await fetchWithAuth(`${getApiBase()}/notifications/unread-count/`);
464     if (!res.ok) await handleApiError(res, 'Failed to fetch unread count');
465     return res.json();
466 }
467
468 export async function fetchNotification(id) {
469     const res = await fetchWithAuth(`${getApiBase()}/notifications/${id}/`);
470     if (!res.ok) await handleApiError(res, 'Failed to fetch notification');
471     return res.json();
472 }
473
474 export async function markNotificationAsRead(id) {
475     const res = await fetchWithAuth(`${getApiBase()}/notifications/${id}/read/`, {
476         method: 'POST',
477     });
478     if (!res.ok) await handleApiError(res, 'Failed to mark notification as read');
479     return res.json();
480 }
481
482 export async function markAllNotificationsAsRead(params = {}) {
483     const queryParams = new URLSearchParams();
484     if (params.type) queryParams.append('type', params.type);
485
486     const queryString = queryParams.toString();
487     const url = `${getApiBase()}/notifications/read-all/${queryString ? `?${queryString}` : ''}`;
488
489     const res = await fetchWithAuth(url, {
490         method: 'POST',
491         headers: { 'Content-Type': 'application/json' },
492         body: params.ids ? JSON.stringify({ ids: params.ids }) : undefined,
493     });
494     if (!res.ok) await handleApiError(res, 'Failed to mark all as read');
495     return res.json();
496 }
497
498 export async function deleteNotification(id) {
499     const res = await fetchWithAuth(`${getApiBase()}/notifications/${id}/`, {
500         method: 'DELETE',
501     });
502     if (!res.ok) await handleApiError(res, 'Failed to delete notification');
503     return true;
504 }
505
506 export async function bulkDeleteNotifications(params = {}) {
507     const res = await fetchWithAuth(`${getApiBase()}/notifications/bulk-delete/`, {
508         method: 'DELETE',
509         headers: { 'Content-Type': 'application/json' },
510         body: JSON.stringify(params),
511     });
512     if (!res.ok) await handleApiError(res, 'Failed to delete notifications');
513     return res.json();
514 }
515
516 // Dashboard Layout
517
518 export async function fetchDashboardLayout() {
519     const res = await fetchWithAuth(`${getApiBase()}/dashboard-layout/`);
520     if (!res.ok) await handleApiError(res, 'Failed to fetch dashboard layout');
521     return res.json();
522 }
523
524 export async function saveDashboardLayout(layout, deviceOrder) {
525     const body = { layout };
526     if (deviceOrder) body.device_order = deviceOrder;
527     const res = await fetchWithAuth(`${getApiBase()}/dashboard-layout/`, {
528         method: 'PUT',
529         headers: { 'Content-Type': 'application/json' },

```

```

530     body: JSON.stringify(body),
531   });
532   if (!res.ok) await handleApiError(res, 'Failed to save dashboard layout');
533   return res.json();
534 }
535
536 export async function deleteDashboardLayout() {
537   const res = await fetchWithAuth(`${getApiBase()}/dashboard-layout/`, {
538     method: 'DELETE',
539   });
540   if (!res.ok && res.status !== 204) await handleApiError(res, 'Failed to delete dashboard layout');
541   return true;
542 }
543
544 export async function fetchSharedDashboardLayout() {
545   const res = await fetchWithAuth(`${getApiBase()}/admin/dashboard-layout/`);
546   if (!res.ok) await handleApiError(res, 'Failed to fetch shared dashboard layout');
547   return res.json();
548 }
549
550 export async function saveSharedDashboardLayout(layout) {
551   const res = await fetchWithAuth(`${getApiBase()}/admin/dashboard-layout/`, {
552     method: 'PUT',
553     headers: { 'Content-Type': 'application/json' },
554     body: JSON.stringify({ layout }),
555   });
556   if (!res.ok) await handleApiError(res, 'Failed to save shared dashboard layout');
557   return res.json();
558 }
559
560 // Device Order
561
562 export async function fetchDeviceOrder() {
563   const res = await fetchWithAuth(`${getApiBase()}/device-order/`);
564   if (!res.ok) await handleApiError(res, 'Failed to fetch device order');
565   return res.json();
566 }
567
568 export async function updateDeviceOrder(deviceOrder) {
569   const res = await fetchWithAuth(`${getApiBase()}/device-order/`, {
570     method: 'PATCH',
571     headers: { 'Content-Type': 'application/json' },
572     body: JSON.stringify({ device_order: deviceOrder }),
573   });
574   if (!res.ok) await handleApiError(res, 'Failed to update device order');
575   return res.json();
576 }
577
578 // Device Collection
579
580 export async function fetchDeviceType(id) {
581   const res = await fetchWithAuth(`${getApiBase()}/device-types/${id}/`);
582   if (!res.ok) await handleApiError(res, 'Failed to fetch device type');
583   return res.json();
584 }
585
586 export async function uploadWiringDiagramImage(deviceTypeId, imageFile) {
587   const form = new FormData();
588   form.append('image', imageFile);
589   const res = await fetchWithAuth(`${getApiBase()}/device-types/${deviceTypeId}/wiring-image/`, {
590     method: 'POST',
591     body: form,
592   });
593   if (!res.ok) await handleApiError(res, 'Failed to upload wiring diagram image');
594   return res.json();
595 }
596
597 // Device Type Import/Export
598
599 export async function getImportDefaults() {
600   const res = await fetchWithAuth(`${getApiBase()}/device-types/import-defaults/`);
601   if (!res.ok) await handleApiError(res, 'Failed to fetch default device types');
602   return res.json();
603 }
604
605 export async function importDefaults() {
606   const res = await fetchWithAuth(`${getApiBase()}/device-types/import-defaults/`, {
607     method: 'POST',

```

```

608     });
609     if (!res.ok) await handleApiError(res, 'Failed to import default device types');
610     return res.json();
611 }
612
613 export async function importDeviceTypesFromFile(file) {
614     const form = new FormData();
615     form.append('file', file);
616     const res = await fetchWithAuth(`${getApiBase()}/device-types/import/`, {
617         method: 'POST',
618         body: form,
619     });
620     if (!res.ok) await handleApiError(res, 'Failed to import device types');
621     return res.json();
622 }
623
624 export async function exportDeviceTypes(ids = null) {
625     const params = ids && ids.length > 0 ? `?ids=${ids.join(',')}` : '';
626     const res = await fetchWithAuth(`${getApiBase()}/device-types/export/${params}`);
627     if (!res.ok) await handleApiError(res, 'Failed to export device types');
628     return res;
629 }
630
631 export async function exportSingleDeviceType(id) {
632     const res = await fetchWithAuth(`${getApiBase()}/device-types/${id}/export/`);
633     if (!res.ok) await handleApiError(res, 'Failed to export device type');
634     return res;
635 }
636
637 // Documentation Image Upload
638
639 export async function uploadDocumentationImage(imageFile, deviceTypeId = null) {
640     const form = new FormData();
641     form.append('image', imageFile);
642     if (deviceTypeId) {
643         form.append('device_type_id', deviceTypeId);
644     }
645     const res = await fetchWithAuth(`${getApiBase()}/device-types/doc-images/`, {
646         method: 'POST',
647         body: form,
648     });
649     if (!res.ok) await handleApiError(res, 'Failed to upload image');
650     return res.json();
651 }
652
653 // System Setup
654
655 export async function getSystemStatus() {
656     const res = await fetch(`${getApiBase()}/system-status/`);
657     if (!res.ok) {
658         // Fallback: if endpoint doesn't exist, assume system is configured
659         return { is_fresh: false };
660     }
661     return res.json();
662 }
663
664 export async function importSelectedDefaults(ids) {
665     const res = await fetchWithAuth(`${getApiBase()}/device-types/import-defaults/`, {
666         method: 'POST',
667         headers: { 'Content-Type': 'application/json' },
668         body: JSON.stringify({ ids }),
669     });
670     if (!res.ok) await handleApiError(res, 'Failed to import device types');
671     return res.json();
672 }

```

B.2 Cardul inteligent și optimistc UI (components/devices/SmartDeviceCard.tsx)

```

1  "use client"
2
3  import React, { useState, useEffect, useRef, useCallback } from 'react';
4  import { useQueryClient, useMutation } from '@tanstack/react-query';
5  import {
6      Cpu,

```

```

7     Power,
8     SlidersHorizontal,
9     Loader2,
10 } from 'lucide-react';
11 import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
12 import { Switch } from "@components/ui/switch";
13 import { Slider } from "@components/ui/slider";
14 import { Badge } from "@components/ui/badge";
15 import { getIconComponent } from "@lib/icons";
16 import { updateDeviceState } from "@lib/apiClient";
17 import { toast } from "sonner";
18 import { cn } from "@lib/utils";
19 import {
20     TemperatureWidget,
21     HumidityWidget,
22     PressureWidget,
23     MotionWidget,
24     LightWidget,
25     CO2Widget,
26     GenericSensorWidget
27 } from "../SensorWidgets";
28
29 interface Control {
30     widget_type: 'TOGGLE' | 'SLIDER' | 'GAUGE' | 'BUTTON' | 'TEMPERATURE' | 'HUMIDITY' | 'MOTION' | 'LIGHT' |
31     ↪ 'CO2' | 'PRESSURE' | 'POWER' | 'BATTERY' | 'STATUS';
32     label: string;
33     variable_mapping: string;
34     min_value?: number | null;
35     max_value?: number | null;
36     step?: number | null;
37     size?: 'sm' | 'md' | 'lg';
38     variant?: 'row' | 'square' | 'compact';
39     unit?: string;
40 }
41
42 interface LayoutConfig {
43     w?: number; // Backend format (width)
44     h?: number; // Backend format (height)
45     columns?: number; // Frontend format
46     gap?: 'sm' | 'md' | 'lg';
47 }
48
49 interface SmartDeviceCardProps {
50     device: any;
51     deviceType: any;
52     roomName?: string;
53     roomIcon?: string;
54     readOnly?: boolean;
55     animationIndex?: number;
56 }
57
58 // Check if this is a simple single-toggle device (ONLY has one toggle, nothing else)
59 function isSingleToggleDevice(controls: Control[]): boolean {
60     const toggles = controls.filter(c => c.widget_type === 'TOGGLE');
61     // A single-toggle device has exactly 1 toggle and no other controls at all
62     return toggles.length === 1 && controls.length === 1;
63 }
64
65 // Get all toggles for tap-to-toggle behavior
66 function getToggleControls(controls: Control[]): Control[] {
67     return controls.filter(c => c.widget_type === 'TOGGLE');
68 }
69
70 export default function SmartDeviceCard({ device, deviceType, roomName, roomIcon, readOnly = false,
71 ↪ animationIndex }: SmartDeviceCardProps) {
72     const [currentState, setCurrentState] = useState<any>(device.current_state || {});
73     // Use a local online state that can be optimistic
74     const [isOnline, setIsOnline] = useState(device.status === 'online' || device.is_online);
75     // Track if we are in a local optimistic state (either debouncing or mutating)
76     const [isDebouncing, setIsDebouncing] = useState(false);
77     // Track if we are waiting for a refetch after mutation
78     const [isRefetching, setIsRefetching] = useState(false);
79
80     const queryClient = useQueryClient();
81
82     const updateMutation = useMutation({
83         mutationFn: async ({ id, newState }: { id: string; newState: any }) => {
84             // Direct call, no artificial delays

```

```

83     return updateDeviceState(id, newState);
84 },
85 onMutate: async ({ id, newState }) => {
86     // Cancel any outgoing refetches (so they don't overwrite our optimistic update)
87     await queryClient.cancelQueries({ queryKey: ['devices'] });
88     await queryClient.cancelQueries({ queryKey: ['device', id] });
89
90     // Snapshot the previous value
91     const previousDevice = queryClient.getQueryData(['device', id]);
92
93     // Optimistically update the CACHE (not just local state)
94     queryClient.setQueryData(['device', id], (old: any) => ({
95         ...old,
96         current_state: newState
97     }));
98
99     return { previousDevice };
100 },
101 onError: (err, newTodo, context: any) => {
102     // Rollback to previous state on ERROR only
103     if (context?.previousDevice) {
104         queryClient.setQueryData(['device', context.previousDevice.id], context.previousDevice);
105         setCurrentState(context.previousDevice.current_state);
106     }
107     setIsRefetching(false);
108     toast.error("Failed to sync device state");
109 },
110 onSettled: () => {
111     // Mark that we are now waiting for the backend to reflect the change
112     setIsRefetching(true);
113     setTimeout(() => {
114         queryClient.invalidateQueries({ queryKey: ['devices'] });
115     }, 300);
116
117     // Safety timeout: stop the loading state after 5 seconds if the backend is stuck
118     setTimeout(() => {
119         setIsRefetching(false);
120     }, 5000);
121 },
122 });
123
124 // Calculate if we are in a syncing state
125 const isSyncing = isDebouncing || updateMutation.isPending || isRefetching;
126
127 // Sync with prop updates - BUT ONLY if we are not currently managing a local update
128 useEffect(() => {
129     // Always sync critical status flags regardless of state syncing
130     const serverOnline = device.status === 'online' || device.is_online;
131     if (serverOnline !== isOnline) {
132         setIsOnline(serverOnline);
133     }
134
135     // If not syncing, always accept server state
136     if (!isSyncing) {
137         setCurrentState(device.current_state || {});
138         return;
139     }
140
141     // If syncing, we usually ignore server state to avoid "jumping"
142     const serverState = device.current_state || {};
143
144     // We only care if the keys we modified match what is in the server state
145     const matchesOptimistic = Object.keys(currentState).length > 0 && Object.keys(currentState).every(key =>
146         String(currentState[key]) === String(serverState[key])
147     );
148
149     // Only clear the loading state if we are actually waiting for a refetch
150     if (matchesOptimistic && isRefetching) {
151         setCurrentState(serverState);
152         setIsRefetching(false);
153     }
154 }, [device, isSyncing, currentState, isRefetching]);
155
156 const Icon = (device.icon ? getIconComponent(device.icon) : null) || Cpu;
157 const cardTemplate = deviceType?.card_template;
158 const controls: Control[] = cardTemplate?.controls || [];
159 const layoutConfig: LayoutConfig = cardTemplate?.layout_config || {};
160

```

```

161 const singleToggle = isSingleToggleDevice(controls);
162 const toggleControls = getToggleControls(controls);
163
164 const mutateRef = useRef(updateMutation.mutate);
165 useEffect(() => { mutateRef.current = updateMutation.mutate; });
166
167 const timerRef = useRef<NodeJS.Timeout | null>(null);
168
169 useEffect(() => {
170   return () => {
171     if (timerRef.current) clearTimeout(timerRef.current);
172   }
173 }, []);
174
175 const handleStateChange = useCallback(async (variable: string, value: any, isImmediate = false) => {
176   // 1. Optimistic Update (Immediate UI change)
177   const newState = { ...currentState, [variable]: value };
178   setCurrentState(newState);
179
180   if (readOnly) return;
181
182   // Clear any existing timer to debounce
183   if (timerRef.current) clearTimeout(timerRef.current);
184
185   // Start local optimistic phase
186   setIsDebouncing(true);
187
188   // 2. Debounced API Call
189   // If "immediate" (like a toggle click), use a very short 50ms debounce
190   // If "slider" (dragging), use standard 500ms debounce
191   const delay = isImmediate ? 50 : 500;
192
193   timerRef.current = setTimeout(() => {
194     // Trigger mutation first so isPending becomes true roughly when we clear debouncing
195     mutateRef.current({ id: device.id, newState });
196     setTimeout(() => setIsDebouncing(false), 50);
197   }, delay);
198
199 }, [currentState, readOnly, device.id]);
200
201 // Handle card tap for single-toggle devices
202 const handleCardTap = useCallback(() => {
203   // Prevent interaction while syncing
204   if (!isOnline || readOnly || !singleToggle || isSyncing) return;
205
206   const toggle = toggleControls[0];
207   const currentValue = !!currentState[toggle.variable_mapping];
208   handleStateChange(toggle.variable_mapping, !currentValue, true);
209 }, [isOnline, readOnly, singleToggle, toggleControls, currentState, handleStateChange, isSyncing]);
210
211 // Determine if card should show "tap to toggle" behavior
212 const isTappable = singleToggle && isOnline && !readOnly && !isSyncing;
213
214 const canTap = singleToggle && isOnline && !readOnly;
215 const isPoweredOn = singleToggle && !!currentState[toggleControls[0]?.variable_mapping];
216
217 // Render individual widget based on type
218 const renderWidget = (control: Control, index: number) => {
219   const value = currentState[control.variable_mapping];
220   // Disable interactions while syncing to prevent race conditions
221   const isDisabled = !isOnline || readOnly || isSyncing;
222   const widgetVariant = (control.variant || 'row') as 'row' | 'square';
223
224   switch(control.widget_type) {
225     case 'TOGGLE':
226       // For single toggle devices, don't render the individual toggle (card is tappable)
227       if (singleToggle) return null;
228
229       const isOn = !!value;
230       const toggleColor = isOn ? 'text-green-500' : 'text-muted-foreground';
231       const toggleBg = isOn ? 'bg-green-500/10' : 'bg-muted/10';
232       const toggleBorder = isOn ? 'border-green-500/30' : 'border-border/50';
233
234       // Square variant for toggle
235       if (widgetVariant === 'square') {
236         return (
237           <div
238             key={control.variable_mapping}

```

```

239     className={cn(
240       "relative flex flex-col items-center justify-center p-2 rounded-xl border transition-all
        ↳ aspect-square overflow-hidden cursor-pointer",
241       toggleBg, toggleBorder,
242       isDisabled ? 'opacity-50' : 'hover:shadow-md active:scale-95',
243       isOn && 'shadow-[0_0_12px_-3px_rgba(34,197,94,0.3)]',
244       isSyncing && "animate-pulse border-primary/50"
245     )}
246     onClick={() => !isDisabled && handleStateChange(control.variable_mapping, !isOn, true)}
247   >
248     {isSyncing && (
249       <div className="absolute inset-0 flex items-center justify-center bg-background/20
        ↳ backdrop-blur-[1px] z-10">
250         <Loader2 className="w-5 h-5 animate-spin text-primary" />
251       </div>
252     )}
253     <Power className={cn("w-5 h-5 mb-1", toggleColor)} />
254     <span className={cn("text-lg font-bold leading-none", toggleColor)}>
255       {isOn ? 'ON' : 'OFF'}
256     </span>
257     <span className="text-[9px] text-muted-foreground uppercase tracking-wider leading-none mt-1
        ↳ text-center truncate w-full px-1">
258       {control.label}
259     </span>
260   </div>
261 );
262 }
263
264 // Row variant (default)
265 return (
266   <div
267     key={control.variable_mapping}
268     className={cn(
269       "flex items-center justify-between px-3 py-2 rounded-xl border transition-all h-14",
270       toggleBg, toggleBorder,
271       isDisabled ? 'opacity-50' : 'hover:shadow-md'
272     )}
273   >
274     <div className="flex items-center gap-3">
275       <div className={cn("p-1.5 rounded-lg", toggleBg)}>
276         <Power className={cn("w-4 h-4", toggleColor)} />
277       </div>
278       <span className="text-sm font-medium">{control.label}</span>
279     </div>
280     <Switch
281       checked={isOn}
282       onChange={checked => handleStateChange(control.variable_mapping, checked, true)}
283       disabled={isDisabled}
284     />
285   </div>
286 );
287
288 case 'SLIDER':
289   const min = control.min_value ?? 0;
290   const max = control.max_value ?? 100;
291   const step = control.step ?? 1;
292   const currentVal = typeof value === 'number' ? value : min;
293
294   return (
295     <div
296       key={control.variable_mapping}
297       className={cn(
298         "space-y-3 p-3 rounded-xl bg-card/50 border-border/50 transition-all",
299         isDisabled ? 'opacity-50' : 'hover:bg-card/80'
300       )}
301     >
302       <div className="flex items-center justify-between">
303         <div className="flex items-center gap-3">
304           <div className="p-2 rounded-lg bg-primary/10">
305             <SlidersHorizontal className="w-4 h-4 text-primary" />
306           </div>
307           <span className="text-sm font-medium">{control.label}</span>
308         </div>
309         <span className="text-sm font-mono font-semibold text-primary">{currentVal}</span>
310       </div>
311       <Slider
312         value={currentVal}
313         min={min}

```

```

314         max={max}
315         step={step}
316         onChange={vals => handleStateChange(control.variable_mapping, vals[0], false)}
317         disabled={isDisabled}
318     />
319 </div>
320 );
321
322 case 'TEMPERATURE':
323     return (
324         <TemperatureWidget
325             key={control.variable_mapping}
326             value={value}
327             label={control.label}
328             unit={control.unit}
329             isDisabled={isDisabled}
330             variant={widgetVariant}
331         />
332     );
333
334 case 'HUMIDITY':
335     return (
336         <HumidityWidget
337             key={control.variable_mapping}
338             value={value}
339             label={control.label}
340             unit={control.unit}
341             isDisabled={isDisabled}
342             variant={widgetVariant}
343         />
344     );
345
346 case 'MOTION':
347     return (
348         <MotionWidget
349             key={control.variable_mapping}
350             value={value}
351             label={control.label}
352             isDisabled={isDisabled}
353             variant={widgetVariant}
354         />
355     );
356
357 case 'LIGHT':
358     return (
359         <LightWidget
360             key={control.variable_mapping}
361             value={value}
362             label={control.label}
363             unit={control.unit}
364             isDisabled={isDisabled}
365             variant={widgetVariant}
366         />
367     );
368
369 case 'CO2':
370     return (
371         <CO2Widget
372             key={control.variable_mapping}
373             value={value}
374             label={control.label}
375             unit={control.unit}
376             isDisabled={isDisabled}
377             variant={widgetVariant}
378         />
379     );
380
381 case 'PRESSURE':
382     return (
383         <PressureWidget
384             key={control.variable_mapping}
385             value={value}
386             label={control.label}
387             unit={control.unit}
388             isDisabled={isDisabled}
389             variant={widgetVariant}
390         />
391     );

```

```

392
393     case 'GAUGE':
394     case 'POWER':
395     case 'BATTERY':
396     case 'STATUS':
397     default:
398         // Generic display for any sensor/gauge type without a dedicated widget.
399         return (
400             <GenericSensorWidget
401                 key={control.variable_mapping}
402                 value={value}
403                 label={control.label}
404                 unit={control.unit}
405                 isDisabled={isDisabled}
406                 variant={widgetVariant}
407             />
408         );
409     }
410 };
411
412 // Group controls by variant for layout
413 const squareControls = controls.filter(c => c.variant === 'square');
414 const rowControls = controls.filter(c => c.variant !== 'square');
415
416 // Determine grid columns (support both backend 'w' and frontend 'columns' format)
417 const getGridCols = () => {
418     const cols = layoutConfig.columns || layoutConfig.w || 2;
419     if (cols === 1) return 'grid-cols-1';
420     if (cols === 3) return 'grid-cols-3';
421     return 'grid-cols-2';
422 };
423
424 const getGapClass = () => {
425     const gap = layoutConfig.gap || 'md';
426     if (gap === 'sm') return 'gap-2';
427     if (gap === 'lg') return 'gap-4';
428     return 'gap-3';
429 };
430
431 return (
432     <Card
433         className={cn(
434             "group transition-all duration-300 relative overflow-hidden select-none",
435             isOnline
436             ? 'border-l-4 border-l-green-500 border-t-border border-r-border border-b-border
437               ↳ shadow-[0_0_20px_-5px_rgba(34,197,94,0.15)] hover:shadow-[0_0_25px_-5px_rgba(34,197,94,0.25)]
438               ↳ bg-card'
439             : 'border-l-4 border-l-zinc-400 dark:border-l-zinc-600 border-t-border/50 border-r-border/50
440               ↳ border-b-border/50 bg-muted/30 saturate-50 hover:saturate-100',
441             isTappable && 'cursor-pointer active:scale-[0.98]',
442             isTappable && isPoweredOn && 'shadow-[0_0_20px_-3px_rgba(245,158,11,0.4)]',
443             animationIndex !== undefined && 'animate-[jelly-in_0.5s_cubic-bezier(0.2,0.6,0.35,1)_both]'
444         )}
445         style={animationIndex !== undefined ? { animationDelay: `${animationIndex * 60}ms` } : undefined}
446         onClick={isTappable ? handleCardTap : undefined}
447     >
448         {/* Syncing Overlay - blurry loading state */}
449         {isSyncing && (
450             <div className="absolute inset-0 z-20 flex items-center justify-center bg-background/10
451               ↳ backdrop-blur-[2px] transition-all duration-300">
452                 <div className="bg-background/80 p-2 rounded-full shadow-lg ring-1 ring-border/20">
453                     <Loader2 className="w-5 h-5 animate-spin text-primary" />
454                 </div>
455             </div>
456         )}
457
458         {/* Offline overlay */}
459         {!isOnline && (
460             <div className="absolute inset-0 bg-background/40 backdrop-blur-[1px] z-10 pointer-events-none" />
461         )}
462
463         {/* Power glow for single-toggle devices */}
464         {isTappable && isPoweredOn && (
465             <div className="absolute inset-0 bg-gradient-to-br from-amber-500/5 to-transparent pointer-events-none"
466               ↳ />
467         )}
468     <CardHeader className="flex flex-row items-start justify-between space-y-0 pb-2">

```

```

465 <div className="flex flex-col gap-1 w-full overflow-hidden">
466   <div className="flex items-center gap-2">
467     <CardTitle className="text-base font-semibold truncate" title={device.name}>
468       {device.name}
469     </CardTitle>
470     /* Only show badge for non-single-toggle devices - single toggle has cleaner status */
471     {!singleToggle && (
472       <Badge
473         variant="secondary"
474         className={cn(
475           "text-[9px] h-4 px-1.5 py-0 font-medium shrink-0",
476           isOnline
477             ? 'bg-green-500/15 text-green-600 dark:text-green-400 border-green-500/30'
478             : 'bg-zinc-500/10 text-zinc-500 dark:text-zinc-400 border-zinc-500/20'
479         )}
480       >
481         {isOnline ? 'CONNECTED' : 'OFFLINE'}
482       </Badge>
483     )}
484   </div>
485   <div className="flex items-center gap-2">
486     {roomName && (
487       <p className="text-xs text-muted-foreground inline-flex items-center gap-1">
488         {roomIcon && (() => { const RI = getIconComponent(roomIcon); return RI ? <RI className="h-3 w-3"
489           ↪ /> : null; })()}
490         {roomName}
491       </p>
492     )}
493     {deviceType && (
494       <Badge variant="outline" className="text-[10px] h-4 px-1 py-0">{deviceType.name}</Badge>
495     )}
496   </div>
497 </CardHeader>
498
499 <CardContent className="space-y-4">
500   /* Single Toggle Device - Clean Home Assistant Style Layout */
501   {singleToggle ? (
502     <div className="flex flex-col items-center py-2">
503       /* Large centered icon with power state */
504       <div className={cn(
505         "relative p-5 rounded-2xl transition-all duration-300 mb-3",
506         isPoweredOn
507           ? 'bg-gradient-to-br from-amber-400/20 via-yellow-500/15 to-orange-500/10 ring-2
508           ↪ ring-amber-500/40 shadow-[0_0_30px_-5px_rgba(245,158,11,0.4)]'
509           : 'bg-secondary/50 ring-1 ring-border/50',
510       )}>
511         {isPoweredOn && (
512           <div className="absolute inset-0 rounded-2xl ring-2 ring-amber-400/30 animate-pulse" />
513         )}
514         <Icon className={cn(
515           "w-10 h-10 relative z-10 transition-colors duration-300",
516           isPoweredOn ? 'text-amber-500 dark:text-amber-400' : 'text-muted-foreground'
517         )} />
518       </div>
519       /* Power state label */
520       <div className={cn(
521         "text-2xl font-bold tracking-tight mb-1 transition-colors",
522         isPoweredOn ? 'text-amber-500 dark:text-amber-400' : 'text-muted-foreground'
523       )}>
524         {isPoweredOn ? 'ON' : 'OFF'}
525       </div>
526       /* Tap hint */
527       {canTap && (
528         <span className="text-[10px] text-muted-foreground uppercase tracking-wider">
529           Tap to {isPoweredOn ? 'turn off' : 'turn on'}
530         </span>
531       )}
532     </div>
533
534     /* Connection status - small and subtle at bottom */
535     <div className="flex items-center gap-1.5 mt-3 pt-3 border-t border-border/50 w-full
536     ↪ justify-center">
537       <div className={cn(
538         "h-1.5 w-1.5 rounded-full",
539         isOnline ? 'bg-green-500' : 'bg-zinc-400'
540       )} />

```

```

540     <span className={cn(
541         "text-[9px] uppercase tracking-wider",
542         isOnline ? 'text-green-600/70 dark:text-green-400/70' : 'text-muted-foreground/70'
543     )}>
544         {isOnline ? 'Connected' : 'Offline'}
545     </span>
546     {device.ip_address && (
547         <>
548             <span className="text-muted-foreground/30 mx-1">*</span>
549             <span className="text-[9px] font-mono text-muted-foreground/50">{device.ip_address}</span>
550         </>
551     )}
552 </div>
553 </div>
554 ) : (
555 <>
556     {/* Multi-control devices - Original Layout */}
557     {/* Top Section: Icon with status */}
558     <div className="flex items-center justify-between">
559         <div className={cn(
560             "relative p-3 rounded-xl transition-all duration-300",
561             isOnline
562             ? 'bg-gradient-to-br from-green-500/20 to-emerald-500/10 text-green-600 dark:text-green-400'
563             : 'ring-2 ring-green-500/30 shadow-[0_0_15px_-3px_rgba(34,197,94,0.4)]'
564             && 'bg-secondary/50 text-muted-foreground'
565         )}>
566             {isOnline && (
567                 <div className="absolute inset-0 rounded-xl ring-2 ring-green-500/40 animate-pulse" />
568             )}
569             <Icon className="w-6 h-6 relative z-10" />
570         </div>
571         <div className="text-right">
572             <div className="flex items-center gap-1.5 justify-end">
573                 <div className={cn(
574                     "h-2 w-2 rounded-full",
575                     isOnline ? 'bg-green-500 animate-pulse' : 'bg-zinc-400 dark:bg-zinc-600'
576                 )} />
577                 <p className={cn(
578                     "text-[10px] uppercase tracking-wider font-semibold",
579                     isOnline ? 'text-green-600 dark:text-green-400' : 'text-muted-foreground'
580                 )}>
581                     {isOnline ? 'Online' : 'Offline'}
582                 </p>
583             </div>
584             <p className="text-xs font-mono text-muted-foreground hidden md:block mt-0.5">
585                 {device.ip_address || 'N/A'}
586             </p>
587         </div>
588     </div>
589     {/* Dynamic Controls */}
590     {controls.length > 0 ? (
591         <div className={cn(
592             "pt-2 animate-in slide-in-from-bottom-2 duration-300",
593             !isOnline && 'pointer-events-none relative z-0'
594         )}>
595             {/* Square widgets in a grid */}
596             {squareControls.length > 0 && (
597                 <div className={cn("grid mb-3", getGridCols(), getGapClass())}>
598                     {squareControls.map((c, i) => renderWidget(c, i))}
599                 </div>
600             )}
601             {/* Row widgets stacked */}
602             {rowControls.length > 0 && (
603                 <div className="space-y-2">
604                     {rowControls.map((c, i) => renderWidget(c, i))}
605                 </div>
606             )}
607         </div>
608     ) : (
609         <div className="py-2 flex items-center justify-center text-xs text-muted-foreground/50 border"
610             && "border-dashed rounded bg-muted/20">
611             No controls configured
612         </div>
613     )}
614 </>
615

```

```

616     })
617     </CardContent>
618   </Card>
619 );
620 }

```

B.3 Hook-ul de layout al dashboard-ului (hooks/ useDashboardLayout.ts)

```

1  "use client"
2
3  import { useState, useCallback, useEffect, useRef } from 'react';
4  import { useQuery, useQueryClient } from '@tanstack/react-query';
5  import {
6    DashboardLayout,
7    GridItem,
8    GridFolderItem,
9    loadLayout,
10   saveLayout,
11   clearLayout,
12   buildDefaultLayout,
13   reconcileLayout,
14   generateFolderId,
15   getAllDeviceIds,
16 } from '@lib/dashboard-grid';
17 import {
18   fetchDashboardLayout,
19   saveDashboardLayout,
20   deleteDashboardLayout,
21   updateDeviceOrder,
22 } from '@lib/apiClient';
23
24 export type DeviceOrder = 'room' | 'type' | 'status' | 'name' | 'custom';
25
26 // manages dashboard grid layout: API/localStorage persistence, reconcile, folder mutations
27 export function useDashboardLayout(deviceIds: number[]) {
28   const queryClient = useQueryClient();
29   const [layout, setLayout] = useState<DashboardLayout | null>(null);
30   const [editMode, setEditModeRaw] = useState(false);
31   const [isSaving, setIsSaving] = useState(false);
32   const [deviceOrder, setDeviceOrderState] = useState<DeviceOrder>('custom');
33   const prevDeviceIdsRef = useRef<string>('');
34   const saveTimerRef = useRef<ReturnType<typeof setTimeout> | null>(null);
35   const initializedRef = useRef(false);
36   const layoutSetCount = useRef(0);
37   const layoutRef = useRef<DashboardLayout | null>(null);
38   const hasPendingSaveRef = useRef(false);
39   const skipNextPersistRef = useRef(false);
40
41   // Keep ref in sync with latest layout so flush/unmount can access it
42   useEffect(() => {
43     layoutRef.current = layout;
44   }, [layout]);
45
46   // Fetch layout from API
47   const { data: apiLayoutData, isSuccess: apiFetched, isError: apiFailed } = useQuery({
48     queryKey: ['dashboardLayout'],
49     queryFn: fetchDashboardLayout,
50     staleTime: Infinity, // never auto-refetch; we manage saves ourselves
51     refetchOnWindowFocus: false,
52     refetchOnMount: false,
53     retry: 1,
54   });
55
56   // Immediate save to API
57   const saveToApi = useCallback((layoutToSave: DashboardLayout) => {
58     hasPendingSaveRef.current = false;
59     setIsSaving(true);
60     saveDashboardLayout(layoutToSave)
61       .then(() => {
62         queryClient.setQueryData(['dashboardLayout'], { layout: layoutToSave, is_personal: true });
63       })
64       .catch(() => {})
65       .finally(() => setIsSaving(false));
66   }, [queryClient]);

```

```

67
68 // Debounced save to API + localStorage
69 const scheduleSave = useCallback((newLayout: DashboardLayout) => {
70   // Always mirror to localStorage immediately (offline cache)
71   saveLayout(newLayout);
72   // Debounce API call
73   if (saveTimerRef.current) clearTimeout(saveTimerRef.current);
74   hasPendingSaveRef.current = true;
75   saveTimerRef.current = setTimeout(() => {
76     saveToApi(newLayout);
77   }, 800);
78 }, [saveToApi]);
79
80 // Flush pending save immediately (cancel timer, save now)
81 const flushSave = useCallback(() => {
82   if (saveTimerRef.current) {
83     clearTimeout(saveTimerRef.current);
84     saveTimerRef.current = null;
85   }
86   const current = layoutRef.current;
87   if (current && hasPendingSaveRef.current) {
88     saveToApi(current);
89   }
90 }, [saveToApi]);
91
92 // On unmount, flush any pending save (fire-and-forget)
93 useEffect(() => {
94   return () => {
95     if (saveTimerRef.current) {
96       clearTimeout(saveTimerRef.current);
97       saveTimerRef.current = null;
98     }
99     if (hasPendingSaveRef.current && layoutRef.current) {
100       saveDashboardLayout(layoutRef.current).catch(() => {});
101       hasPendingSaveRef.current = false;
102     }
103   };
104 }, []);
105
106 // Controlled edit mode, flush save on "Done"
107 const setEditMode = useCallback((mode: boolean) => {
108   // Exiting edit mode -> flush any pending debounced save immediately
109   if (!mode) {
110     flushSave();
111   }
112   setEditModeRaw(mode);
113 }, [flushSave]);
114
115 // Initialize from API response (or localStorage fallback)
116 useEffect(() => {
117   if (initializedRef.current || deviceIds.length === 0) return;
118
119   if (apiFetched) {
120     initializedRef.current = true;
121     // Mark current device IDs so the reconcile effect below doesn't re-fire
122     prevDeviceIdsRef.current = deviceIds.slice().sort((a, b) => a - b).join(',');
123     // Extract device_order from API response
124     const apiOrder = apiLayoutData?.device_order as DeviceOrder | undefined;
125     if (apiOrder) setDeviceOrderState(apiOrder);
126     const apiLayout = apiLayoutData?.layout as DashboardLayout | null;
127     if (apiLayout && apiLayout.version === 1 && Array.isArray(apiLayout.items)) {
128       setLayout(reconcileLayout(apiLayout, deviceIds));
129     } else {
130       // No API layout, try localStorage, else build default
131       const saved = loadLayout();
132       if (saved) {
133         setLayout(reconcileLayout(saved, deviceIds));
134       } else {
135         setLayout(buildDefaultLayout(deviceIds));
136       }
137     }
138   } else if (apiFailed) {
139     // API unreachable, use localStorage offline
140     initializedRef.current = true;
141     prevDeviceIdsRef.current = deviceIds.slice().sort((a, b) => a - b).join(',');
142     const saved = loadLayout();
143     if (saved) {
144       setLayout(reconcileLayout(saved, deviceIds));

```

```

145     } else {
146         setLayout(buildDefaultLayout(deviceIds));
147     }
148 }
149 }, [apiFetched, apiFailed, apiLayoutData, deviceIds]);
150
151 // Reconcile when device list changes (after init)
152 useEffect(() => {
153     const key = deviceIds.slice().sort((a, b) => a - b).join(',');
154     if (key === prevDeviceIdsRef.current || deviceIds.length === 0) return;
155     prevDeviceIdsRef.current = key;
156
157     setLayout((prev) => {
158         if (!prev) return buildDefaultLayout(deviceIds);
159         const reconciled = reconcileLayout(prev, deviceIds);
160         // If new devices were added (reconciled has more device references), save immediately
161         // so the new device isn't lost on refresh
162         const prevIds = new Set(getAllDeviceIds(prev));
163         const newIds = getAllDeviceIds(reconciled);
164         const hasNewDevices = newIds.some((id) => !prevIds.has(id));
165         if (hasNewDevices) {
166             // Schedule an immediate save (not debounced) for new devices
167             saveLayout(reconciled);
168             saveDashboardLayout(reconciled).catch(() => {});
169         }
170         return reconciled;
171     });
172 }, [deviceIds]);
173
174 // Persist on change (skip initial set from init effect)
175 useEffect(() => {
176     if (!layout) return;
177     layoutSetCount.current++;
178     if (layoutSetCount.current <= 1) {
179         // First set is from init, just mirror to localStorage, don't save to API
180         saveLayout(layout);
181         return;
182     }
183     // Skip if flagged (reset/revert already handled their own API calls)
184     if (skipNextPersistRef.current) {
185         skipNextPersistRef.current = false;
186         saveLayout(layout); // still mirror to localStorage
187         return;
188     }
189     scheduleSave(layout);
190 }, [layout, scheduleSave]);
191
192 // Mutation helpers
193
194 /** Move item from one index to another */
195 const moveItem = useCallback((fromIndex: number, toIndex: number) => {
196     setLayout((prev) => {
197         if (!prev) return prev;
198         const items = [...prev.items];
199         const [moved] = items.splice(fromIndex, 1);
200         items.splice(toIndex, 0, moved);
201         return { ...prev, items };
202     });
203 }, []);
204
205 /** Create a folder from two device grid-items by dragging one onto another */
206 const createFolder = useCallback(
207     (deviceIdA: number, deviceIdB: number, name?: string) => {
208         setLayout((prev) => {
209             if (!prev) return prev;
210             const items = prev.items.filter(
211                 (it) =>
212                 !(it.type === 'device' && (it.deviceId === deviceIdA || it.deviceId === deviceIdB)),
213             );
214             const oldIndexA = prev.items.findIndex(
215                 (it) => it.type === 'device' && it.deviceId === deviceIdA,
216             );
217             const insertAt = Math.min(oldIndexA >= 0 ? oldIndexA : items.length, items.length);
218
219             const folder: GridFolderItem = {
220                 type: 'folder',
221                 folderId: generateFolderId(),
222                 name: name || 'New Folder',

```

```

223         deviceIds: [deviceIdA, deviceIdB],
224     };
225     items.splice(insertAt, 0, folder);
226     return { ...prev, items };
227 });
228 },
229 [],
230 );
231
232 /** Add a device into an existing folder (max 4) */
233 const addToFolder = useCallback((folderId: string, deviceId: number) => {
234     setLayout((prev) => {
235         if (!prev) return prev;
236         const items = prev.items
237             .filter((it) => !(it.type === 'device' && it.deviceId === deviceId))
238             .map((it) => {
239                 if (it.type === 'folder' && it.folderId === folderId) {
240                     if (it.deviceIds.length >= 4 || it.deviceIds.includes(deviceId)) return it;
241                     return { ...it, deviceIds: [...it.deviceIds, deviceId] };
242                 }
243                 return it;
244             });
245         return { ...prev, items };
246     });
247 }, []);
248
249 /** Remove a device from a folder (dissolves folder if <2 remain) */
250 const removeFromFolder = useCallback((folderId: string, deviceId: number) => {
251     setLayout((prev) => {
252         if (!prev) return prev;
253         const items: GridItem[] = [];
254         for (const it of prev.items) {
255             if (it.type === 'folder' && it.folderId === folderId) {
256                 const remaining = it.deviceIds.filter((id) => id !== deviceId);
257                 if (remaining.length >= 2) {
258                     items.push({ ...it, deviceIds: remaining });
259                 } else {
260                     remaining.forEach((id) => items.push({ type: 'device', deviceId: id }));
261                 }
262                 items.push({ type: 'device', deviceId });
263             } else {
264                 items.push(it);
265             }
266         }
267         return { ...prev, items };
268     });
269 }, []);
270
271 /** Rename a folder */
272 const renameFolder = useCallback((folderId: string, name: string) => {
273     setLayout((prev) => {
274         if (!prev) return prev;
275         const items = prev.items.map((it) =>
276             it.type === 'folder' && it.folderId === folderId ? { ...it, name } : it,
277         );
278         return { ...prev, items };
279     });
280 }, []);
281
282 /** Delete a folder entirely, returning its devices as standalone items */
283 const dissolveFolder = useCallback((folderId: string) => {
284     setLayout((prev) => {
285         if (!prev) return prev;
286         const items: GridItem[] = [];
287         for (const it of prev.items) {
288             if (it.type === 'folder' && it.folderId === folderId) {
289                 it.deviceIds.forEach((id) => items.push({ type: 'device', deviceId: id }));
290             } else {
291                 items.push(it);
292             }
293         }
294         return { ...prev, items };
295     });
296 }, []);
297
298 /** Reset to default layout, flat list, no folders, natural device order.
299  * Deletes the personal layout from the API to free server memory. */
300 const resetLayout = useCallback(() => {

```

```

301     clearLayout();
302     skipNextPersistRef.current = true; // prevent persist effect from re-saving
303     const defaultLayout = buildDefaultLayout(deviceIds);
304     setLayout(defaultLayout);
305     // Delete personal layout from API, the default is computed client-side
306     deleteDashboardLayout().catch(() => {});
307     queryClient.setQueryData(['dashboardLayout'], { layout: null, is_personal: false });
308   }, [deviceIds, queryClient]);
309
310   /** Revert to admin's shared layout (delete personal, re-fetch cascade) */
311   const revertToShared = useCallback(async () => {
312     try {
313       // Delete personal layout from API to free server memory
314       await deleteDashboardLayout().catch(() => {});
315       clearLayout();
316       skipNextPersistRef.current = true; // prevent persist effect from re-saving
317       // Re-fetch from the regular endpoint, it cascades: personal -> shared -> null
318       const res = await fetchDashboardLayout().catch(() => null);
319       const apiOrder = res?.device_order as DeviceOrder | undefined;
320       if (apiOrder) setDeviceOrderState(apiOrder);
321       const shared = res?.layout as DashboardLayout | null;
322       if (shared && shared.version === 1 && Array.isArray(shared.items)) {
323         const reconciled = reconcileLayout(shared, deviceIds);
324         setLayout(reconciled);
325         saveLayout(reconciled); // localStorage cache only
326         queryClient.setQueryData(['dashboardLayout'], { layout: shared, is_personal: false });
327       } else {
328         // No shared layout exists either, go to default
329         const defaultLayout = buildDefaultLayout(deviceIds);
330         setLayout(defaultLayout);
331         queryClient.setQueryData(['dashboardLayout'], { layout: null, is_personal: false });
332       }
333     } catch {
334       // Fallback to default if anything goes wrong
335       const defaultLayout = buildDefaultLayout(deviceIds);
336       setLayout(defaultLayout);
337     }
338   }, [deviceIds, queryClient]);
339
340   /** Update the device order/grouping preference and persist to API immediately */
341   const setDeviceOrder = useCallback((order: DeviceOrder) => {
342     setDeviceOrderState(order);
343     // Fire immediately, no debounce, so it persists even on instant refresh
344     updateDeviceOrder(order)
345       .then(() => {
346         // Update cached query data so the preference survives React Query reads
347         queryClient.setQueryData(['dashboardLayout'], (prev: any) => {
348           if (!prev) return { layout: null, device_order: order, is_personal: false };
349           return { ...prev, device_order: order };
350         });
351       })
352       .catch(() => {});
353   }, [queryClient]);
354
355   return {
356     layout,
357     editMode,
358     setEditMode,
359     moveItem,
360     createFolder,
361     addToFolder,
362     removeFromFolder,
363     renameFolder,
364     dissolveFolder,
365     resetLayout,
366     revertToShared,
367     isSaving,
368     deviceOrder,
369     setDeviceOrder,
370   };
371 }

```

B.4 Editorul vizual de tipuri de dispozitive (device-builder/ DeviceUICreator.tsx)

```
1
2 import React, { useState, useEffect } from 'react';
3 import { Node } from 'reactflow';
4 import {
5   DndContext,
6   closestCenter,
7   KeyboardSensor,
8   PointerSensor,
9   TouchSensor,
10  useSensor,
11  useSensors,
12  DragEndEvent,
13 } from '@dnd-kit/core';
14 import {
15   arrayMove,
16   SortableContext,
17   sortableKeyboardCoordinates,
18   useSortable,
19   verticalListSortingStrategy,
20   rectSortingStrategy,
21 } from '@dnd-kit/sortable';
22 import { CSS } from '@dnd-kit/utilities';
23 import {
24   ToggleLeft,
25   SlidersHorizontal,
26   Trash2,
27   Settings2,
28   ArrowLeft,
29   CheckCircle2,
30   Plus,
31   Thermometer,
32   Droplets,
33   Activity,
34   Sun,
35   Wind,
36   Gauge,
37   Wand2,
38   LayoutGrid,
39   Rows3,
40   GripVertical,
41 } from 'lucide-react';
42 import { Button } from "@/components/ui/button";
43 import { Input } from "@/components/ui/input";
44 import { Label } from "@/components/ui/label";
45 import { Card, CardContent, CardHeader, CardTitle } from "@/components/ui/card";
46 import {
47   Select,
48   SelectContent,
49   SelectItem,
50   SelectTrigger,
51   SelectValue,
52 } from "@/components/ui/select";
53 import { Switch } from "@/components/ui/switch";
54 import { Slider } from "@/components/ui/slider";
55 import { Badge } from "@/components/ui/badge";
56 import { Separator } from "@/components/ui/separator";
57 import {
58   Dialog,
59   DialogContent,
60   DialogDescription,
61   DialogFooter,
62   DialogHeader,
63   DialogTitle,
64 } from "@/components/ui/dialog";
65 import { ScrollArea } from "@/components/ui/scroll-area";
66 import { toast } from "sonner";
67 import { cn } from "@/lib/utls";
68
69 type WidgetType = 'TOGGLE' | 'SLIDER' | 'TEMPERATURE' | 'HUMIDITY' | 'PRESSURE' | 'MOTION' | 'LIGHT' | 'CO2';
70 type WidgetVariant = 'row' | 'square';
71 type WidgetSize = 'sm' | 'md' | 'lg';
72
73 interface Widget {
74   id: string;
```

```

75     type: WidgetType;
76     label: string;
77     variable_mapping: string;
78     min?: number;
79     max?: number;
80     step?: number;
81     variant?: WidgetVariant;
82     size?: WidgetSize;
83     unit?: string;
84 }
85
86 interface CardTemplateControl {
87     widget_type: WidgetType;
88     label: string;
89     variable_mapping: string;
90     min_value?: number | null;
91     max_value?: number | null;
92     step?: number | null;
93     variant?: WidgetVariant;
94     size?: WidgetSize;
95     unit?: string;
96 }
97
98 interface CardTemplate {
99     layout_config?: { columns?: number; gap?: string };
100     controls?: CardTemplateControl[];
101 }
102
103 interface DeviceUICreatorProps {
104     nodes: Node[];
105     onBack: () => void;
106     onSave: (template: any) => void;
107     isSubmitting: boolean;
108     initialCardTemplate?: CardTemplate | null;
109     editMode?: boolean;
110     reviewMode?: boolean;
111     deviceName?: string;
112     onNameChange?: (name: string) => void;
113 }
114
115 // Control widgets (for relays/switches)
116 const CONTROL_WIDGET_TYPES = [
117     { type: 'TOGGLE' as WidgetType, icon: ToggleLeft, label: 'Toggle Switch', defaultLabel: 'Power', category:
118       ↪ 'control' },
119     { type: 'SLIDER' as WidgetType, icon: SlidersHorizontal, label: 'Range Slider', defaultLabel: 'Level',
120       ↪ category: 'control' },
121 ];
122
123 // Sensor display widgets (read-only monitoring).
124 const SENSOR_WIDGET_TYPES = [
125     { type: 'TEMPERATURE' as WidgetType, icon: Thermometer, label: 'Temperature', defaultLabel: 'Temperature',
126       ↪ category: 'sensor', unit: ' degC' },
127     { type: 'HUMIDITY' as WidgetType, icon: Droplets, label: 'Humidity', defaultLabel: 'Humidity', category:
128       ↪ 'sensor', unit: '%' },
129     { type: 'PRESSURE' as WidgetType, icon: Gauge, label: 'Pressure', defaultLabel: 'Pressure', category:
130       ↪ 'sensor', unit: 'hPa' },
131     { type: 'MOTION' as WidgetType, icon: Activity, label: 'Motion Sensor', defaultLabel: 'Motion', category:
132       ↪ 'sensor', comingSoon: true },
133     { type: 'LIGHT' as WidgetType, icon: Sun, label: 'Light Sensor', defaultLabel: 'Light Level', category:
134       ↪ 'sensor', unit: 'lux', comingSoon: true },
135     { type: 'CO2' as WidgetType, icon: Wind, label: 'Air Quality', defaultLabel: 'CO2', category: 'sensor', unit:
136       ↪ 'ppm', comingSoon: true },
137 ];
138
139 const ALL_WIDGET_TYPES = [...CONTROL_WIDGET_TYPES, ...SENSOR_WIDGET_TYPES];
140
141 // Map node types to widget types for auto-generation.
142 const NODE_TO_WIDGET_MAP: Record<string, WidgetType> = {
143     'switch': 'TOGGLE',
144     'temperature': 'TEMPERATURE',
145     'humidity': 'HUMIDITY',
146     'pressure': 'PRESSURE',
147 };
148
149 // Get widget-specific colors
150 const getWidgetColors = (type: WidgetType) => {
151     switch(type) {
152         case 'TOGGLE': return { bg: 'bg-green-500/10', border: 'border-green-500/30', text: 'text-green-500' };

```

```

145     case 'SLIDER': return { bg: 'bg-primary/10', border: 'border-primary/30', text: 'text-primary' };
146     case 'TEMPERATURE': return { bg: 'bg-orange-500/10', border: 'border-orange-500/30', text: 'text-orange-500'
    ↪ };
147     case 'HUMIDITY': return { bg: 'bg-cyan-500/10', border: 'border-cyan-500/30', text: 'text-cyan-500' };
148     case 'MOTION': return { bg: 'bg-red-500/10', border: 'border-red-500/30', text: 'text-red-500' };
149     case 'LIGHT': return { bg: 'bg-yellow-500/10', border: 'border-yellow-500/30', text: 'text-yellow-500' };
150     case 'CO2': return { bg: 'bg-emerald-500/10', border: 'border-emerald-500/30', text: 'text-emerald-500' };
151     default: return { bg: 'bg-muted/10', border: 'border-border/50', text: 'text-muted-foreground' };
152   }
153 };
154
155 // Sortable Square Widget Component
156 interface SortableSquareWidgetProps {
157   widget: Widget;
158   isSelected: boolean;
159   onSelect: () => void;
160 }
161
162 function SortableSquareWidget({ widget, isSelected, onSelect }: SortableSquareWidgetProps) {
163   const {
164     attributes,
165     listeners,
166     setNodeRef,
167     transform,
168     transition,
169     isDragging,
170   } = useSortable({ id: widget.id });
171
172   const style = {
173     transform: CSS.Transform.toString(transform),
174     transition,
175     opacity: isDragging ? 0.5 : 1,
176     zIndex: isDragging ? 50 : 'auto',
177   };
178
179   const colors = getWidgetColors(widget.type);
180
181   // Size classes for grid spanning
182   const sizeClasses = {
183     sm: '',
184     md: '',
185     lg: 'col-span-2 row-span-2',
186   };
187
188   return (
189     <div
190       ref={setNodeRef}
191       style={style}
192       className={cn(
193         "group relative p-2 rounded-xl border transition-all cursor-pointer aspect-square overflow-hidden",
194         "flex flex-col items-center justify-center touch-manipulation",
195         isSelected
196           ? 'ring-2 ring-primary border-primary'
197           : `${colors.border} ${colors.bg} hover:shadow-md`,
198         isDragging && 'shadow-xl ring-2 ring-primary',
199         sizeClasses[widget.size] || 'md'
200       )}
201       onClick={onSelect}
202     >
203       {/* Drag handle */}
204       <div
205         {...attributes}
206         {...listeners}
207         className="absolute top-1 left-1 p-1 rounded bg-background/80 hover:bg-background border shadow-sm
    ↪   cursor-grab active:cursor-grabbing opacity-0 group-hover:opacity-100 transition-opacity z-10
    ↪   touch-manipulation"
208         onClick={(e) => e.stopPropagation()}
209       >
210         <GripVertical className="w-3 h-3 text-muted-foreground" />
211       </div>
212
213       {/* TOGGLE */}
214       {widget.type === 'TOGGLE' && (
215         <>
216           <ToggleLeft className={`w-5 h-5 ${colors.text} mb-1`} />
217           <span className={`text-lg font-bold leading-none ${colors.text}`}>ON</span>
218           <span className="text-[9px] text-muted-foreground uppercase tracking-wider leading-none mt-1
    ↪   text-center truncate w-full px-1">{widget.label}</span>

```

```

219     </>
220   )}
221
222   {/* SLIDER */
223   {widget.type === 'SLIDER' && (
224     <>
225     <SlidersHorizontal className={`w-4 h-4 ${colors.text} mb-1`} />
226     <span className={`text-lg font-bold leading-none ${colors.text}`}>50%</span>
227     <span className="text-[9px] text-muted-foreground uppercase tracking-wider leading-none mt-1
    ↪   text-center truncate w-full px-1">{widget.label}</span>
228   </>
229   )}
230
231   {/* Sensor icons */
232   {widget.type === 'TEMPERATURE' && <Thermometer className={`w-4 h-4 ${colors.text} mb-1`} />}
233   {widget.type === 'HUMIDITY' && <Droplets className={`w-4 h-4 ${colors.text} mb-1`} />}
234   {widget.type === 'PRESSURE' && <Gauge className={`w-4 h-4 ${colors.text} mb-1`} />}
235   {widget.type === 'MOTION' && <Activity className={`w-6 h-6 ${colors.text}`} />}
236   {widget.type === 'LIGHT' && <Sun className={`w-4 h-4 ${colors.text} mb-1`} />}
237   {widget.type === 'CO2' && <Wind className={`w-4 h-4 ${colors.text} mb-1`} />}
238
239   {/* Motion display */
240   {widget.type === 'MOTION' && (
241     <span className={`text-xs font-semibold uppercase tracking-wide mt-1 ${colors.text}`}>Clear</span>
242   )}
243
244   {/* Sensor values */
245   {(widget.type === 'TEMPERATURE' || widget.type === 'HUMIDITY' || widget.type === 'LIGHT' || widget.type
    ↪   === 'CO2') && (
246     <>
247     <div className="flex items-baseline">
248       <span className={`text-lg font-bold tabular-nums leading-none ${colors.text}`}>
249         {widget.type === 'TEMPERATURE' && '23.5'}
250         {widget.type === 'HUMIDITY' && '65'}
251         {widget.type === 'LIGHT' && '450'}
252         {widget.type === 'CO2' && '520'}
253       </span>
254       <span className="text-xs text-muted-foreground ml-0.5">
255         {widget.unit || (widget.type === 'TEMPERATURE' ? ' degC' : widget.type === 'HUMIDITY' ? '%' :
    ↪   widget.type === 'LIGHT' ? 'lux' : 'ppm')}
256       </span>
257     </div>
258     <span className="text-[9px] text-muted-foreground uppercase tracking-wider leading-none
    ↪   mt-1">{widget.label}</span>
259   </>
260   )}
261
262   {!widget.variable_mapping && (
263     <Badge variant="destructive" className="absolute top-1 right-1 text-[8px] px-1 py-0">!</Badge>
264   )}
265 </div>
266 );
267 }
268
269 // Sortable Row Widget Component
270 interface SortableRowWidgetProps {
271   widget: Widget;
272   isSelected: boolean;
273   onSelect: () => void;
274   mappedLabel?: string;
275 }
276
277 function SortableRowWidget({ widget, isSelected, onSelect, mappedLabel }: SortableRowWidgetProps) {
278   const {
279     attributes,
280     listeners,
281     setNodeRef,
282     transform,
283     transition,
284     isDragging,
285   } = useSortable({ id: widget.id });
286
287   const style = {
288     transform: CSS.Transform.toString(transform),
289     transition,
290     opacity: isDragging ? 0.5 : 1,
291     zIndex: isDragging ? 50 : 'auto',
292   };

```

```

293
294 // Size classes for row height
295 const sizeClasses = {
296   sm: 'h-12 py-1.5',
297   md: 'h-14 py-2',
298   lg: 'h-20 py-4',
299 };
300
301 return (
302   <div
303     ref={setNodeRef}
304     style={style}
305     className={cn(
306       "group relative px-3 rounded-xl border transition-all cursor-pointer touch-manipulation",
307       sizeClasses[widget.size || 'md'],
308       isSelected
309         ? 'border-primary ring-2 ring-primary bg-primary/5'
310         : 'border-border bg-card/40 hover:bg-card hover:border-primary/20 hover:shadow-md',
311       isDragging && 'shadow-xl'
312     )}
313     onClick={onSelect}
314   >
315     /* Drag handle */
316     <div
317       {...attributes}
318       {...listeners}
319       className="absolute left-1 top-1/2 -translate-y-1/2 p-1 rounded bg-background/80 hover:bg-background
320         ↳ border shadow-sm cursor-grab active:cursor-grabbing opacity-0 group-hover:opacity-100
321         ↳ transition-opacity z-10 touch-manipulation"
322       onClick={e => e.stopPropagation()}
323     >
324       <GripVertical className="w-3 h-3 text-muted-foreground" />
325     </div>
326
327     <div className="flex items-center justify-between gap-3 pl-6">
328       <div className="flex items-center gap-3 overflow-hidden">
329         <div className="p-2 rounded-lg bg-primary/10 text-primary">
330           {widget.type === 'TOGGLE' && <ToggleLeft className="w-5 h-5" />}
331           {widget.type === 'SLIDER' && <SlidersHorizontal className="w-5 h-5" />}
332           {widget.type === 'TEMPERATURE' && <Thermometer className="w-5 h-5 text-orange-500" />}
333           {widget.type === 'HUMIDITY' && <Droplets className="w-5 h-5 text-cyan-500" />}
334           {widget.type === 'PRESSURE' && <Gauge className="w-5 h-5 text-indigo-500" />}
335           {widget.type === 'MOTION' && <Activity className="w-5 h-5 text-red-500" />}
336           {widget.type === 'LIGHT' && <Sun className="w-5 h-5 text-yellow-500" />}
337           {widget.type === 'CO2' && <Wind className="w-5 h-5 text-gray-500" />}
338         </div>
339         <div className="flex flex-col min-w-0">
340           <span className="text-sm font-medium truncate">{widget.label}</span>
341           {!widget.variable_mapping ? (
342             <span className="text-[10px] text-destructive font-medium">Unmapped</span>
343           ) : (
344             <span className="text-[10px] text-muted-foreground truncate">{mappedLabel}</span>
345           )}
346         </div>
347       </div>
348
349       <div>
350         <div className="shrink-0">
351           {widget.type === 'TOGGLE' && <Switch checked={true} className="pointer-events-none" />}
352           {widget.type === 'TEMPERATURE' && <span className="text-sm font-bold text-orange-500">23.5
353             ↳ degC</span>}
354           {widget.type === 'HUMIDITY' && <span className="text-sm font-bold text-cyan-500">65%</span>}
355           {widget.type === 'MOTION' && <Badge variant="secondary" className="text-[10px]">Clear</Badge>}
356           {widget.type === 'LIGHT' && <span className="text-sm font-bold text-yellow-500">450 lux</span>}
357           {widget.type === 'CO2' && <span className="text-sm font-bold text-gray-500">520 ppm</span>}
358         </div>
359       </div>
360
361       {widget.type === 'SLIDER' && (
362         <div className="mt-4 px-1 pb-1 pointer-events-none">
363           <Slider defaultValue={[50]} max={100} step={1} className="w-full" />
364         </div>
365       )}
366     </div>
367   );
368 }
369
370 // Convert card_template controls to Widget format
371 function convertControlsToWidgets(controls: CardTemplateControl[]): Widget[] {

```

```

368 return controls.map((control, index) => ({
369   id: `widget-${Date.now()}-${index}`,
370   type: control.widget_type,
371   label: control.label,
372   variable_mapping: control.variable_mapping,
373   variant: control.variant || 'row',
374   size: control.size || 'md',
375   unit: control.unit,
376   ...(control.widget_type === 'SLIDER' && {
377     min: control.min_value ?? 0,
378     max: control.max_value ?? 100,
379     step: control.step ?? 1
380   })
381 }));
382 }
383
384 // Auto-generate widgets from nodes
385 function autoGenerateWidgets(nodes: Node[]): Widget[] {
386   const mappableNodes = nodes.filter(n => n.data.type !== 'mcu');
387   const widgets: Widget[] = [];
388
389   // Separate sensors and controls.
390   const sensorTypes = Object.keys(NODE_TO_WIDGET_MAP).filter(t => t !== 'switch');
391   const sensorNodes = mappableNodes.filter(n => sensorTypes.includes(n.data.type));
392   const controlNodes = mappableNodes.filter(n => n.data.type === 'switch');
393
394   // Detect related sensor pairs for layout grouping
395   const hasTemp = sensorNodes.some(n => n.data.type === 'temperature');
396   const hasHumidity = sensorNodes.some(n => n.data.type === 'humidity');
397   const hasTempHumidityPair = hasTemp && hasHumidity;
398
399   // Determine sensor layout strategy based on count
400   const sensorCount = sensorNodes.length;
401   let sensorVariant: WidgetVariant = 'square';
402   let defaultSensorSize: WidgetSize = 'md';
403
404   if (sensorCount === 1) {
405     sensorVariant = 'row';
406     defaultSensorSize = 'lg';
407   } else if (sensorCount === 2) {
408     sensorVariant = 'square';
409     defaultSensorSize = 'md';
410   } else if (sensorCount >= 3) {
411     sensorVariant = 'square';
412     defaultSensorSize = 'md';
413   }
414
415   // Add sensor widgets with intelligent sizing
416   sensorNodes.forEach((node, index) => {
417     const widgetType = NODE_TO_WIDGET_MAP[node.data.type];
418     const widgetDef = ALL_WIDGET_TYPES.find(w => w.type === widgetType);
419     if (widgetType && widgetDef) {
420       let size: WidgetSize = defaultSensorSize;
421
422       // Apply smart sizing rules
423       if (sensorCount === 1) {
424         size = 'lg';
425       } else if (sensorCount === 3 && index === 0) {
426         size = 'lg';
427       } else if (sensorCount >= 4) {
428         if (index === 0 && node.data.type !== 'motion') {
429           size = 'lg';
430         } else {
431           size = 'md';
432         }
433       }
434
435       if (node.data.type === 'motion') {
436         size = 'md';
437       }
438
439       if (hasTempHumidityPair && (node.data.type === 'temperature' || node.data.type === 'humidity')) {
440         size = sensorCount <= 2 ? 'md' : 'md';
441       }
442
443       widgets.push({
444         id: `widget-auto-${Date.now()}-${index}`,
445         type: widgetType,

```

```

446     label: node.data.label || widgetDef.defaultLabel,
447     variable_mapping: node.id,
448     variant: sensorVariant,
449     size,
450     unit: 'unit' in widgetDef ? (widgetDef as any).unit : undefined,
451   });
452 }
453 });
454
455 // Determine control layout strategy based on count
456 const controlCount = controlNodes.length;
457 let controlSize: WidgetSize = 'md';
458
459 if (controlCount === 1) {
460   controlSize = 'lg';
461 } else if (controlCount >= 4) {
462   controlSize = 'sm';
463 }
464
465 // Add control widgets (always row layout for easy toggling)
466 controlNodes.forEach((node, index) => {
467   widgets.push({
468     id: `widget-auto-ctrl-${Date.now()}-${index}`,
469     type: 'TOGGLE',
470     label: node.data.label || 'Power',
471     variable_mapping: node.id,
472     variant: 'row',
473     size: controlSize,
474   });
475 });
476
477 return widgets;
478 }
479
480 export default function DeviceUICreator({ nodes, onBack, onSave, isSubmitting, initialCardTemplate, editMode =
481   ↪ false, reviewMode = false, deviceName = '', onNameChange }: DeviceUICreatorProps) {
482   const [widgets, setWidgets] = useState<Widget[]>(() => {
483     // Initialize from card_template if provided
484     if (initialCardTemplate?.controls && initialCardTemplate.controls.length > 0) {
485       return convertControlsToWidgets(initialCardTemplate.controls);
486     }
487     return [];
488   });
489   const [layoutColumns, setLayoutColumns] = useState<number>(2);
490   const [selectedWidgetId, setSelectedWidgetId] = useState<string | null>(null);
491   const [isAddSheetOpen, setIsAddSheetOpen] = useState(false);
492
493   // Filter nodes suitable for mapping
494   const mappableNodes = nodes.filter(n => n.data.type !== 'mcu');
495
496   const selectedWidget = widgets.find(w => w.id === selectedWidgetId);
497
498   // Auto-generate widgets from topology
499   const handleAutoGenerate = () => {
500     const autoWidgets = autoGenerateWidgets(nodes);
501     if (autoWidgets.length === 0) {
502       toast.error("No mappable sensors found", { description: "Add sensors to your topology first." });
503       return;
504     }
505     setWidgets(autoWidgets);
506     toast.success(`Generated ${autoWidgets.length} widgets from topology`);
507   };
508
509   const addWidget = (type: WidgetType) => {
510     const def = ALL_WIDGET_TYPES.find(t => t.type === type);
511     if (!def) return;
512
513     const isSensor = SENSOR_WIDGET_TYPES.some(s => s.type === type);
514     const newWidget: Widget = {
515       id: `widget-${Date.now()}`,
516       type,
517       label: def.defaultLabel,
518       variable_mapping: '',
519       variant: isSensor ? 'square' : 'row',
520       size: 'md',
521       unit: 'unit' in def ? (def as any).unit : undefined,
522       ...(type === 'SLIDER' && { min: 0, max: 100, step: 1 })

```

```

523     setWidgets([...widgets, newWidget]);
524     toast.success(`Added ${def.label}`);
525   };
526
527   const updateWidget = (id: string, updates: Partial<Widget>) => {
528     setWidgets(widgets.map(w => w.id === id ? { ...w, ...updates } : w));
529   };
530
531   const removeWidget = (id: string) => {
532     setWidgets(widgets.filter(w => w.id !== id));
533     if (selectedWidgetId === id) setSelectedWidgetId(null);
534   };
535
536   // DnD sensors - supports mouse, touch, and keyboard
537   const sensors = useSensors(
538     useSensor(PointerSensor, {
539       activationConstraint: {
540         distance: 8,
541       },
542     }),
543     useSensor(TouchSensor, {
544       activationConstraint: {
545         delay: 200,
546         tolerance: 5,
547       },
548     }),
549     useSensor(KeyboardSensor, {
550       coordinateGetter: sortableKeyboardCoordinates,
551     })
552   );
553
554   const handleDragEnd = (event: DragEndEvent) => {
555     const { active, over } = event;
556
557     if (over && active.id !== over.id) {
558       setWidgets((items) => {
559         const oldIndex = items.findIndex((i) => i.id === active.id);
560         const newIndex = items.findIndex((i) => i.id === over.id);
561         return arrayMove(items, oldIndex, newIndex);
562       });
563     }
564   };
565
566   const handleSave = () => {
567     const unmappedWidgets = widgets.filter(w => !w.variable_mapping);
568     if (unmappedWidgets.length > 0) {
569       toast.error("Validation Failed", { description: "All widgets must be mapped to a variable." });
570       return;
571     }
572
573     // Only require switch nodes to have widgets (sensors are optional displays)
574     const requiredNodes = nodes.filter(n => n.data.type === 'switch');
575     const mappedIds = new Set(widgets.map(w => w.variable_mapping));
576     const missingNodes = requiredNodes.filter(n => !mappedIds.has(n.id));
577
578     if (missingNodes.length > 0) {
579       toast.error("Validation Failed", {
580         description: `You must add widgets for the following output components: ${missingNodes.map(n =>
581           ↪ n.data.label).join(', ')}`,
582       });
583       return;
584     }
585
586     // Build template - backend now supports all widget types directly
587     const template = {
588       layout_config: { w: layoutColumns, h: 2 },
589       controls: widgets.map(w => {
590         const control: any = {
591           widget_type: w.type,
592           label: w.label,
593           variable_mapping: w.variable_mapping,
594         };
595
596         // Add optional display config
597         if (w.variant && w.variant !== 'row') control.variant = w.variant;
598         if (w.size && w.size !== 'md') control.size = w.size;
599         if (w.unit) control.unit = w.unit;

```

```

600 // Slider-specific fields
601 if (w.type === 'SLIDER') {
602     control.min_value = w.min;
603     control.max_value = w.max;
604     control.step = w.step;
605 }
606
607 return control;
608 })
609 };
610 onSave(template);
611 };
612
613 return (
614     <div className="flex flex-col h-full bg-background">
615         {/ Header Toolbar */}
616         <div className="h-14 flex items-center justify-between px-3 md:px-6 border-b border-border bg-background
        ↳ backdrop-blur supports-[backdrop-filter]:bg-background/60 z-50 shrink-0">
617             <div className="flex items-center gap-2 md:gap-4 min-w-0">
618                 <Button variant="ghost" size="sm" onClick={onBack} className="-ml-2 text-muted-foreground
        ↳ hover:text-foreground shrink-0">
619                     <ArrowLeft className="w-4 h-4 mr-1 md:mr-2" />
620                     <span className="hidden sm:inline">Back: Topology</span>
621                     <span className="sm:hidden">Back</span>
622                 </Button>
623                 <div className="h-4 w-px bg-border hidden md:block" />
624                 {onNameChange && (
625                     <input
626                         value={deviceName}
627                         onChange={(e) => onNameChange(e.target.value)}
628                         className="h-7 w-28 md:w-48 px-2 rounded-md border border-border bg-muted/50 text-sm
        ↳ text-foreground placeholder:text-muted-foreground focus:outline-none focus:ring-1
        ↳ focus:ring-primary/50 truncate"
629                         placeholder="Device Name"
630                         readOnly={reviewMode}
631                     />
632                 )}
633             </div>
634             <div className="flex items-center gap-2 shrink-0">
635                 <Button
636                     onClick={handleSave}
637                     className="bg-primary hover:bg-primary/90 text-primary-foreground border-0 shadow-lg
        ↳ shadow-primary/20"
638                     size="sm"
639                     disabled={isSubmitting || widgets.length === 0}
640                 >
641                     <span className="hidden sm:inline">{isSubmitting ? "Submitting..." : "Next: Firmware"}</span>
642                     <span className="sm:hidden">{isSubmitting ? "...": "Next"}</span>
643                     <CheckCircle2 className="w-4 h-4 ml-1 md:ml-2" />
644                 </Button>
645             </div>
646         </div>
647
648         {/ Mobile Add Button */}
649         <div className="md:hidden fixed bottom-6 left-1/2 -translate-x-1/2 z-40 animate-in zoom-in duration-300">
650             <Button
651                 size="icon"
652                 className="h-14 w-14 rounded-full shadow-lg shadow-primary/30"
653                 onClick={() => setIsAddSheetOpen(true)}
654             >
655                 <Plus className="w-6 h-6" />
656             </Button>
657         </div>
658
659         <div className="flex flex-1 overflow-hidden relative md:p-4 bg-background/20">
660             <div className="flex w-full h-full md:rounded-2xl overflow-hidden md:border md:border-border
        ↳ bg-background shadow-2xl relative">
661
662                 {/ Left Panel: Toolbox */}
663                 <div className="w-80 border-r border-border p-4 hidden md:flex flex-col bg-card/30 backdrop-blur-xl
        ↳ overflow-y-auto">
664                     {/ Auto-Generate Button */}
665                     <Button
666                         variant="default"
667                         className="w-full mb-4 bg-gradient-to-r from-primary to-primary/80"
668                         onClick={handleAutoGenerate}
669                     >
670                         <Wand2 className="w-4 h-4 mr-2" />

```

```

671     Auto-Generate from Topology
672   </Button>
673
674   <Separator className="mb-4" />
675
676   {/Control Widgets /}
677   <h3 className="text-xs font-semibold mb-3 text-muted-foreground uppercase tracking-wider">Control
678   ↪ Widgets</h3>
679   <div className="grid gap-2 mb-4">
680     {CONTROL_WIDGET_TYPES.map(t => (
681       <Button
682         key={t.type}
683         variant="outline"
684         className="h-auto py-3 px-3 justify-start gap-3 border-dashed border hover:border-primary/50
685         ↪ hover:bg-primary/5 transition-all group"
686         onClick={() => addWidget(t.type)}
687       >
688         <div className="p-1.5 rounded-md bg-muted group-hover:bg-primary/10 transition-colors">
689           <t.icon className="w-4 h-4 text-muted-foreground group-hover:text-primary" />
690           </div>
691           <span className="font-medium text-sm">{t.label}</span>
692         </Button>
693       )})
694   </div>
695
696   {/Sensor Widgets /}
697   <h3 className="text-xs font-semibold mb-3 text-muted-foreground uppercase tracking-wider">Sensor
698   ↪ Displays</h3>
699   <div className="grid grid-cols-2 gap-2 mb-4">
700     {SENSOR_WIDGET_TYPES.map(t => (
701       t.comingSoon ? (
702         <div
703           key={t.type}
704           aria-disabled
705           title="Coming soon"
706           className="relative h-auto py-3 px-2 flex flex-col items-center justify-center gap-1
707           ↪ border-dashed border rounded-md bg-muted/30 opacity-60 cursor-not-allowed select-none"
708         >
709           <span className="absolute top-1 right-1 text-[8px] font-semibold uppercase tracking-wider
710           ↪ text-muted-foreground bg-muted px-1 py-0.5 rounded">
711             Soon
712           </span>
713           <t.icon className="w-5 h-5 text-muted-foreground" />
714           <span className="font-medium text-[10px] text-muted-foreground">{t.label}</span>
715         </div>
716       ) : (
717         <Button
718           key={t.type}
719           variant="outline"
720           className="h-auto py-3 px-2 flex-col gap-1 border-dashed border hover:border-primary/50
721           ↪ hover:bg-primary/5 transition-all group"
722           onClick={() => addWidget(t.type)}
723         >
724           <t.icon className="w-5 h-5 text-muted-foreground group-hover:text-primary" />
725           <span className="font-medium text-[10px]">{t.label}</span>
726         </Button>
727       )
728     )})
729   </div>
730
731   <Separator className="my-4" />
732
733   {/Layout Options /}
734   <h3 className="text-xs font-semibold mb-3 text-muted-foreground uppercase tracking-wider">Grid
735   ↪ Columns</h3>
736   <div className="flex gap-2 mb-4">
737     {[1, 2].map(cols => (
738       <Button
739         key={cols}
740         variant={layoutColumns === cols ? "default" : "outline"}
741         size="sm"
742         className="flex-1"
743         onClick={() => setLayoutColumns(cols)}
744       >
745         {cols}
746       </Button>
747     )})
748   </div>

```

```

742 <Separator className="my-4" />
743
744
745 <h3 className="text-xs font-semibold mb-3 text-muted-foreground uppercase tracking-wider">Variable
↪ Status</h3>
746 <ScrollArea className="flex-1 -mx-2 px-2">
747   <div className="space-y-2">
748     {mappableNodes.length === 0 ? (
749       <p className="text-xs text-muted-foreground italic">No sensors available.</p>
750     ) : (
751       mappableNodes.map(node => {
752         const mappedCount = widgets.filter(w => w.variable_mapping === node.id).length;
753         const isMapped = mappedCount > 0;
754         return (
755           <div
756             key={node.id}
757             className={`
758               flex items-center justify-between p-2 rounded-lg border text-xs
759               ↪ transition-colors
760               ${isMapped ? 'bg-primary/5 border-primary/20' : 'bg-muted/30
761               ↪ border-transparent'}
762             `}
763             >
764               <div className="flex items-center gap-2 overflow-hidden">
765                 <div className={`w-1.5 h-1.5 rounded-full ${isMapped ? 'bg-primary' :
766                 ↪ 'bg-muted-foreground/50'} `} />
767                 <span className="font-medium truncate">{node.data.label}</span>
768               </div>
769               {isMapped && <Badge variant="secondary" className="text-[9px] h-4
770               ↪ px-1.5">{mappedCount}</Badge>}
771             </div>
772           </div>
773         );
774       })
775     )}
776   </div>
777 </ScrollArea>
778 </div>
779
780 {/* Center Panel: Canvas */}
781 <div className="flex-1 bg-background relative overflow-hidden flex flex-col items-center justify-center
↪ p-8">
782   <div className="absolute inset-0 bg-[radial-gradient(#e5e7eb_1px,transparent_1px)]
↪ dark:bg-[radial-gradient(#333_1px,transparent_1px)] [background-size:24px_24px] opacity-[0.4]
↪ pointer-events-none" />
783
784   <div className="w-full max-w-[450px] space-y-8 animate-in zoom-in-95 duration-300">
785     <div className="text-center space-y-1">
786       <h2 className="text-lg font-semibold tracking-tight">UI Preview</h2>
787       <p className="text-sm text-muted-foreground">Configure the controls for your device.</p>
788     </div>
789
790     {/* Smart Device Card Preview */}
791     <Card className="overflow-hidden border border-border/50 shadow-xl bg-card/60 backdrop-blur-xl">
792       <CardHeader className="flex flex-row items-center justify-between space-y-0 p-4 pb-2">
793         <div className="space-y-1">
794           <CardTitle className="text-base font-semibold">Device Name</CardTitle>
795           <div className="flex items-center gap-2">
796             <span className="flex h-2 w-2 rounded-full bg-green-500
797             ↪ shadow-[0_0_8px_rgba(34,197,94,0.6)]" />
798             <span className="text-xs text-muted-foreground font-medium">Online</span>
799           </div>
800         </div>
801         <div className="p-2 rounded-full bg-muted/50">
802           <Settings2 className="h-4 w-4 text-muted-foreground" />
803         </div>
804       </CardHeader>
805
806       <CardContent className="p-4 space-y-3">
807         {widgets.length === 0 ? (
808           <div className="flex flex-col items-center justify-center py-12
809           ↪ text-muted-foreground border border-dashed rounded-lg bg-muted/20">
810             <Wand2 className="w-8 h-8 mb-3 opacity-30" />
811             <span className="text-sm font-medium">No Controls Added</span>
812             <span className="text-xs opacity-60">Click "Auto-Generate" or add items
813             ↪ manually</span>
814           </div>
815         ) : (
816           <DndContext

```

```

809         sensors={sensors}
810         collisionDetection={closestCenter}
811         onDragEnd={handleDragEnd}
812     >
813     {/* Square widgets in a grid */}
814     {widgets.filter(w => w.variant === 'square').length > 0 && (
815         <SortableContext
816             items={widgets.filter(w => w.variant === 'square').map(w => w.id)}
817             strategy={rectSortingStrategy}
818         >
819             <div className={`grid gap-2 ${layoutColumns === 1 ? 'grid-cols-1' :
820                 ↪ 'grid-cols-2'}`}>
821                 {widgets.filter(w => w.variant === 'square').map((widget) => (
822                     <SortableSquareWidget
823                         key={widget.id}
824                         widget={widget}
825                         isSelected={selectedWidgetId === widget.id}
826                         onSelect={() => setSelectedWidgetId(widget.id)}
827                     >
828                 </div>
829             </SortableContext>
830         )}
831
832     {/* Row widgets stacked */}
833     {widgets.filter(w => w.variant === 'row').length > 0 && (
834         <SortableContext
835             items={widgets.filter(w => w.variant === 'row').map(w => w.id)}
836             strategy={verticalListSortingStrategy}
837         >
838             <div className="space-y-2">
839                 {widgets.filter(w => w.variant === 'row').map((widget) => (
840                     <SortableRowWidget
841                         key={widget.id}
842                         widget={widget}
843                         isSelected={selectedWidgetId === widget.id}
844                         onSelect={() => setSelectedWidgetId(widget.id)}
845                         mappedLabel={nodes.find(n => n.id ===
846                             ↪ widget.variable_mapping)?.data.label}
847                     >
848                 </div>
849             </SortableContext>
850         )}
851     </DndContext>
852 )}
853 </CardContent>
854 </Card>
855 </div>
856 </div>
857 </div>
858 </div>
859
860 {/* Mobile Add Sheet (Custom Implementation) */}
861 <div className={`fixed inset-0 z-[60] md:hidden pointer-events-none`}>
862     <div
863         className={`absolute inset-0 bg-black/60 backdrop-blur-sm transition-opacity duration-300
864             ↪ ${isAddSheetOpen ? 'opacity-100 pointer-events-auto' : 'opacity-0'}`}
865         onClick={() => setIsAddSheetOpen(false)}
866     >
867     <div className={`
868         absolute bottom-0 left-0 right-0
869         bg-card border-t border-border rounded-t-3xl
870         transition-transform duration-300 ease-out pointer-events-auto
871         ${isAddSheetOpen ? 'translate-y-0' : 'translate-y-full'}
872         shadow-2xl pb-8 max-h-[70vh] overflow-y-auto
873     `}>
874         <div className="p-6">
875             <div className="w-12 h-1.5 bg-muted rounded-full mx-auto mb-6" />
876
877             {/* Auto Generate Button */}
878             <Button
879                 variant="default"
880                 className="w-full mb-6 bg-gradient-to-r from-primary to-primary/80"
881                 onClick={() => {
882                     handleAutoGenerate();
883                     setIsAddSheetOpen(false);

```

```

884     }}
885   >
886     <Wand2 className="w-4 h-4 mr-2" />
887     Auto-Generate from Topology
888   </Button>
889
890   {/* Control Widgets */}
891   <h4 className="text-xs font-semibold text-muted-foreground uppercase tracking-wider
892   ↳ mb-3">Controls</h4>
893   <div className="grid grid-cols-4 gap-3 mb-6">
894     {CONTROL_WIDGET_TYPES.map((t) => (
895       <div
896         key={t.type}
897         className="flex flex-col items-center gap-2 group active:scale-95 transition-transform"
898         onClick={() => {
899           addWidget(t.type);
900           setIsAddSheetOpen(false);
901         }}
902       >
903         <div className="w-14 h-14 rounded-xl flex items-center justify-center border border-border
904         ↳ bg-card/60 shadow-sm">
905           <t.icon className="w-6 h-6 text-primary" />
906         </div>
907         <span className="text-[10px] font-medium text-muted-foreground text-center leading-tight">
908           {t.label}
909         </span>
910       </div>
911     ))}
912   </div>
913
914   {/* Sensor Widgets */}
915   <h4 className="text-xs font-semibold text-muted-foreground uppercase tracking-wider
916   ↳ mb-3">Sensors</h4>
917   <div className="grid grid-cols-5 gap-3">
918     {SENSOR_WIDGET_TYPES.map((t) => (
919       <div
920         key={t.type}
921         className="flex flex-col items-center gap-2 group active:scale-95 transition-transform"
922         onClick={() => {
923           addWidget(t.type);
924           setIsAddSheetOpen(false);
925         }}
926       >
927         <div className="w-12 h-12 rounded-xl flex items-center justify-center border border-border
928         ↳ bg-card/60 shadow-sm">
929           <t.icon className="w-5 h-5 text-muted-foreground" />
930         </div>
931         <span className="text-[9px] font-medium text-muted-foreground text-center leading-tight">
932           {t.label}
933         </span>
934       </div>
935     ))}
936   </div>
937
938   {/* Configuration Dialog (Popup) */}
939   <Dialog open={!selectedWidgetId} onOpenChange={(open) => !open && setSelectedWidgetId(null)}>
940     <DialogContent className="sm:max-w-[400px]">
941       <DialogHeader>
942         <DialogTitle>Configure Widget</DialogTitle>
943         <DialogDescription>
944           Adjust settings for this interface element.
945         </DialogDescription>
946       </DialogHeader>
947       {selectedWidget && (
948         <div className="grid gap-6 py-4">
949           <div className="grid gap-2">
950             <Label>Label</Label>
951             <Input
952               value={selectedWidget.label}
953               onChange={(e) => updateWidget(selectedWidget.id, { label: e.target.value })}
954               placeholder="e.g. Living Room Light"
955             />
956           </div>
957         </div>
958       )}
959     </DialogContent>
960   </Dialog>

```

```

958 <div className="grid gap-2">
959   <Label>Connected Variable</Label>
960   <Select
961     value={selectedWidget.variable_mapping}
962     onChange={val => updateWidget(selectedWidget.id, { variable_mapping: val })}
963   >
964     <SelectTrigger>
965       <SelectValue placeholder="Select a variable..." />
966     </SelectTrigger>
967     <SelectContent>
968       {mappableNodes.map(n => (
969         <SelectItem key={n.id} value={n.id}>
970           <div className="flex items-center gap-2">
971             <div className={`w-2 h-2 rounded-full ${n.data.color} ||
972               ↵ `).replace('text-', 'bg-').split(' ')[0]}` />
973             <span className="font-medium">{n.data.label}</span>
974           </div>
975         </SelectItem>
976       )
977     )}
978   </SelectContent>
979 </Select>
980 <p className="text-[11px] text-muted-foreground">The sensor or actuator this controls.</p>
981 </div>
982
983 {/ Layout Options */}
984 <div className="grid grid-cols-2 gap-4 pt-2 border-t mt-2">
985   <div className="space-y-2">
986     <Label className="text-xs">Layout Style</Label>
987     <Select
988       value={selectedWidget.variant || 'row'}
989       onChange={val => updateWidget(selectedWidget.id, { variant: val as
990         ↵ WidgetVariant })}
991     >
992       <SelectTrigger className="h-9">
993         <SelectValue />
994       </SelectTrigger>
995       <SelectContent>
996         <SelectItem value="row">
997           <div className="flex items-center gap-2">
998             <Rows3 className="w-3 h-3" />
999             <span>Row</span>
1000           </div>
1001         </SelectItem>
1002         <SelectItem value="square">
1003           <div className="flex items-center gap-2">
1004             <LayoutGrid className="w-3 h-3" />
1005             <span>Square</span>
1006           </div>
1007         </SelectItem>
1008       </SelectContent>
1009     </Select>
1010   </div>
1011   <div className="space-y-2">
1012     <Label className="text-xs">Size</Label>
1013     <Select
1014       value={selectedWidget.size || 'md'}
1015       onChange={val => updateWidget(selectedWidget.id, { size: val as WidgetSize
1016         ↵ })}
1017     >
1018       <SelectTrigger className="h-9">
1019         <SelectValue />
1020       </SelectTrigger>
1021       <SelectContent>
1022         <SelectItem value="sm">Small</SelectItem>
1023         <SelectItem value="md">Medium</SelectItem>
1024         <SelectItem value="lg">Large</SelectItem>
1025       </SelectContent>
1026     </Select>
1027   </div>
1028 </div>
1029
1030 {/ Unit (for sensors) */}
1031 {SENSOR_WIDGET_TYPES.some(s => s.type === selectedWidget.type) && (
1032   <div className="grid gap-2">
1033     <Label>Unit</Label>
1034     <Input
1035       value={selectedWidget.unit || ''}

```

```

1033         onChange={(e) => updateWidget(selectedWidget.id, { unit: e.target.value })}
1034         placeholder="e.g. degC, %, lux"
1035     />
1036 </div>
1037 })
1038
1039 {selectedWidget.type === 'SLIDER' && (
1040     <div className="space-y-4 pt-2 border-t mt-2">
1041         <Label>Range</Label>
1042         <div className="grid grid-cols-3 gap-3">
1043             <div className="space-y-1">
1044                 <span className="text-[10px] text-muted-foreground uppercase">Min</span>
1045                 <Input
1046                     type="number"
1047                     className="h-8"
1048                     value={selectedWidget.min}
1049                     onChange={(e) => updateWidget(selectedWidget.id, { min:
1050                         ↪ Number(e.target.value) })}
1051                 />
1052             </div>
1053             <div className="space-y-1">
1054                 <span className="text-[10px] text-muted-foreground uppercase">Max</span>
1055                 <Input
1056                     type="number"
1057                     className="h-8"
1058                     value={selectedWidget.max}
1059                     onChange={(e) => updateWidget(selectedWidget.id, { max:
1060                         ↪ Number(e.target.value) })}
1061                 />
1062             </div>
1063             <div className="space-y-1">
1064                 <span className="text-[10px] text-muted-foreground uppercase">Step</span>
1065                 <Input
1066                     type="number"
1067                     className="h-8"
1068                     value={selectedWidget.step}
1069                     onChange={(e) => updateWidget(selectedWidget.id, { step:
1070                         ↪ Number(e.target.value) })}
1071                 />
1072             </div>
1073         </div>
1074     </div>
1075 )}
1076 <DialogFooter className="flex-row sm:justify-between items-center bg-muted/50 -mx-6 -mb-6 p-6 mt-2
1077     ↪ border-t">
1078     <Button
1079         variant="ghost"
1080         size="sm"
1081         className="text-destructive hover:text-destructive hover:bg-destructive/10"
1082         onClick={() => selectedWidgetId && removeWidget(selectedWidgetId)}
1083     >
1084         <Trash2 className="w-4 h-4 mr-2" /> Delete
1085     </Button>
1086     <Button size="sm" onClick={() => setSelectedWidgetId(null)}>
1087         Done
1088     </Button>
1089 </DialogFooter>
1090 </DialogContent>
1091 </Dialog>
1092 </div>
1093 );
}

```